

CERTIFICATION PREP • NVIDIA AI NETWORKING

NCP-AIN Certification Guide

NVIDIA AI Networking with Spectrum-X and InfiniBand

Design • Configure • Monitor • Troubleshoot • Automate

Vakeesan Thevarajah

CCIE #43898

Complete Edition — Chapters 1-23, Three Mock Exams and Appendices

September 2026

Cloudfoxy Ltd

NCP-AIN Certification Guide: NVIDIA AI Networking with Spectrum-X and InfiniBand

First edition, 2026

Copyright © 2026 Vakeesan Thevarajah. All rights reserved.

Published by **Cloudfoxy Ltd**, United Kingdom.

No part of this book may be reproduced, stored or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews.

ISBN 978-1-0654464-0-8 (paperback, Volume I)

ISBN 978-1-0654464-1-5 (paperback, Volume II)

ISBN 978-1-0654464-2-2 (ebook)

Independent publication. This is an independent study guide published by Cloudfoxy Ltd. It is not produced, sponsored, endorsed or reviewed by NVIDIA Corporation or by any exam provider. NVIDIA, the NVIDIA logo, Spectrum, Spectrum-X, Quantum, ConnectX, BlueField, DOCA, NVLink, NVSwitch, DGX, HGX, UFM, NetQ, Cumulus, SHARP and Air are trademarks or registered trademarks of NVIDIA Corporation in the United States and other countries. Other names are the property of their respective owners and are used here only to identify the products discussed.

Exam content. All practice questions, mock exams and scenarios in this book are original and written from NVIDIA's public exam blueprint and public documentation. They are not actual exam questions, and no confidential exam material was used.

Software versions. Commands and defaults were checked against NVIDIA's public documentation in September 2026, principally Cumulus Linux 5.12–5.18, NetQ 4.x–5.x, UFM Enterprise 6.x, MLNX-OS 3.12, DOCA 3.x and NVIDIA Network Operator 26.x. Version notes in the text flag behaviour that differs between releases.

How this book was produced. Portions of this book were drafted with the assistance of generative AI tools. Technical content was checked against NVIDIA's public documentation, and the author is responsible for its content.

Limit of liability. The information in this book is provided as is, without warranty. Always confirm configurations against current vendor documentation and test changes in a lab (such as NVIDIA Air) before applying them to production networks. The author accepts no liability for loss or damage arising from the use of this information.

Errata and updates. Corrections, updated commands and companion files (flashcards, lab files) are published at <https://cloudfoxy-ltd.github.io/ncp-ain-guide/>.

*For Shaminah,
my wife, my partner and my constant encouragement.*

*And for Aarush and Isha,
who fill every day with joy, laughter and endless happiness.*

Contents

About This Book.....	17
Who this book is for.....	17
About the NCP-AIN exam.....	17
The official exam blueprint.....	18
How each chapter is built.....	19
Errata, updates and companion files.....	19
Callouts used in this book.....	19
Visual legend.....	19
Study plans.....	20
Book Plan and Blueprint Map.....	22
The learning arc.....	22
Table of contents and chapter plan.....	22
Blueprint coverage matrix.....	23
Tool coverage matrix.....	24
The Helix AI Factory: Running Example and Visual Kit.....	26
Helix at a glance.....	26
Naming and addressing plan.....	29
Chapter 1 — AI Factory Networking Architecture.....	32
Exam map.....	32
"Do I know this already?" pre-quiz.....	32
Opening problem: why Helix's first cluster underperformed.....	32
1.1 What an AI factory is, and why its network is different.....	33
1.2 The separate networks of an AI factory.....	35
1.3 Inside a GPU node.....	37
1.4 NICs, DPUs and SuperNICs.....	38
1.5 Switches: choosing the right family for each network.....	39
1.6 Scalable units: building AI factories from repeatable blocks.....	42
1.7 The management and software plane.....	43
1.8 Putting it together: the Helix AI Factory.....	45
Hands-on lab 1 — Network inventory of a GPU node (reference walkthrough).....	45
Troubleshooting scenario — "One node makes every job slow".....	48
Chapter quick reference.....	50
Chapter FAQ.....	50
Chapter summary.....	51
Further reading.....	52
Figure index.....	53
Chapter 1 — Q&A Pack.....	54
A. Objective map.....	54
B. Exam-style question bank.....	54
C. Deep-understanding set.....	62
D. Interview and design-defence questions.....	62
E. Flashcards.....	62
F. Cross-domain preview questions.....	63
G. Challenge lab — design Helix-C.....	64
H. Answer key.....	65
Chapter 2 — GPU-to-GPU Communication, NCCL and Rail-Optimized Topologies.....	70
Exam map.....	70
"Do I know this already?" pre-quiz.....	70
Opening problem: the spines are hot but the leaves are cool.....	70
2.1 Scale-up and scale-out.....	71
2.2 GPU-to-GPU data paths.....	73
2.3 Collective operations and parallelism.....	74
2.4 NCCL: how GPUs actually communicate.....	75
2.5 Rail-optimized topologies.....	77

2.6 Fat-trees, bisection bandwidth and scaling.....	81
2.7 NCCL settings every AI network engineer should know.....	82
2.8 Validating rails and GPU communication.....	83
Hands-on lab 2 — Trace an all-reduce across the rails (reference walkthrough).....	84
Troubleshooting scenario — "Inter-node all-reduce is only a third of expected".....	86
Chapter quick reference.....	88
Chapter FAQ.....	88
Chapter summary.....	89
Further reading.....	90
Figure index.....	91
Chapter 2 — Q&A Pack.....	92
A. Objective map.....	92
B. Exam-style question bank.....	92
C. Deep-understanding set.....	100
D. Interview and design-defence questions.....	100
E. Flashcards.....	100
F. Cumulative questions (Chapters 1–2).....	101
G. Challenge lab — prove the rails on Helix-B.....	102
H. Answer key.....	103
Chapter 3 — RDMA Fundamentals: Verbs, InfiniBand Transport and RoCEv2.....	107
Exam map.....	107
"Do I know this already?" pre-quiz.....	107
Opening problem: "Why can't we just use TCP?".....	107
3.1 Why RDMA.....	107
3.2 RDMA transports: InfiniBand, RoCE and iWARP.....	109
3.3 The verbs object model.....	109
3.4 RDMA operations.....	111
3.5 Connecting queue pairs.....	112
3.6 InfiniBand packet format.....	114
3.7 RoCEv2 packet format.....	114
3.8 Addressing: LIDs, GUIDs and GIDs.....	116
3.9 Reliability, ordering and why the fabric must (nearly) never drop.....	116
3.10 The host RDMA stack and tools.....	117
Hands-on lab 3 — Your first RDMA connection with Soft-RoCE.....	118
Troubleshooting scenario — "ib_write_bw works on some Helix-B nodes but not others".....	120
Chapter quick reference.....	122
Chapter FAQ.....	122
Chapter summary.....	123
Further reading.....	124
Figure index.....	125
Chapter 3 — Q&A Pack.....	126
A. Objective map.....	126
B. Exam-style question bank.....	126
C. Deep-understanding set.....	134
D. Interview and design-defence questions.....	134
E. Flashcards.....	134
F. Cumulative questions (Chapters 1–3).....	135
G. Challenge lab — build and break Soft-RoCE.....	135
H. Answer key.....	136
Chapter 4 — Cumulus Linux and NVUE Essentials.....	141
Exam map.....	141
"Do I know this already?" pre-quiz.....	141
Opening problem: "Who changed leaf-r3?".....	141
4.1 Cumulus Linux in one page.....	141
4.2 The NVUE model.....	144
4.3 Pending, applied and startup revisions.....	146

4.4 Configuring the basics with NVUE.....	148
4.5 The NVUE REST API.....	150
4.6 NVUE (Cumulus Linux 5.x) versus NCLU (Cumulus Linux 4.x).....	151
Hands-on lab 4 — Build the Helix-B underlay in NVIDIA Air.....	151
Troubleshooting scenario — "My change came back after the reboot".....	153
Chapter quick reference.....	155
Chapter FAQ.....	155
Chapter summary.....	156
Further reading.....	157
Figure index.....	158
Chapter 4 — Q&A Pack.....	159
A. Objective map.....	159
B. Exam-style question bank.....	159
C. Deep-understanding set.....	166
D. Interview and design-defence questions.....	166
E. Flashcards.....	166
F. Cumulative questions (Chapters 1–4).....	167
G. Challenge lab — build and roll back.....	167
H. Answer key.....	169
Chapter 5 — Lossless RoCE: QoS, PFC and ECN.....	172
Exam map.....	172
"Do I know this already?" pre-quiz.....	172
Opening problem: a 1-in-10,000 loss rate halves training speed.....	172
5.1 The switch QoS pipeline.....	173
5.2 Configuring RoCE on Cumulus Linux.....	174
5.3 Priority Flow Control (PFC).....	176
5.4 ECN and DCQCN.....	179
5.5 The host side: matching the NIC to the fabric.....	181
5.6 Lossless or lossy? RoCE on Spectrum-X.....	182
5.7 Rolling out RoCE QoS safely.....	183
Hands-on lab 5 — Enable and verify RoCE QoS.....	184
Troubleshooting scenario — "RoCE is configured, but we still see drops".....	187
Chapter quick reference.....	188
Chapter FAQ.....	188
Chapter summary.....	189
Further reading.....	190
Figure index.....	191
Chapter 5 — Q&A Pack.....	192
A. Objective map.....	192
B. Exam-style question bank.....	192
C. Deep-understanding set.....	200
D. Interview and design-defence questions.....	200
E. Flashcards.....	200
F. Cumulative questions (Chapters 1–5).....	201
G. Challenge lab — prove the RoCE profile end to end.....	201
H. Answer key.....	203
Chapter 6 — Spectrum-X Adaptive Routing, Congestion Control and Telemetry.....	207
Exam map.....	207
"Do I know this already?" pre-quiz.....	207
Opening problem: four spines, two of them hot.....	207
6.1 Why ECMP is not enough.....	207
6.2 Spectrum-X adaptive routing.....	209
6.3 Telemetry-based congestion control.....	210
6.4 Performance isolation.....	211
6.5 Telemetry on Spectrum switches.....	212
6.6 ECMP settings that still matter.....	216

Hands-on lab 6 — Enable adaptive routing and HFT.....	216
Troubleshooting scenario — "AR is on, but the uplinks are still unbalanced".....	218
Chapter quick reference.....	220
Chapter FAQ.....	220
Chapter summary.....	221
Further reading.....	222
Figure index.....	223
Chapter 6 — Q&A Pack.....	224
A. Objective map.....	224
B. Exam-style question bank.....	224
C. Deep-understanding set.....	231
D. Interview and design-defence questions.....	231
E. Flashcards.....	231
F. Cumulative questions (Chapters 1–6).....	232
G. Challenge lab — prove AR's value.....	232
H. Answer key.....	233
Chapter 7 — Multi-Tenancy with BGP-EVPN.....	235
Exam map.....	235
"Do I know this already?" pre-quiz.....	235
Opening problem: two tenants, one fabric.....	235
7.1 Overlay and underlay.....	235
7.2 EVPN: the control plane.....	238
7.3 L2 VNIs, L3 VNIs and symmetric routing.....	239
7.4 Configuring Helix-B tenants with NVUE.....	241
7.5 RoCE, QoS and adaptive routing over VXLAN.....	242
7.6 Verification.....	243
Hands-on lab 7 — Build AURORA and BOREALIS in NVIDIA Air.....	243
Troubleshooting scenario — "AURORA hosts on leaf-r4 can't reach the rest of AURORA".....	245
Chapter quick reference.....	246
Chapter FAQ.....	246
Chapter summary.....	247
Further reading.....	248
Figure index.....	249
Chapter 7 — Q&A Pack.....	250
A. Objective map.....	250
B. Exam-style question bank.....	250
C. Deep-understanding set.....	257
D. Interview and design-defence questions.....	257
E. Flashcards.....	257
F. Cumulative questions (Chapters 1–7).....	258
G. Challenge lab — add a shared-services VRF.....	258
H. Answer key.....	259
Chapter 8 — BlueField, SuperNICs and DOCA.....	261
Exam map.....	261
"Do I know this already?" pre-quiz.....	261
Opening problem: the "NIC" that wouldn't pass traffic.....	261
8.1 The NVIDIA adapter family.....	261
8.2 Inside BlueField-3 and its operating modes.....	262
8.3 DOCA: the software framework.....	265
8.4 Installing DOCA.....	268
8.5 SuperNIC functionality for Spectrum-X.....	270
Hands-on lab 8 — Install DOCA and set the SuperNIC mode (reference walkthrough).....	272
Troubleshooting scenario — "Compute port is up, but RDMA doesn't work".....	274
Chapter quick reference.....	276
Chapter FAQ.....	276
Chapter summary.....	277

Further reading.....	278
Figure index.....	279
Chapter 8 — Q&A Pack.....	280
A. Objective map.....	280
B. Exam-style question bank.....	280
C. Deep-understanding set.....	284
D. Interview and design-defence questions.....	284
E. Flashcards.....	284
F. Cumulative questions (Chapters 1–8).....	285
G. Challenge lab — acceptance script.....	285
H. Answer key.....	286
Chapter 9 — Lab Simulation with NVIDIA Air.....	288
Exam map.....	288
"Do I know this already?" pre-quiz.....	288
Opening problem: "We only find out in production".....	288
9.1 What NVIDIA Air is (and isn't).....	288
9.2 Building a simulation.....	290
9.3 Access and the OOB management network.....	292
9.4 Simulation management.....	293
9.5 Air in the Helix change process.....	294
Hands-on lab 9 — Build the Helix-B twin from a file.....	294
Troubleshooting scenario — "It worked in Air but failed in production".....	296
Chapter quick reference.....	298
Chapter FAQ.....	298
Chapter summary.....	299
Further reading.....	300
Figure index.....	301
Chapter 9 — Q&A Pack.....	302
A. Objective map.....	302
B. Exam-style question bank.....	302
C. Deep-understanding set.....	306
D. Interview and design-defence questions.....	306
E. Flashcards.....	306
F. Cumulative questions (Chapters 1–9).....	307
G. Challenge lab — twin from JSON.....	307
H. Answer key.....	307
Chapter 10 — Monitoring: NetQ, WJH and In-Band Telemetry.....	309
Exam map.....	309
"Do I know this already?" pre-quiz.....	309
Opening problem: "Something drops packets — somewhere".....	309
10.1 NetQ architecture.....	309
10.2 What Just Happened (WJH).....	311
10.3 Diagnosing congestion and packet loss.....	315
10.4 Latency measurement and congestion detection in NetQ.....	317
Hands-on lab 10 — Fabric-wide monitoring with NetQ.....	319
Troubleshooting scenario — "Intermittent RoCE retransmissions for one tenant".....	321
Chapter quick reference.....	323
Chapter FAQ.....	323
Chapter summary.....	324
Further reading.....	325
Figure index.....	326
Chapter 10 — Q&A Pack.....	327
A. Objective map.....	327
B. Exam-style question bank.....	327
C. Deep-understanding set.....	331
D. Interview and design-defence questions.....	331

E. Flashcards.....	331
F. Cumulative questions (Chapters 1–10).....	332
G. Challenge lab — build an alerting pack.....	332
H. Answer key.....	333
Chapter 11 — InfiniBand Architecture and the Subnet Manager.....	336
Exam map.....	336
"Do I know this already?" pre-quiz.....	336
Opening problem: the fabric that configures itself.....	336
11.1 InfiniBand building blocks.....	336
11.2 What the Subnet Manager does.....	338
11.3 Port states.....	339
11.4 Where the SM runs, and SM high availability.....	342
11.5 Routing engines.....	343
11.6 Discovery and inspection commands (preview).....	344
Hands-on lab 11 — Read an InfiniBand subnet (reference walkthrough).....	344
Troubleshooting scenario — "New nodes stuck in Initializing".....	346
Chapter quick reference.....	348
Chapter FAQ.....	348
Chapter summary.....	349
Further reading.....	350
Figure index.....	351
Chapter 11 — Q&A Pack.....	352
A. Objective map.....	352
B. Exam-style question bank.....	352
C. Deep-understanding set.....	355
D. Interview and design-defence questions.....	355
E. Flashcards.....	355
F. Cumulative questions (Chapters 1–11).....	356
G. Challenge lab — SM failover drill.....	356
H. Answer key.....	357
Chapter 12 — Initial Configuration, Provisioning and High Availability.....	359
Exam map.....	359
"Do I know this already?" pre-quiz.....	359
Opening problem: the bring-up that took three weeks.....	359
12.1 Bring-up order.....	360
12.2 Initial configuration of a Quantum switch (MLNX-OS).....	361
12.3 Installing UFM Enterprise.....	361
12.4 Configuring the Subnet Manager in UFM.....	362
12.5 UFM high availability.....	363
12.6 Validating before handover.....	364
Hands-on lab 12 — Build and fail over a UFM HA pair (reference walkthrough).....	365
Troubleshooting scenario — "After failover, training got slower".....	367
Chapter quick reference.....	369
Chapter FAQ.....	369
Chapter summary.....	370
Further reading.....	371
Figure index.....	372
Chapter 12 — Q&A Pack.....	373
A. Objective map.....	373
B. Exam-style question bank.....	373
C. Deep-understanding set.....	376
D. Interview and design-defence questions.....	376
E. Flashcards.....	376
F. Cumulative questions (Chapters 1–12).....	377
G. Challenge lab — HA acceptance test.....	377
H. Answer key.....	378

Chapter 13 — Partition Keys and Secure Multi-Tenancy.....	380
Exam map.....	380
"Do I know this already?" pre-quiz.....	380
Opening problem: research must not see production.....	380
13.1 How partitions work.....	380
13.2 Where PKeys are enforced.....	382
13.3 Configuring partitions in OpenSM (partitions.conf).....	382
13.4 Managing PKeys with UFM.....	384
13.5 The host side.....	384
13.6 Multi-tenancy on InfiniBand versus Ethernet.....	385
Hands-on lab 13 — Build RESEARCH, PRODUCTION and SHARED partitions (reference walkthrough).....	385
Troubleshooting scenario — "New research nodes can't reach storage".....	386
Chapter quick reference.....	388
Chapter FAQ.....	388
Chapter summary.....	389
Further reading.....	390
Figure index.....	391
Chapter 13 — Q&A Pack.....	392
A. Objective map.....	392
B. Exam-style question bank.....	392
C. Deep-understanding set.....	396
D. Interview and design-defence questions.....	396
E. Flashcards.....	396
F. Cumulative questions (Chapters 1–13).....	397
G. Challenge lab — partition audit.....	397
H. Answer key.....	397
Chapter 14 — InfiniBand QoS, Adaptive Routing and SHARP.....	399
Exam map.....	399
"Do I know this already?" pre-quiz.....	399
Opening problem: faster collectives without more links.....	399
14.1 InfiniBand QoS: SL, VL and arbitration.....	399
14.2 Congestion control on InfiniBand.....	401
14.3 Adaptive routing on InfiniBand.....	402
14.4 SHARP: in-network computing.....	404
Hands-on lab 14 — QoS, AR and SHARP on a lab fabric (reference walkthrough).....	405
Troubleshooting scenario — "SHARP enabled, no improvement".....	407
Chapter quick reference.....	409
Chapter FAQ.....	409
Chapter summary.....	410
Further reading.....	411
Figure index.....	412
Chapter 14 — Q&A Pack.....	413
A. Objective map.....	413
B. Exam-style question bank.....	413
C. Deep-understanding set.....	416
D. Interview and design-defence questions.....	416
E. Flashcards.....	416
F. Cumulative questions (Chapters 1–14).....	417
G. Challenge lab — design and measure.....	417
H. Answer key.....	417
Chapter 15 — Monitoring with UFM.....	419
Exam map.....	419
"Do I know this already?" pre-quiz.....	419
Opening problem: one slow rail in 512 GPUs.....	419
15.1 UFM Enterprise monitoring capabilities.....	419
15.2 Monitoring link status.....	420

15.3 Monitoring bandwidth utilisation with UFM Telemetry.....	423
15.4 The counters you must understand.....	424
15.5 Events, alarms and thresholds.....	425
15.6 The UFM REST API.....	425
Hands-on lab 15 — Monitor Helix-A with UFM (reference walkthrough).....	426
Troubleshooting scenario — "Training 8% slower, nothing is down".....	428
Chapter quick reference.....	430
Chapter FAQ.....	430
Chapter summary.....	431
Further reading.....	432
Figure index.....	433
Chapter 15 — Q&A Pack.....	434
A. Objective map.....	434
B. Exam-style question bank.....	434
C. Deep-understanding set.....	437
D. Interview and design-defence questions.....	437
E. Flashcards.....	437
F. Cumulative questions (Chapters 1–15).....	438
G. Challenge lab — degraded-link report.....	438
H. Answer key.....	439
Chapter 16 — NVIDIA Network Operator: Deploy and Verify.....	442
Exam map.....	442
"Do I know this already?" pre-quiz.....	442
Opening problem: GPUs in pods, but slow networking.....	442
16.1 What the Network Operator does.....	442
16.2 Deploying the operator.....	444
16.3 InfiniBand specifics: partitions for pods.....	446
16.4 GPUDirect RDMA in Kubernetes.....	447
16.5 Verifying the Network Operator (objective 4.2).....	448
Hands-on lab 16 — Deploy and verify on a lab cluster.....	450
Troubleshooting scenario — "RDMA pods stuck in Pending".....	451
Chapter quick reference.....	453
Chapter FAQ.....	453
Chapter summary.....	454
Further reading.....	455
Figure index.....	456
Chapter 16 — Q&A Pack.....	457
A. Objective map.....	457
B. Exam-style question bank.....	457
C. Deep-understanding set.....	461
D. Interview and design-defence questions.....	461
E. Flashcards.....	461
F. Cumulative questions (Chapters 1–16).....	462
G. Challenge lab — Spectrum-X pods on Helix-B.....	462
H. Answer key.....	463
Chapter 17 — The InfiniBand Diagnostic Toolkit.....	466
Exam map.....	466
"Do I know this already?" pre-quiz.....	466
Opening problem: "the network is slow" with 538 nodes.....	466
17.1 The toolkit at a glance.....	466
17.2 Local port tools: ibstat, ibstatus, ibv_devinfo.....	469
17.3 Fabric inventory and topology: ibnodes, ibnetdiscover, iblinkinfo.....	470
17.4 SM and path tools: sminfo, smpquery, ibaddr, ibtracert.....	473
17.5 Counter and PHY tools: perfquery, ibqueryerrors, mlxlink.....	473
17.6 ibping: reachability over InfiniBand.....	474
17.7 ibdiagnet: the whole-fabric check.....	475

17.8 perftest: ib_write_bw and ib_write_lat.....	478
17.9 A bottom-up diagnostic method.....	480
Hands-on lab 17 — Diagnose Helix-A from the command line (reference walkthrough).....	480
Troubleshooting scenario — "Cross-SU jobs are slower on Helix-A"	482
Chapter quick reference.....	484
Chapter FAQ.....	485
Chapter summary.....	485
Further reading.....	486
Figure index.....	487
Chapter 17 — Q&A Pack.....	488
A. Objective map.....	488
B. Exam-style question bank.....	488
C. Deep-understanding set.....	492
D. Interview and design-defence questions.....	492
E. Flashcards.....	492
F. Cumulative questions (Chapters 1-17).....	493
G. Challenge lab — a one-page fabric health script.....	493
H. Answer key.....	494
Chapter 18 — Spectrum-X Troubleshooting: cl-resource-query and WJH.....	496
Exam map.....	496
"Do I know this already?" pre-quiz.....	496
Opening problem: two incidents, one week.....	496
18.1 How a Spectrum switch holds forwarding state.....	497
18.2 cl-resource-query: reading ASIC utilisation.....	499
18.3 What exhaustion looks like.....	501
18.4 WJH on the switch.....	504
18.5 The drop-reason taxonomy.....	505
18.6 WJH in NetQ: fabric-wide and historical.....	508
18.7 Using both tools together.....	509
Hands-on lab 18 — Exhaust a table, then explain a drop.....	510
Troubleshooting scenario — "Some storage addresses unreachable from some leaves".....	511
Chapter quick reference.....	513
Chapter FAQ.....	513
Chapter summary.....	514
Further reading.....	515
Figure index.....	516
Chapter 18 — Q&A Pack.....	517
A. Objective map.....	517
B. Exam-style question bank.....	517
C. Deep-understanding set.....	521
D. Interview and design-defence questions.....	521
E. Flashcards.....	521
F. Cumulative questions (Chapters 1-18).....	522
G. Challenge lab — resource and drop report.....	522
H. Answer key.....	523
Chapter 19 — Verifying End-to-End Performance: perftest and NCCL Tests.....	525
Exam map.....	525
"Do I know this already?" pre-quiz.....	525
Opening problem: "prove it is fast".....	525
19.1 Test methodology: bottom-up.....	525
19.2 The data path under test.....	528
19.3 The perftest suite.....	530
19.4 Interpreting bandwidth results.....	532
19.5 Interpreting latency results.....	533
19.6 GPUDirect RDMA with perftest.....	534
19.7 nccl-tests: collectives across the fabric.....	535

19.8 NCCL environment variables for testing.....	537
19.9 Baselines and the Helix test matrix.....	538
19.10 Running the tests from Kubernetes pods.....	540
19.11 Common pitfalls.....	541
Hands-on lab 19 — Bottom-up performance verification.....	541
Troubleshooting scenario — "Helix-B all-reduce 30% below baseline".....	543
Chapter quick reference.....	545
Chapter FAQ.....	545
Chapter summary.....	546
Further reading.....	547
Figure index.....	548
Chapter 19 — Q&A Pack.....	549
A. Objective map.....	549
B. Exam-style question bank.....	549
C. Deep-understanding set.....	553
D. Interview and design-defence questions.....	553
E. Flashcards.....	553
F. Cumulative questions (Chapters 1–19).....	554
G. Challenge lab — automated pairwise sweep.....	554
H. Answer key.....	554
Chapter 20 — UFM Health and Fabric-Wide Diagnosis.....	557
Exam map.....	557
"Do I know this already?" pre-quiz.....	557
Opening problem: timeouts that nobody can reproduce.....	557
20.1 The System Health window.....	558
20.2 UFM server health.....	560
20.3 Fabric Health reports.....	561
20.4 Fabric Validation tests.....	563
20.5 Link quality: raw, effective and symbol BER.....	564
20.6 Unhealthy ports and port isolation.....	566
20.7 Logs, events and correlation.....	567
20.8 System dump.....	570
20.9 Health through the REST API.....	570
Hands-on lab 20 — Health-check Helix-A and isolate a bad link (reference walkthrough).....	571
Troubleshooting scenario — "Intermittent NCCL timeouts, nothing is down".....	573
Chapter quick reference.....	575
Chapter FAQ.....	576
Chapter summary.....	576
Further reading.....	578
Figure index.....	578
Chapter 20 — Q&A Pack.....	579
A. Objective map.....	579
B. Exam-style question bank.....	579
C. Deep-understanding set.....	583
D. Interview and design-defence questions.....	583
E. Flashcards.....	583
F. Cumulative questions (Chapters 1–20).....	584
G. Challenge lab — automated health gate for maintenance windows.....	584
H. Answer key.....	585
Chapter 21 — NVUE Templates and Configuration Management.....	588
Exam map.....	588
"Do I know this already?" pre-quiz.....	588
Opening problem: eight leaves, eight slightly different switches.....	588
21.1 NVUE as a declarative target.....	589
21.2 NVUE configuration files.....	590
21.3 Patch, replace and the revision workflow.....	592

21.4 Snippets: configuration NVUE doesn't model.....	594
21.5 Templating Helix-B leaves with Jinja2.....	595
21.6 Pushing configuration with the REST API.....	600
21.7 Golden configuration, drift detection and git.....	602
21.8 Zero Touch Provisioning with rendered NVUE files.....	604
Hands-on lab 21 — Template-driven Helix-B leaves.....	605
Troubleshooting scenario — "The template push wiped half the switch".....	607
Chapter quick reference.....	609
Chapter FAQ.....	610
Chapter summary.....	610
Further reading.....	612
Figure index.....	613
Chapter 21 — Q&A Pack.....	614
A. Objective map.....	614
B. Exam-style question bank.....	614
C. Deep-understanding set.....	618
D. Interview and design-defence questions.....	618
E. Flashcards.....	618
F. Cumulative questions (Chapters 1–21).....	619
G. Challenge lab — complete templates and a gated pipeline.....	619
H. Answer key.....	620
Chapter 22 — Ansible Automation for Spectrum-X.....	622
Exam map.....	622
"Do I know this already?" pre-quiz.....	622
Opening problem: sixteen switches, four VLANs, one typo.....	622
22.1 Ansible basics for network engineers.....	623
22.2 How Ansible talks to NVUE.....	625
22.3 The `nvidia.nvue` collection.....	626
22.4 Playbook: create the Helix-B tenant VLANs.....	628
22.5 Playbook: enable lossless RoCE and verify it.....	629
22.6 Alternatives: `uri` against the REST API, and SSH with `nv`.....	631
22.7 Roles, handlers, vault and safe rollout.....	634
22.8 CI pipeline and testing with NVIDIA Air.....	635
Hands-on lab 22 — Automate VLANs and RoCE on the Helix-B twin.....	637
Troubleshooting scenario — "The playbook reports changed on every run, and some runs can't reach port 8765".....	639
Chapter quick reference.....	641
Chapter FAQ.....	641
Chapter summary.....	642
Further reading.....	643
Figure index.....	644
Chapter 22 — Q&A Pack.....	645
A. Objective map.....	645
B. Exam-style question bank.....	645
C. Deep-understanding set.....	649
D. Interview and design-defence questions.....	649
E. Flashcards.....	649
F. Cumulative questions (Chapters 1–22).....	650
G. Challenge lab — VLAN and RoCE compliance role.....	650
H. Answer key.....	651
Chapter 23 — Exam Strategy and Final Review.....	654
Exam map.....	654
"Do I know this already?" pre-quiz.....	654
Opening problem: the engineer who knew everything and ran out of time.....	654
23.1 Exam logistics.....	654
23.2 Time management.....	655

23.3 Question types and how to attack them.....	657
23.4 Elimination techniques.....	659
23.5 Domain-by-domain final review.....	659
23.6 The final seven days.....	662
23.7 Exam-day checklist.....	665
23.8 Using the mock exams and the score-to-chapter map.....	665
Troubleshooting scenario — "I keep scoring about 72%".....	666
Chapter quick reference.....	667
Chapter FAQ.....	667
Chapter summary.....	668
Further reading.....	668
Figure index.....	669
Mock Exam 1.....	670
Instructions.....	670
Questions.....	670
Answer key and explanations.....	677
Score by domain.....	682
Mock Exam 2.....	683
Instructions.....	683
Questions.....	683
Answer key and explanations.....	693
Score by domain.....	697
Mock Exam 3.....	699
Instructions.....	699
Questions.....	699
Answer key and explanations.....	709
Score by domain.....	712
Appendix A — Domain Quick-Review Sheets.....	714
D1 — AI Data Center Design and Optimization (5%).....	714
D2 — NVIDIA Spectrum Networking (30%).....	715
D3 — NVIDIA InfiniBand Networking (30%).....	717
D4 — Kubernetes Integration (5%).....	718
D5 — Troubleshooting Tools (20%).....	720
D6 — Automation and Configuration (10%).....	721
Appendix B — Final-Week Cram Sheet.....	723
B.1 Spectrum-X RoCE QoS.....	723
B.2 Spectrum-X platform, AR and telemetry.....	723
B.3 InfiniBand and UFM.....	724
B.4 Hosts, Kubernetes and tests.....	725
B.5 Status words to recognise instantly.....	725
Appendix C — Command Reference.....	727
C.1 Cumulus Linux and NVUE.....	727
C.2 NetQ.....	728
C.3 What Just Happened (WJH).....	728
C.4 cl-resource-query and forwarding resources.....	729
C.5 Host RDMA, QoS and DOCA.....	729
C.6 InfiniBand diagnostic tools.....	729
C.7 OpenSM, MLNX-OS and NVOS.....	730
C.8 UFM.....	730
C.9 perfctest.....	731
C.10 nccl-tests.....	731
C.11 Kubernetes and Network Operator.....	731
C.12 Ansible.....	732
Appendix D — Glossary.....	733
Appendix E — Flashcard Deck.....	737
Appendix F — Official Resources.....	740

F.1 Certification.....	740
F.2 Spectrum-X and Cumulus Linux.....	740
F.3 NetQ and NVIDIA Air.....	740
F.4 InfiniBand, UFM and OpenSM.....	741
F.5 Hosts, DOCA and Kubernetes.....	741
F.6 Benchmarks and NCCL.....	741
About the Author.....	742
Index.....	743

About This Book

Who this book is for

This book is for network and infrastructure engineers who already know data centre networking and now need to run the networks that connect GPUs. You should be comfortable with IP routing, BGP basics, VLANs and VXLAN, the Linux command line and simple YAML. You are **not** expected to know RDMA, InfiniBand, lossless Ethernet, NCCL or Kubernetes networking internals. The book teaches all of these from first principles.

The book has two goals, and it treats them as equally important:

- **Pass the NVIDIA-Certified Professional: AI Networking (NCP-AIN) exam.** Every chapter maps to numbered objectives in NVIDIA's official exam blueprint, and every question is tagged with the objective it tests.
- **Be able to do the job.** Passing an exam you could not apply at work is a poor return on your time. Every concept is tied to a real design decision, configuration, verification step or failure mode in a production AI factory.

Disclaimer: This is an independent study guide. It is not produced, endorsed or reviewed by NVIDIA Corporation. NVIDIA, Spectrum, Spectrum-X, Quantum, InfiniBand-related product names, ConnectX, BlueField, DOCA, NVLink, NVSwitch, DGX, HGX, UFM, NetQ, Cumulus and other marks belong to their respective owners. Product details change quickly. Always confirm commands, defaults and hardware specifications against current NVIDIA documentation before using them in production. All practice questions and mock exams are original; none are real exam questions.

About the NCP-AIN exam

These facts come from NVIDIA's certification page and exam study guide (checked September 2026). Confirm them on the certification page before you book, because NVIDIA revises its exams.

Item	Detail
Exam	NVIDIA-Certified Professional: AI Networking (NCP-AIN)
Level	Professional (intermediate)
Length	120 minutes
Questions	70–75 (about 1.6 minutes per question)
Delivery	Online, remotely proctored
Language	English
Price	USD 400
Validity	Two years from issue; recertify by retaking the exam
Recommended experience	Two to three years of operational experience in a data centre with NVIDIA hardware
Registration	Certiverse exam portal (linked from the NVIDIA certification page)

Exam focus: NVIDIA publishes no sample questions and no pass mark. This book assumes mainly scenario-based multiple-choice questions, both single-answer and "choose TWO/THREE". NVIDIA's certification page does not state the question formats or the pass mark, so confirm them on the current exam page before booking.

The official exam blueprint

The blueprint has six domains. Their weights tell you where to spend your time. The two platform domains, Spectrum and InfiniBand, make up 60% of the exam, and troubleshooting adds another 20%.

#	Domain	Weight	Objectives
1	AI Data Center Design and Optimization	5%	1.1 AI factory networking architecture and components (GPUs, BlueField, scalable units, switches) · 1.2 Rail-optimized topologies · 1.3 GPU-to-GPU communications
2	NVIDIA Spectrum Networking	30%	2.1 Configure Spectrum-X switches for RoCE · 2.2 Enable QoS, ECN, PFC, adaptive routing and telemetry · 2.3 Multi-tenant BGP-EVPN · 2.4 Simulate with NVIDIA Air · 2.5 Diagnose congestion and packet loss with in-band telemetry and WJH · 2.6 NetQ real-time monitoring · 2.7 Install NVIDIA DOCA · 2.8 Configure SuperNIC functionality
3	NVIDIA InfiniBand Networking	30%	3.1 Initial configuration, provisioning and HA · 3.2 PKeys for secure multi-tenancy · 3.3 QoS and adaptive routing · 3.4 UFM monitoring of link status and bandwidth
4	Kubernetes Integration	5%	4.1 Deploy NVIDIA Network Operator · 4.2 Verify Network Operator functionality
5	Troubleshooting Tools	20%	5.1 cl-resource-query · 5.2 WJH real-time event analysis · 5.3 Verify low-latency interconnects · 5.4 UFM system health · 5.5 ib_write_lat, ib_write_bw, ibping, ibstat, ibdiagnet, ibnodes, iblinkinfo
6	Automation and Configuration	10%	6.1 NVUE templates · 6.2 Ansible playbooks (VLAN creation, RoCE configuration)

NCP-AIN blueprint weighting (%)

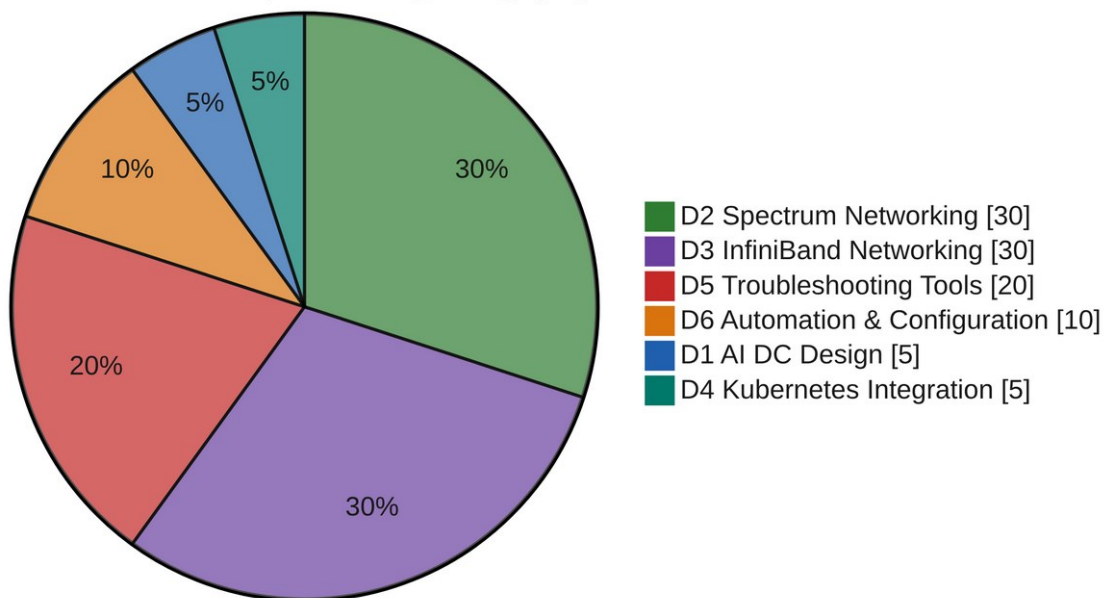


Figure 0.1: Exam weight by domain

Spectrum and InfiniBand together are 60% of the exam. Add Troubleshooting and they reach 80%. Plan your study hours in roughly the same proportions.

How each chapter is built

Every chapter follows the same pattern, so you always know where to find things:

1. **Exam map:** the objectives covered, the domain weight, and what the exam expects you to be able to *do*.
2. **"Do I know this already?" pre-quiz:** three questions. If you answer all three correctly and confidently, skim the chapter and go straight to its Q&A pack.
3. **Opening problem:** a real situation in the Helix AI Factory, the running example used throughout the book.
4. **Core sections:** concept, figures, configuration, verification, and Ethernet-versus-InfiniBand comparisons, each ending with a *Check Your Understanding* box.
5. **Hands-on lab:** built on NVIDIA Air, Linux hosts or a local Kubernetes cluster wherever possible. If a lab needs hardware you are unlikely to have, it is labelled **Reference walkthrough** and shows documented outputs.
6. **Troubleshooting scenario:** symptom, then commands, then root cause, then fix, with a flowchart.
7. **Quick reference, FAQ, summary and visual recap.**
8. **Q&A pack:** at least 40 exam-style questions, a deeper-understanding set, interview questions, flashcards, a challenge lab and a full answer key that explains why each wrong option is wrong.

Errata, updates and companion files

NVIDIA ships new software releases several times a year, so some commands will change after publication. Corrections, updated commands, the flashcard deck (CSV, ready for Anki or Quizlet) and lab files are published at <https://cloudfoxy-ltd.github.io/ncp-ain-guide/>. If you find an error, please report it there; confirmed fixes go into the next printing and ebook update.

Callouts used in this book

Exam focus (x.y): What the exam is likely to test about this topic, tied to a blueprint objective.

Exam trap: A common misconception that often shows up as a wrong option in exam questions.

Field note: Practical advice from real deployments that goes beyond the exam.

Version note: Behaviour that differs between software releases, such as Cumulus Linux 4.x (NCLU) and 5.x (NVUE).

Warning: A step that can cause an outage, data loss or a security exposure.

Where a detail varies by release or could not be confirmed against NVIDIA's public documentation, the text says so, for example "(confirm against your release notes)" or a **Version note**. Check these details against the current NVIDIA documentation before relying on them.

Visual legend

Every diagram in the book uses the same colours and line styles.

Element	Meaning
Green	NVIDIA Spectrum / Ethernet (Spectrum-X compute, RoCE)
Purple	NVIDIA Quantum / InfiniBand

Element	Meaning
Blue	GPU and NVLink scale-up domain
Teal	Storage fabric
Orange	Control or management plane (SM, UFM, NetQ, BGP)
Grey	Out-of-band (OOB) management
Red	Fault path, congestion or failure
Solid line	Data plane
Dashed line	Control plane or logical relationship
Numbered steps ① ② ③	Sequence that matches numbered text

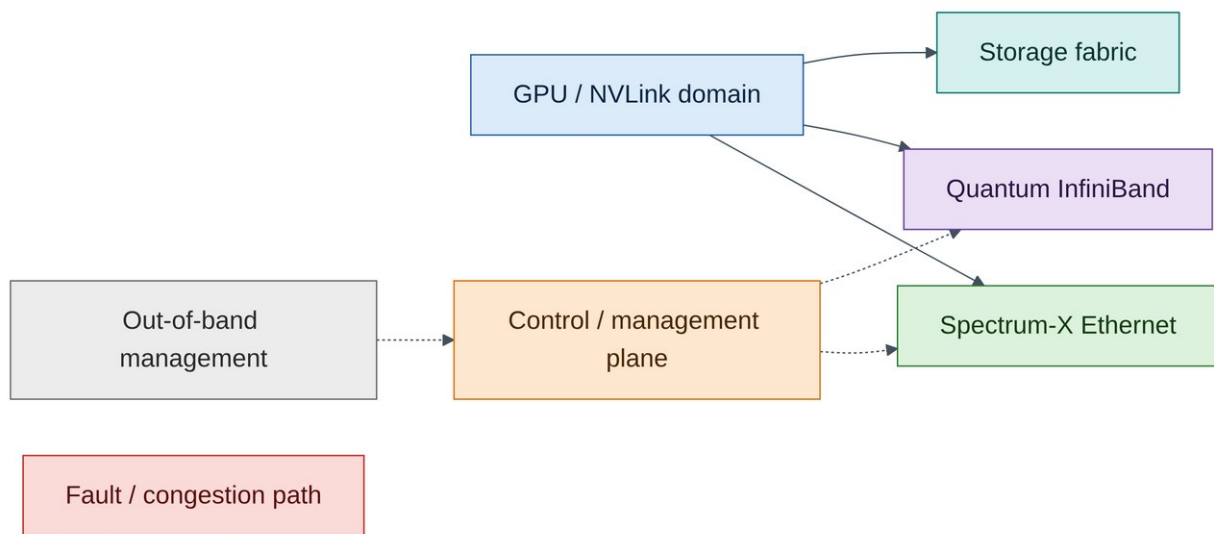


Figure 0.2: Visual legend applied

The same colour code appears in every figure. Learn it once and you can read any diagram at a glance.

Study plans

Choose the plan that fits your timeline. Both follow the blueprint weighting.

14-day intensive plan

This plan assumes 3–4 focused hours a day and some hands-on NVIDIA background.

Day	Chapters	Domain	Labs and question targets
1	1–3	D1 and foundations	Node inventory lab; 120 questions
2	4–5	D2	NVIDIA Air: NVUE basics, RoCE lossless config; 80 questions
3	6–7	D2	Adaptive routing and telemetry; EVPN tenants; 80 questions
4	8–10	D2 and D5	DOCA install, SuperNIC, NetQ and WJH; 120 questions
5	Review D2	D2	Domain 2 quick-review sheet; 60-question D2 drill
6	11–12	D3	Subnet Manager, provisioning, HA; 80 questions
7	13–14	D3	PKeys, QoS, adaptive routing, SHARP; 80 questions
8	15, 20	D3 and D5	UFM monitoring and health; 80 questions
9	17	D5	IB diagnostic toolkit drill (every tool, every flag); 60 questions

Day	Chapters	Domain	Labs and question targets
10	18–19	D5	cl-resource-query, WJH, perftest, NCCL tests; 80 questions
11	16, 21–22	D4 and D6	Network Operator on kind; NVUE templates; Ansible; 100 questions
12	Mock Exam 1	All	Timed, 75 questions; review every wrong answer
13	Weak domains, Mock Exam 2	All	Re-read weakest two chapters; timed mock
14	Cram sheet, Mock Exam 3	All	Light review; book the exam when all mocks score 80% or more

6-week standard plan

Week	Focus	Chapters	Milestone
1	Foundations and D1	1–3	Can draw an AI factory and a rail-optimized SU from memory
2	Spectrum-X, part 1	4–7	Working lossless RoCE and EVPN lab in NVIDIA Air
3	Spectrum-X, part 2, and monitoring	8–10, 18	DOCA and NetQ lab done; D2 drill of 80% or more
4	InfiniBand	11–15	Can explain SM, PKeys, SL-to-VL mapping and UFM; D3 drill of 80% or more
5	Troubleshooting, K8s, automation	16–17, 19–22	Every blueprint tool practised; D4–D6 drills
6	Exam readiness	23, mocks	Three mock exams at 80% or more, then book the exam

Book Plan and Blueprint Map

The learning arc

The book moves in a set order. First you learn *why* AI networks are different. Then you build and operate each platform: Spectrum-X Ethernet first, because its concepts (QoS, PFC, ECN, EVPN) build on Ethernet knowledge you already have, then InfiniBand. After that come Kubernetes, troubleshooting as a skill in its own right, and automation. Exam readiness comes last.

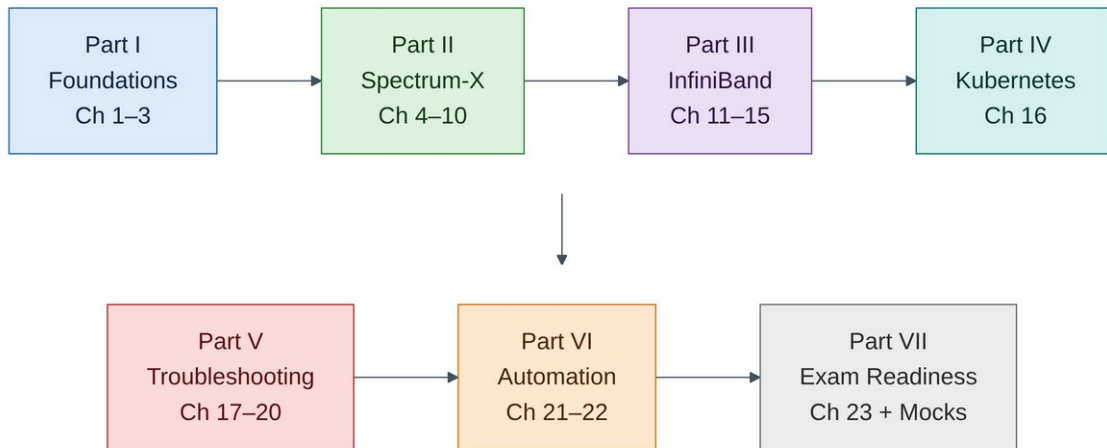


Figure 0.3: The learning arc

Each part builds on the one before it. Troubleshooting (Part V) draws on every earlier part, which is why it carries 20% of the exam weight.

Table of contents and chapter plan

Ch	Title	Objectives	Key lab	Key figures
Part I	Foundations			
1	AI Factory Networking Architecture	1.1	GPU node network inventory	Four fabrics; node internals; SU building block
2	GPU-to-GPU Communication, NCCL and Rail-Optimized Topologies	1.2, 1.3	Trace an all-reduce across rails	NVLink vs GPUDirect RDMA; ring/tree all-reduce; rail-to-leaf
3	RDMA Fundamentals: Verbs, InfiniBand Transport and RoCEv2	Supports 2.1, 3.x, 5.3	Soft-RoCE on Linux	RDMA vs TCP path; RoCEv2 and IB headers; QP model
Part II	NVIDIA Spectrum-X Ethernet			
4	Cumulus Linux and NVUE Essentials	2.1, 6.1	First Spectrum-X switch in NVIDIA Air	NVUE object model; set, apply, save workflow
5	Lossless RoCE: QoS, PFC and ECN	2.1, 2.2	Configure RoCE lossless mode	DSCP to priority to queue; PFC pause; ECN/CNP sequence
6	Spectrum-X Adaptive Routing, Congestion Control and Telemetry	2.2	Enable AR and telemetry	ECMP vs packet spraying; SuperNIC reordering
7	Multi-Tenancy with BGP-EVPN	2.3	Two tenant VRFs on Helix-B	Tenant VRFs, L2/L3 VNIs, type-2/type-5 routes
8	BlueField, SuperNICs and DOCA	2.7, 2.8	Install DOCA; set	DPU vs SuperNIC roles; DOCA stack

Ch	Title	Objectives	Key lab	Key figures
			SuperNIC mode	
9	Lab Simulation with NVIDIA Air	2.4	Build Helix-B digital twin	Air workflow; topology import
10	Monitoring: NetQ, WJH and In-Band Telemetry	2.5, 2.6, 5.2	Detect congestion with NetQ	NetQ architecture; WJH event flow
Part III	NVIDIA InfiniBand			
11	InfiniBand Architecture and the Subnet Manager	3.1	Explore a subnet with ibnetdiscover	SM/SA; LID assignment; port state machine
12	Initial Configuration, Provisioning and High Availability	3.1	SM and UFM HA failover	Master/standby SM; UFM HA pair
13	Partition Keys and Secure Multi-Tenancy	3.2	Create full and limited partitions	PKey membership; IPoIB per partition
14	InfiniBand QoS, Adaptive Routing and SHARP	3.3	Configure SL to VL mapping and AR	SL to VL; AR decision; SHARP aggregation tree
15	Monitoring with UFM	3.4, 5.4	Link and bandwidth monitoring	UFM architecture; telemetry flow
Part IV	Kubernetes Integration			
16	NVIDIA Network Operator: Deploy and Verify	4.1, 4.2	Deploy on kind or k3s; RDMA pod test	Operator, NicClusterPolicy, CNI stack
Part V	Troubleshooting			
17	The InfiniBand Diagnostic Toolkit	5.5	Every IB tool, healthy and broken	Tool-per-layer map; ibdiagnet flow
18	Spectrum-X Troubleshooting: cl-resource-query and WJH	5.1, 5.2	Resource exhaustion and WJH drops	Drop-reason taxonomy
19	Verifying End-to-End Performance: perfctest and NCCL Tests	5.3	ib_write_bw/lat; nccl-tests	Test matrix; GPU to NIC to fabric path
20	UFM Health and Fabric-Wide Diagnosis	5.4	UFM health and system dump	Health workflow
Part VI	Automation and Configuration			
21	NVUE Templates and Configuration Management	6.1	Template-driven leaf builds	Template to rendered config pipeline
22	Ansible Automation for Spectrum-X	6.2	VLAN and RoCE playbooks	Ansible to NVUE API workflow
Part VII	Exam Readiness			
23	Exam Strategy, Final Review and Mock Exams	All	Three 75-question timed mocks	Domain score to chapter map

Blueprint coverage matrix

Minimum coverage targets for each objective across the whole book: one or more chapters, one or more labs, three or more figures and 40 or more questions.

Obj	Primary chapter(s)	Lab	Target figures	Target questions
1.1	1	Ch 1	12	60

Obj	Primary chapter(s)	Lab	Target figures	Target questions
1.2	2	Ch 2	6	40
1.3	2, 3	Ch 2, 3	6	40
2.1	4, 5	Ch 4, 5	8	70
2.2	5, 6	Ch 5, 6	10	80
2.3	7	Ch 7	8	50
2.4	9	Ch 9	4	40
2.5	10, 18	Ch 10	5	45
2.6	10	Ch 10	4	45
2.7	8	Ch 8	3	40
2.8	8, 6	Ch 8	4	40
3.1	11, 12	Ch 11, 12	10	70
3.2	13	Ch 13	5	50
3.3	14	Ch 14	7	60
3.4	15	Ch 15	4	45
4.1	16	Ch 16	4	40
4.2	16	Ch 16	3	40
5.1	18	Ch 18	3	40
5.2	10, 18	Ch 18	3	40
5.3	19	Ch 19	4	45
5.4	15, 20	Ch 20	3	40
5.5	17	Ch 17	6	80
6.1	4, 21	Ch 21	3	45
6.2	22	Ch 22	3	45

Tool coverage matrix

Every tool named in the blueprint gets: its purpose, syntax, key flags, annotated healthy and faulty output, exam traps, and at least five questions.

Tool	Where taught	Where drilled
NVUE (nv set / show / config apply)	Ch 4	Ch 5–7, 21, 22
NVIDIA Air	Ch 9	Labs in Ch 4–7, 10, 18, 21, 22
NetQ	Ch 10	Ch 18, 23
What Just Happened (WJH)	Ch 10	Ch 18
In-band telemetry	Ch 6, 10	Ch 18
cl-resource-query	Ch 18	Ch 23
DOCA	Ch 8	Ch 16
SuperNIC configuration	Ch 8	Ch 6
UFM	Ch 12, 15	Ch 20
ibstat	Ch 11	Ch 17

Tool	Where taught	Where drilled
ibnodes	Ch 11	Ch 17
iblinkinfo	Ch 11	Ch 17
ibping	Ch 17	Ch 19
ibdiagnet	Ch 17	Ch 20
ib_write_bw / ib_write_lat	Ch 19	Ch 17
NVIDIA Network Operator	Ch 16	Ch 23
Ansible (NVUE)	Ch 22	Ch 23

The Helix AI Factory: Running Example and Visual Kit

Every figure, lab and scenario question in this book uses one fictional organisation: **Helix**, a research and analytics company building an AI factory. Using a single example has one big benefit. By the end of the book you will know one realistic network completely, and every question will feel like your own network.

Warning: Helix is fictional. Its topology follows the patterns in NVIDIA's published reference architectures, but switch counts, cable plans and hardware choices are simplified for teaching. Use NVIDIA's reference architecture documents, not this book, as the source for a real bill of materials.

Helix at a glance

Component	Helix-A (training cluster)	Helix-B (multi-tenant inference cluster)
Purpose	Large-scale model training, single tenant	GPU inference and fine-tuning for two internal tenants
GPU nodes	64 × 8-GPU nodes (DGX B200-class) in 2 scalable units of 32	16 × 8-GPU nodes (HGX-class) in 1 scalable unit
Compute (east-west) fabric	InfiniBand NDR 400 Gb/s, Quantum-2 switches	Spectrum-X Ethernet 400 GbE, Spectrum-4 SN5600 switches
Compute NICs per node	8 × ConnectX-7 (1 per GPU, 1 per rail)	8 × BlueField-3 SuperNICs (1 per GPU, 1 per rail)
Topology	Rail-optimized two-tier fat-tree (8 rail leaves per SU)	Rail-optimized leaf-spine (8 rail leaves, 4 spines)
Fabric management	UFM Enterprise HA pair; Subnet Manager on UFM	NetQ; NVUE; Ansible
Multi-tenancy	PKeys (research vs production partitions)	BGP-EVPN VRFs: tenants AURORA and BOREALIS
Shared services	Ethernet storage fabric; in-band management; OOB management; Kubernetes with NVIDIA Network Operator	Same

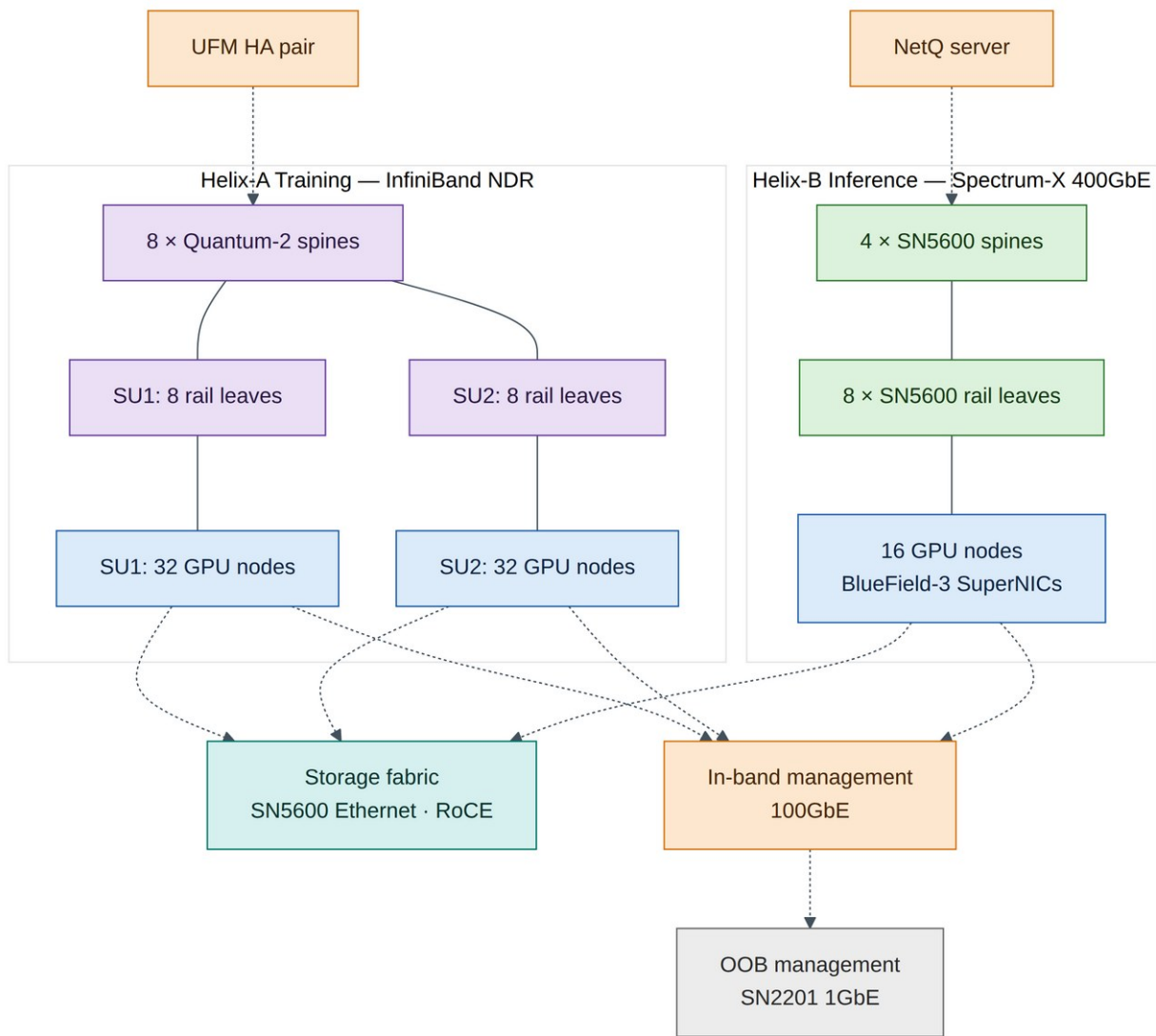


Figure 0.4: Helix AI Factory — master reference topology (overview)

The four networks stay physically separate. The two compute fabrics, InfiniBand for Helix-A and Spectrum-X for Helix-B, never carry storage or management traffic.

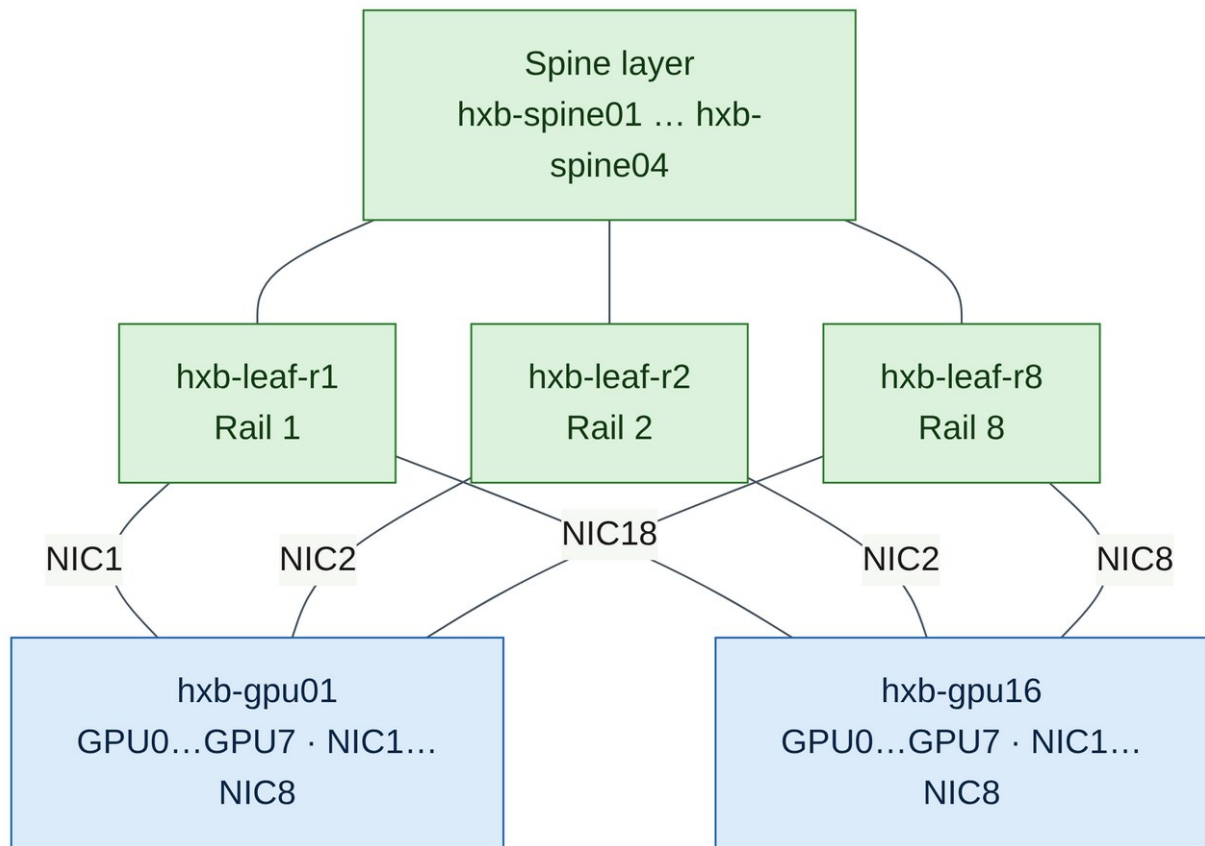


Figure 0.5: Helix-B rail-optimized leaf-spine (logical view)

Every node's NIC *n* connects to rail leaf *n*. GPUs with the same local index on different nodes are therefore one switch hop apart. Only traffic between different rails crosses a spine.

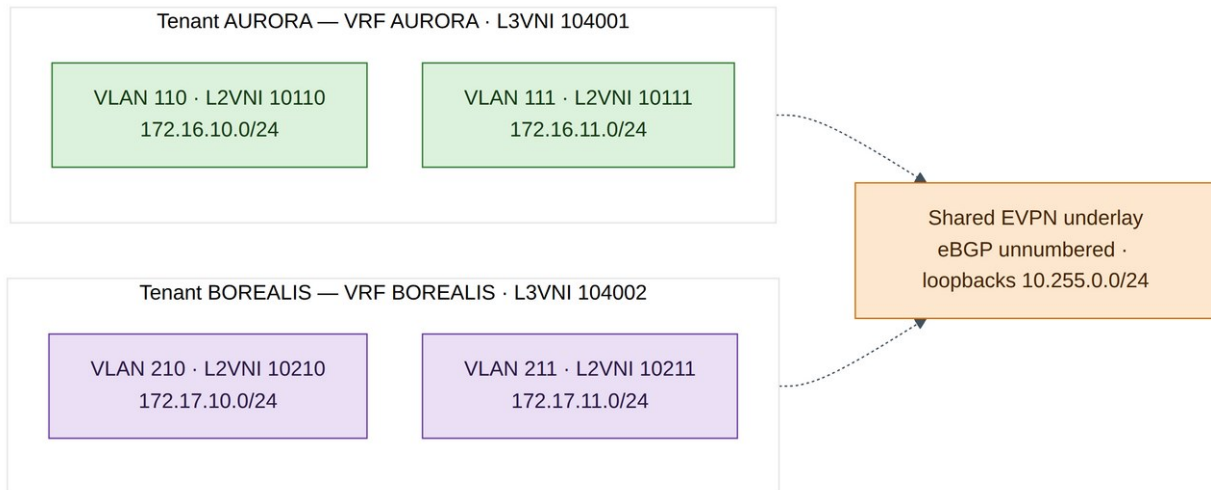


Figure 0.6: Helix-B tenant model (BGP-EVPN)

Two tenant VRFs share the same physical fabric. Chapter 7 builds this. The L3VNI carries routed traffic for each tenant, and each L2VNI extends one tenant subnet across the rail leaves.

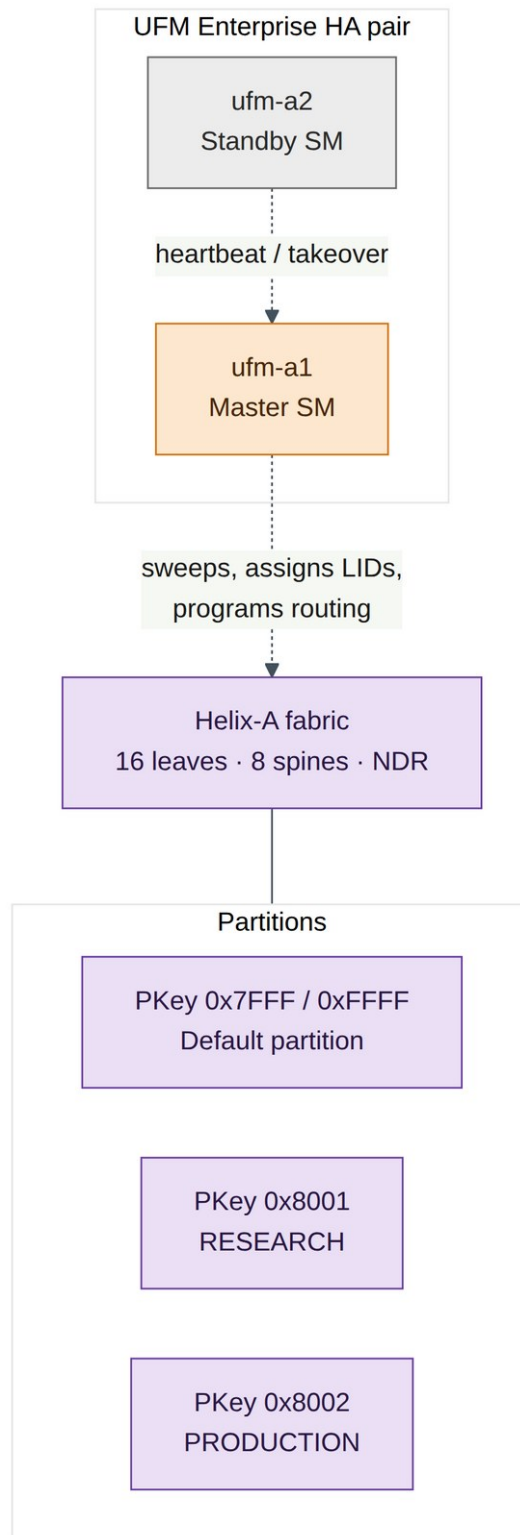


Figure 0.7: Helix-A InfiniBand subnet (logical view)

One Subnet Manager, the master, runs on the active UFM appliance and assigns LIDs. A standby SM on the second appliance takes over if the master fails. PKeys split the subnet into partitions.

Naming and addressing plan

Every chapter uses these names and numbers. When a question mentions **hxb-leaf-r3** or PKey **0x8002**, this table tells you exactly where it sits.

Item	Convention / value
Helix-A GPU nodes	<code>hxa-su1-gpu01 ... hxa-su1-gpu32, hxa-su2-gpu01 ... hxa-su2-gpu32</code>
Helix-A leaves	<code>hxa-su1-leaf-r1 ... r8, hxa-su2-leaf-r1 ... r8</code>
Helix-A spines	<code>hxa-spine01 ... hxa-spine08</code>
Helix-A UFM	<code>ufm-a1</code> (active / master SM), <code>ufm-a2</code> (standby)
Helix-A PKeys	<code>0x8001</code> RESEARCH, <code>0x8002</code> PRODUCTION (plus default partition)
Helix-B GPU nodes	<code>hxb-gpu01 ... hxb-gpu16</code>
Helix-B leaves / spines	<code>hxb-leaf-r1 ... r8 / hxb-spine01 ... 04</code>
Helix-B BGP ASNs	Spines <code>65100</code> ; leaves <code>65101 ... 65108</code> (eBGP unnumbered underlay)
Helix-B loopbacks	<code>10.255.0.1–4</code> spines; <code>10.255.0.11–18</code> leaves
Helix-B tenant VRFs	AURORA L3VNI <code>104001</code> ; BOREALIS L3VNI <code>104002</code>
AURORA subnets	VLAN 110 / L2VNI 10110 / <code>172.16.10.0/24</code> ; VLAN 111 / L2VNI 10111 / <code>172.16.11.0/24</code>
BOREALIS subnets	VLAN 210 / L2VNI 10210 / <code>172.17.10.0/24</code> ; VLAN 211 / L2VNI 10211 / <code>172.17.11.0/24</code>
RoCE traffic marking	DSCP 26 → switch priority 3 (lossless); CNP DSCP 48 → priority 6 (Cumulus Linux lossless RoCE defaults)
Storage fabric	<code>hxs-leaf01/02</code> (SN5600); <code>10.20.0.0/16</code>
In-band management	<code>10.10.0.0/16</code>
OOB management	<code>192.168.0.0/16</code> ; switches <code>hx-oob-sw01...</code> (SN2201)
Monitoring	<code>netq-01</code> (NetQ server); UFM as above
Kubernetes	Cluster <code>helix-k8s</code> ; operator namespace <code>nvidia-network-operator</code>

Part I — Foundations

Chapter 1 — AI Factory Networking Architecture

Exam map

Domain	1 — AI Data Center Design and Optimization (5%)
Objective	1.1 Describe AI factory networking architecture and components (GPUs, BlueField, scalable units, switches)
Also prepares	1.2 and 1.3 (Chapter 2), 2.8 (Chapter 8), 3.1 (Chapter 11)
What you must be able to do	Name the separate networks in an AI factory and what each one carries. Explain what is inside a GPU node and how its NICs map to GPUs. Tell a ConnectX NIC, a BlueField DPU and a SuperNIC apart. Explain what a scalable unit (SU) is and why AI clusters are built from them. Pick the right switch family for each network.

"Do I know this already?" pre-quiz

Answer these three questions before you read on. The answers are at the end of the chapter's answer key.

PQ1. In an NVIDIA DGX SuperPOD-style design, which network carries GPU-to-GPU traffic between nodes during distributed training?

A. In-band management network · B. Storage network · C. Compute (east-west) fabric · D. Out-of-band network

PQ2. A DGX B200 system has eight B200 GPUs. How many ConnectX-7 ports does it dedicate to the compute fabric?

A. 2 · B. 4 · C. 8 · D. 16

PQ3. Which statement about a scalable unit (SU) is correct?

A. It is a single switch chassis. B. It is a repeatable block of GPU nodes plus the leaf switches and cabling that serve them. C. It is a Kubernetes namespace for GPU jobs. D. It is a UFM licence tier.

Opening problem: why Helix's first cluster underperformed

Helix bought its first 16 GPU servers two years ago and connected them the way it connected everything else: two 100 GbE ports per server into a pair of general-purpose top-of-rack switches, with storage, management and GPU traffic sharing the same links. The GPUs were the latest generation. Training jobs still ran at barely half the expected speed. Monitoring showed the GPUs *waiting* for 40–50% of each training step.

The GPUs were not the problem. The network was. Every step of a distributed training job ends with a **collective operation**: all GPUs exchange gradient data and cannot start the next step until the slowest transfer completes. On a shared, lossy network with too little bandwidth, every step waited on the worst-case transfer.

This chapter explains how an **AI factory** network is built so that doesn't happen: separate purpose-built fabrics, one high-speed NIC per GPU, repeatable scalable units, and switches chosen for each job. By the end you will be able to draw the Helix AI Factory from memory, and you will need that picture in every later chapter.

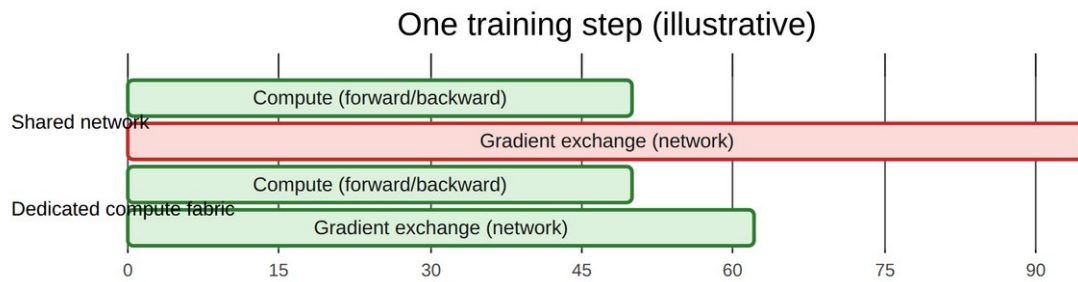


Figure 1.1: Where the time goes: a training step on a shared network versus a dedicated compute fabric

On a shared or lossy network (top), communication stretches every step and GPUs sit idle. A dedicated, lossless, high-bandwidth compute fabric (bottom) shrinks the communication phase. The GPUs are the same; only the network differs.

1.1 What an AI factory is, and why its network is different

NVIDIA uses the term **AI factory** for a data centre built to produce intelligence at scale. Its "raw material" is data and its "product" is trained models and inference results (tokens). The term matters for the exam because it signals a mindset: the whole data centre, including its networks, is designed as **one computer** whose job is to keep GPUs busy.

That leads to the metric that shapes every design choice in this book:

- **Job completion time (JCT):** how long a training job takes from start to finish. For a synchronous training job, JCT depends on the *slowest* participant in every step, not the average. That is why **tail latency** (the worst few percent of transfers) matters more than average latency in AI networking.

How AI traffic differs from traditional data centre traffic

Characteristic	Traditional enterprise / cloud DC	AI training fabric
Dominant direction	North-south (users ↔ applications) plus mixed east-west	Overwhelmingly east-west (GPU ↔ GPU)
Flow profile	Many small, short-lived flows (mice)	Few, very large, long-lived flows (elephant flows)
Entropy	High (many 5-tuples), so ECMP hashing spreads load well	Low (few 5-tuples), so ECMP hash collisions leave some links congested while others sit idle
Synchronisation	Independent requests	Synchronous collectives: everyone waits for the slowest
Loss tolerance	TCP retransmits; small impact	A single loss can stall an RDMA transfer and the whole step
Burstiness	Moderate	Extreme, with many-to-one incast at step boundaries
Key metric	Throughput, availability	JCT, tail latency, effective bandwidth

Exam trap: Questions sometimes offer "higher average throughput" as the goal of an AI fabric. The better answer is almost always about **predictable, low tail latency and lossless behaviour**, because synchronous collectives are limited by the slowest flow.

Two technologies answer these needs, and the rest of this book is built on them:

- **RDMA (Remote Direct Memory Access):** the NIC moves data directly between the memories of two machines, and between GPU memories with **GPUDirect RDMA**, without involving the CPU or the kernel TCP/IP stack. InfiniBand uses RDMA natively. Ethernet uses it through **RoCE (RDMA over Converged Ethernet)**. Chapter 3 covers RDMA in depth.

- **Purpose-built fabrics:** InfiniBand (NVIDIA Quantum) and Spectrum-X Ethernet (NVIDIA Spectrum-4 switches plus SuperNICs). Both provide lossless or near-lossless transport, adaptive routing and congestion control designed for AI traffic.

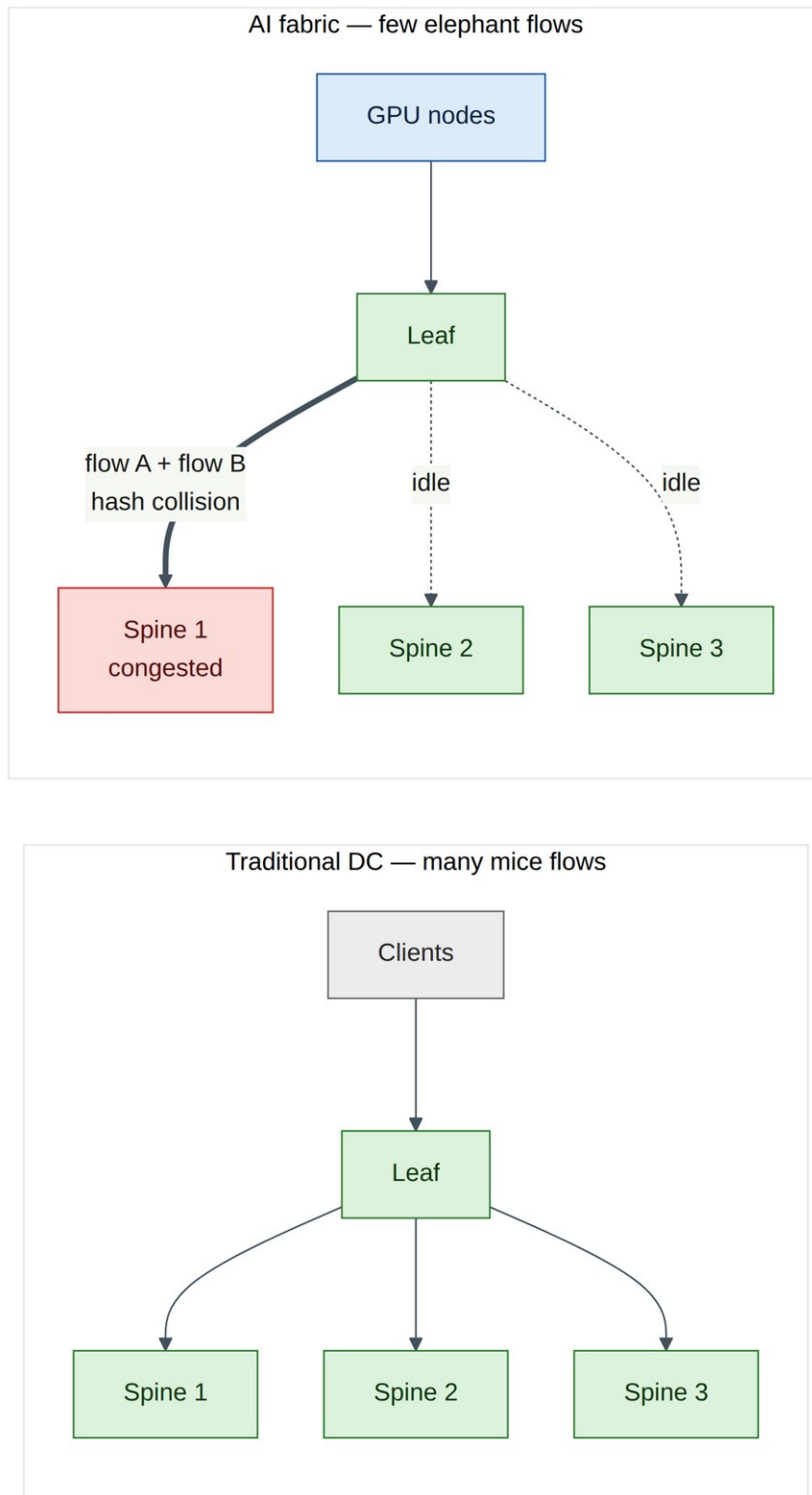


Figure 1.2: Traditional versus AI traffic patterns

Left: many small, independent flows spread well across ECMP paths. Right: a few huge synchronous flows collide on the same paths (red) and leave others idle. This is the core reason AI fabrics need adaptive routing and congestion control.

Check your understanding 1.1: Why is ECMP load balancing less effective for AI training traffic than for web

traffic? (Answer: AI traffic has low entropy, meaning a few large long-lived flows. Hash-based ECMP pins each flow to one path, so collisions overload some links while others sit idle.)

1.2 The separate networks of an AI factory

NVIDIA's DGX SuperPOD reference architectures separate traffic onto **physically distinct networks**. Each has its own switches, NICs and design rules. Learn this list well. It appears in almost every scenario question, even when the question is really about something else.

Network	Carries	Typical technology	Design priorities
Compute fabric (east-west, "backend", "scale-out")	GPU-to-GPU traffic between nodes: NCCL collectives, GPUDirect RDMA	InfiniBand (Quantum-2 / Quantum-X800) or Spectrum-X Ethernet; one NIC per GPU; rail-optimized	Highest bandwidth, lossless, lowest tail latency, full bisection bandwidth
Storage fabric	Dataset reads, checkpoint writes to high-performance storage	InfiniBand or Ethernet with RoCE; typically served by the node's BlueField-3 DPUs	High throughput (NVIDIA's SuperPOD guidance calls for over 40 GBps per node); may be slightly oversubscribed
In-band management network ("frontend", "north-south")	Cluster services: provisioning, job scheduling, SSH, NFS home directories, container images, access to external networks	Ethernet (e.g. Spectrum switches, 100–400 GbE)	Reliable general-purpose connectivity; security boundary to the enterprise
Out-of-band (OOB) management network	BMC/IPMI/Redfish of servers, management ports of switches, PDUs, storage controllers	1 GbE Ethernet (e.g. SN2201)	Must work when everything else is down; logically separated for security
NVLink (scale-up) domain	GPU-to-GPU traffic <i>inside</i> a node (or inside an NVL72 rack)	NVLink and NVSwitch	Not a "network" in the switching sense here; covered in Chapter 2

Field note: Some newer enterprise reference architectures merge the storage and in-band networks into one **converged north-south** Ethernet network for smaller deployments, while keeping the east-west compute fabric separate. The principle that never changes is this: **GPU-to-GPU compute traffic gets its own fabric.**

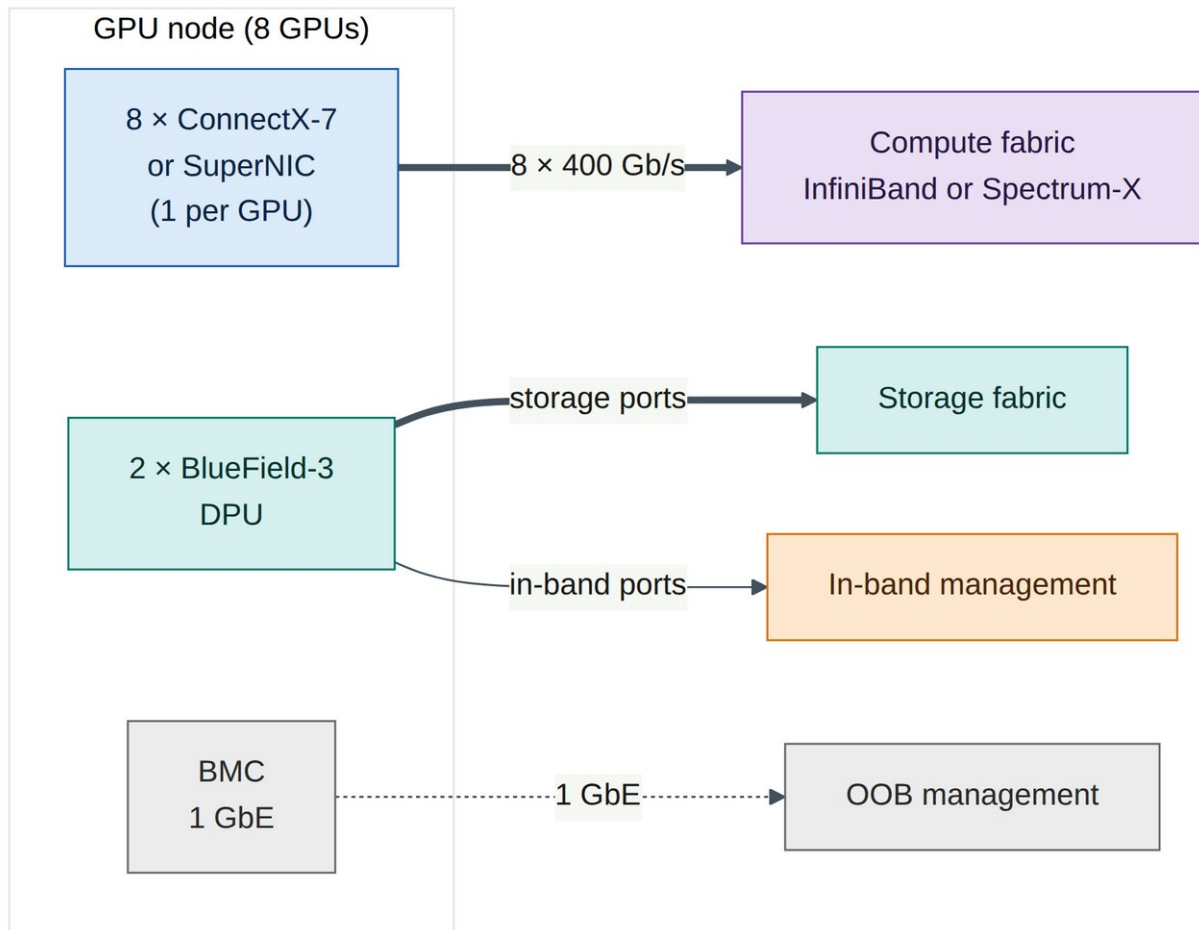


Figure 1.3: The four networks of an AI factory

Each GPU node connects to four physically separate networks. Only the compute fabric carries GPU-to-GPU collective traffic, and it gets eight of the node's high-speed ports.

Why keep the networks separate?

1. **Performance isolation.** A checkpoint write burst on the storage network cannot cause congestion that delays an all-reduce on the compute fabric.
2. **Different design rules.** The compute fabric needs lossless behaviour, full bisection bandwidth and rail alignment. The in-band network needs routing to the enterprise, firewalls and standard services. Mixing them forces bad compromises on both.
3. **Security and blast radius.** The OOB network stays reachable when the in-band network is down or compromised, and it is logically separated from user-facing networks.
4. **Operations.** Each fabric has its own management tool. UFM manages InfiniBand. NetQ and NVUE manage Spectrum Ethernet. BMC tooling manages OOB.

Exam focus (1.1): Expect scenarios such as "A user reports slow checkpoint writes. Which network should you investigate?" (storage) or "You cannot reach a node's BMC after a kernel crash. Which network?" (OOB). Match each traffic type to its network without hesitating.

Check your understanding 1.2: A data scientist's job pulls a container image from a registry outside the cluster. Which network carries that traffic? (Answer: the in-band management network. Only GPU-to-GPU collective traffic uses the compute fabric.)

1.3 Inside a GPU node

To understand the fabric, you first need to understand what plugs into it. Figure 1.4 shows the network-relevant parts of a DGX B200-class node.

Component	DGX B200 (per NVIDIA user guide)	Why it matters to the network
GPUs	8 × B200	The traffic sources and sinks
Scale-up interconnect	5th-generation NVLink via 2 NVLink switch chips; 14.4 TB/s aggregate	Intra-node GPU traffic never touches the NICs
Compute NICs	8 × ConnectX-7 (via four dual-port OSFP cages), up to 400 Gb/s each; InfiniBand by default, Ethernet capable	One NIC per GPU. Each NIC is a separate rail.
Storage / in-band	2 × BlueField-3 DPU dual-port cards	Storage fabric and in-band management
OoB	1 GbE BMC port (Redfish, IPMI, SNMP, KVM)	Out-of-band management
CPUs	2 × Intel Xeon 8570	Host software and NCCL control. With GPUDirect RDMA they are not in the data path.

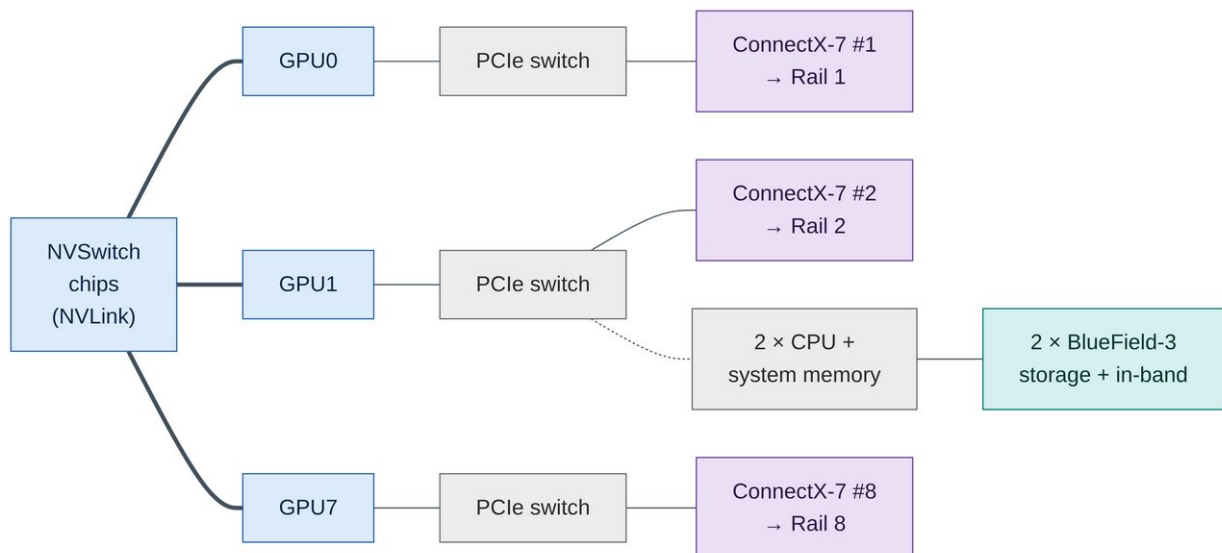


Figure 1.4: Network-relevant internals of an 8-GPU node

Each GPU shares a PCIe switch with its own ConnectX-7 NIC. GPUDirect RDMA therefore moves data GPU → PCIe switch → NIC without crossing the CPU or system memory. NVLink/NVSwitch carries GPU-to-GPU traffic inside the node.

The 1:1 GPU-to-NIC rule

The most important design fact in this chapter is **one compute NIC per GPU**. Each GPU has a dedicated 400 Gb/s path into the compute fabric through a NIC on the same PCIe switch. This gives:

- **Bandwidth parity:** 8 GPUs × 400 Gb/s = 3.2 Tb/s of scale-out bandwidth per node.
- **Topology affinity:** data moves GPU → PCIe switch → NIC without crossing the CPU interconnect. The `nvidia-smi topo -m` command shows this affinity (see the lab).
- **Rails:** NIC *n* of every node connects to the same leaf switch (or leaf group). That is a **rail**, the basis of rail-optimized topologies (Section 1.6 introduces them; Chapter 2 covers them in depth).

Exam trap: Do not confuse the **compute NICs** (ConnectX-7 or SuperNICs, one per GPU, east-west) with the **BlueField-3 DPUs** in a DGX node (storage and in-band, north-south). A question that says "the node's BlueField-3 cards carry the all-reduce traffic" describes a DGX node wrongly.

Check your understanding 1.3: How many 400 Gb/s compute ports does a DGX B200 have, and what is its total scale-out bandwidth? (Answer: 8 ports; $8 \times 400 \text{ Gb/s} = 3.2 \text{ Tb/s}$.)

1.4 NICs, DPUs and SuperNICs

NVIDIA uses three names for network adapters. The exam expects you to know which is which and where each one goes.

	ConnectX (e.g. ConnectX-7)	BlueField-3 DPU	BlueField-3 / ConnectX-8 SuperNIC
What it is	High-performance NIC (SmartNIC) with RDMA, GPUDirect and offloads	Data Processing Unit: a NIC plus on-board Arm cores and memory that can run its own OS and services	A network accelerator optimised for east-west AI traffic in Spectrum-X
Protocols	InfiniBand or Ethernet (VPI)	InfiniBand or Ethernet	Ethernet (Spectrum-X). The ConnectX-8 SuperNIC also supports InfiniBand (up to XDR)
Typical role	Compute fabric NIC (InfiniBand clusters); general RDMA NIC	North-south: storage, in-band, security, isolation, infrastructure offload (virtual switching, firewall, encryption)	East-west compute fabric in Spectrum-X: works with Spectrum-4 switches for adaptive routing, congestion control and performance isolation
Key Spectrum-X functions	—	—	Reorders packets sprayed by adaptive routing before placing them in memory (direct data placement); reacts to switch telemetry for congestion control
Programmability	Firmware and driver	Full, through DOCA	Through DOCA , in a SuperNIC-specific profile
Exam objectives	1.1, 3.x, 5.x	1.1, 2.7	1.1, 2.8, 2.2

The key distinction is simple:

- **DPU = infrastructure computer on a card.** It offloads, accelerates and *isolates* data centre infrastructure services, mostly north-south. Use it for multi-tenant cloud isolation, storage offload and zero-trust security.
- **SuperNIC = AI east-west accelerator.** It is built to move GPU-to-GPU RDMA traffic across a Spectrum-X fabric at full line rate with adaptive routing and congestion control. It usually has one 400 Gb/s (or faster) port per GPU and a smaller power budget than a full DPU.

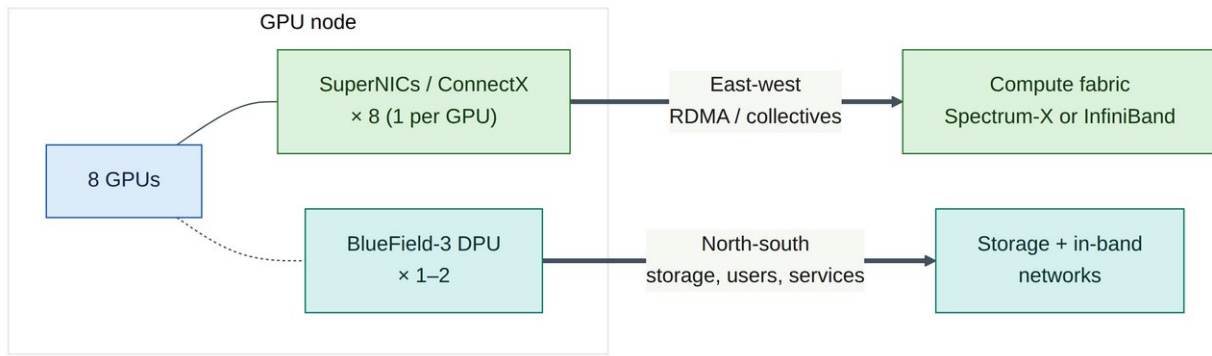


Figure 1.5: Where each adapter type sits

SuperNICs (or ConnectX in InfiniBand clusters) face the east-west compute fabric, one per GPU. BlueField-3 DPUs face the north-south networks: storage, in-band and the tenant edge.

Version note: NVIDIA's enterprise reference architectures name specific BlueField-3 models by role, for example B3140H (or ConnectX-7) for east-west and B3220 for north-south, and newer designs use **ConnectX-8 SuperNICs**. Model numbers change between generations. Learn the *roles*, not the part numbers.

Exam focus (1.1, 2.8): Know that **Spectrum-X = Spectrum-4 (or later) switches + SuperNICs working together**. Adaptive routing with packet spraying works end to end because the SuperNIC reorders packets at the receiver. A Spectrum-4 switch paired with ordinary NICs does not deliver the full Spectrum-X feature set.

Check your understanding 1.4: A tenant-isolation requirement says infrastructure services (virtual switching, firewalling, storage encryption) must run on the network card rather than the host CPU. Which adapter fits? *(Answer: a BlueField-3 DPU, because it has on-board Arm cores and runs DOCA services. A SuperNIC is optimised for east-west AI traffic, not infrastructure offload.)*

1.5 Switches: choosing the right family for each network

Switch family	Example model	Key characteristics	Typical role in an AI factory
NVIDIA Quantum-2 (InfiniBand NDR)	QM9700 / QM9790	64 × NDR 400 Gb/s ports in 32 OSFP cages; 51.2 Tb/s bidirectional; SHARPv3 in-network computing; QM9700 has an on-board Subnet Manager option, QM9790 is externally managed	Compute fabric leaves and spines (IB); IB storage fabric
NVIDIA Quantum-X800 (InfiniBand XDR)	Q3400-RA (Q3200-RA smaller)	144 × XDR 800 Gb/s ports in 72 OSFP cages (Q3200-RA: 72 ports in 36 cages)	Next-generation IB compute fabrics
NVIDIA Spectrum-4 (Ethernet)	SN5600	64 × 800 GbE OSFP ports (breakout to 128 × 400 GbE); 51.2 Tb/s; the Spectrum-X switch	Spectrum-X compute fabric; Ethernet storage fabric; high-speed in-band
NVIDIA Spectrum-3 (Ethernet)	SN4600C / SN4700	100/200/400 GbE data centre switches	In-band management, general Ethernet
NVIDIA Spectrum (1 GbE management)	SN2201	48 × 1 GbE RJ45 + 4 × 100 GbE uplinks	OOB management

Switch operating systems follow the family:

- **Spectrum Ethernet switches** run **Cumulus Linux**, configured with **NVUE** (Chapter 4), or **SONiC**. The exam blueprint focuses on Cumulus Linux and NVUE.
- **Quantum InfiniBand switches** run **MLNX-OS** (Quantum-2) or, on newer platforms, **NVOS** (with an NVUE-style CLI), for example the Quantum-X800 Q3200/Q3400 XDR switches. Either way, the fabric itself is controlled by a **Subnet Manager**, usually run on UFM or a managed switch (Chapter 11).

Exam trap: "The Quantum-2 switches compute InfiniBand routes, configured with NVUE" is a wrong option. InfiniBand routes are computed and programmed by the **Subnet Manager** (for example OpenSM running inside UFM), whatever CLI the switch runs. Quantum-2 switches run **MLNX-OS**. Newer NVIDIA InfiniBand switches (for example the Quantum-X800 generation) run **NVOS**, which has an NVUE-style CLI for switch management. On Spectrum Ethernet, NVUE is the Cumulus Linux configuration interface.

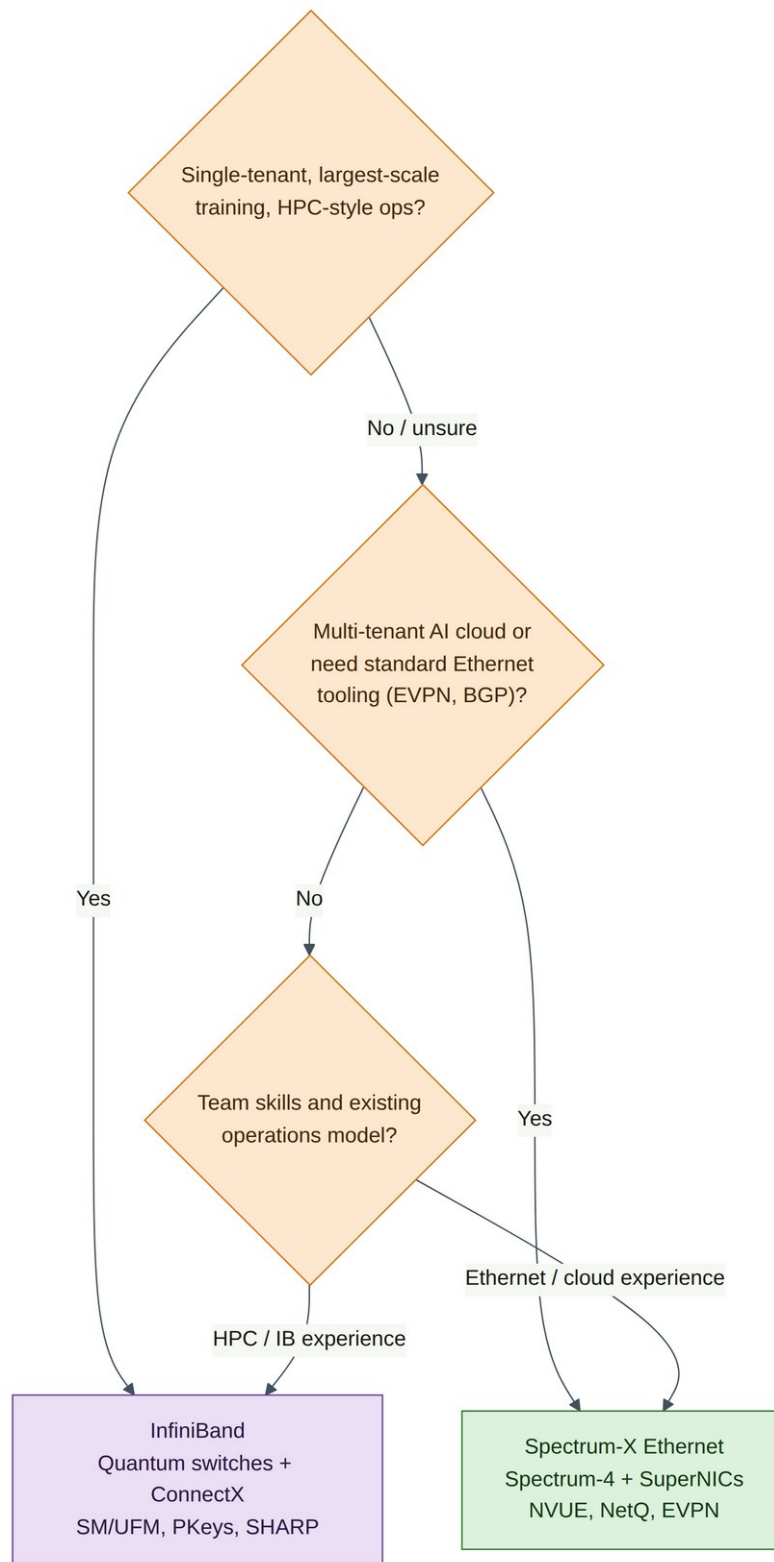


Figure 1.6: Choosing a compute fabric: InfiniBand or Spectrum-X

A simplified decision tree. Both are valid AI fabrics. The choice usually depends on multi-tenancy needs, operational skills and integration with existing Ethernet operations.

InfiniBand versus Spectrum-X at a glance

You will return to this table often. Each row becomes one or more chapters later in the book.

Concept	InfiniBand (Quantum)	Spectrum-X Ethernet
Transport	Native IB with RDMA; credit-based link-level flow	RoCEv2 over UDP/IP; lossless or near-lossless

Concept	InfiniBand (Quantum)	Spectrum-X Ethernet
	control, so lossless by design	through PFC, ECN and congestion control
Control plane	Centralised Subnet Manager (assigns LIDs, computes routes)	Distributed routing (BGP), with EVPN for overlays
Multi-tenancy	PKeys (partitions)	BGP-EVPN VRFs / VNIs
Adaptive routing	IB adaptive routing (SM-enabled)	Spectrum-X adaptive routing, with packet spraying and SuperNIC reordering
Congestion control	IB congestion control	ECN, CNP/DCQCN, and Spectrum-X telemetry-based congestion control
In-network computing	SHARP	—
Fabric management	UFM	NetQ , NVUE, Ansible
Addressing	LIDs (local), GUIDs (global)	MAC/IP

Check your understanding 1.5: Which management tool monitors link status and bandwidth on an InfiniBand fabric, and which gives real-time monitoring of a Spectrum-X fabric? (Answer: *UFM for InfiniBand (objective 3.4); NetQ for Spectrum-X (objective 2.6).*)

1.6 Scalable units: building AI factories from repeatable blocks

A **scalable unit (SU)** is a repeatable building block: a fixed number of GPU nodes together with the leaf switches, cabling pattern and rack layout that serve them. You grow the cluster by adding SUs and connecting them through a spine (and, for very large systems, core) layer.

In NVIDIA's DGX SuperPOD reference architectures for H100/H200/B200, **an SU is 32 DGX nodes**. The reference designs scale to four SUs (about 127–128 nodes and around 1,000 GPUs), with one node position used for UFM in the largest configuration. Newer rack-scale systems define the SU in racks instead: a DGX GB200 SU is **8 NVL72 racks** (576 GPUs). SU sizes change between generations, so check the reference architecture for your platform.

Why build in SUs?

- **Predictable performance.** Every SU has the same bandwidth, hop count and oversubscription (non-blocking in the compute fabric).
- **Faster deployment.** Rack layouts, cable maps and validation tests are reused. Cabling errors are much easier to spot when every SU looks identical.
- **Rail alignment.** Inside an SU, NIC *n* of all 32 nodes connects to leaf *n*. The SuperPOD reference architecture describes traffic on each rail as always **one hop away** from the other 31 nodes in the SU.

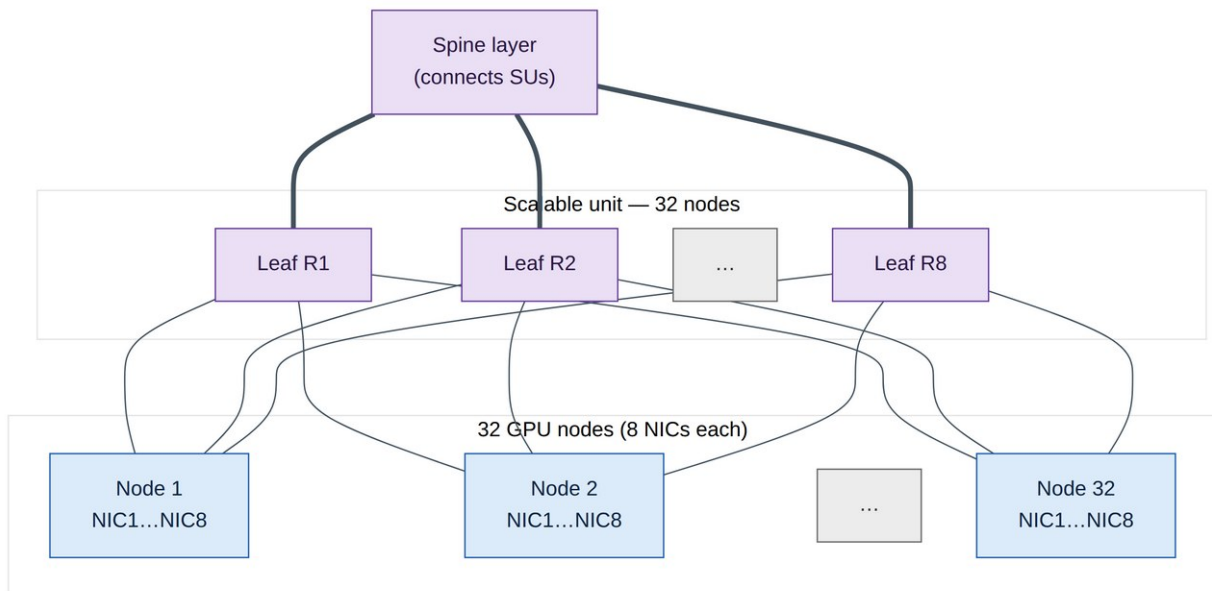


Figure 1.7: One scalable unit (compute fabric view)

32 nodes $\times$ 8 rails. Rail leaf n connects NIC n of every node in the SU, so same-rail traffic between any two nodes in the SU crosses exactly one switch. Leaf uplinks to the spine layer connect SUs together.

Sizing an SU: a worked example

Take a Helix-A SU built on Quantum-2 QM9700 switches (64 $\times$ NDR 400 Gb/s ports):

1. **Endpoints:** 32 nodes $\times$ 8 NICs = **256 compute ports**.
2. **Leaves:** one leaf per rail, so **8 leaves**. Each leaf connects 32 node ports (one per node).
3. **Non-blocking:** each leaf has 32 downlinks, so it needs **32 uplinks** to the spine layer (32 + 32 = 64 ports, a full QM9700).
4. **Two SUs:** 16 leaves $\times$ 32 uplinks = **512 spine-facing ports**. 512 $\div$ 64 = **8 spines** in a two-tier fat-tree.

Quantity	Per SU	Helix-A (2 SUs)
GPU nodes	32	64
GPUs	256	512
Compute NIC ports	256	512
Leaf switches (1 per rail)	8	16
Leaf uplinks	256	512
Spine switches (64-port)	—	8

Field note: Real reference architectures add details this simplified model leaves out, such as spine groups aligned to rails, reserved ports for UFM and management appliances, and a core layer beyond a certain size. Always build a production bill of materials from NVIDIA's reference architecture cable plans.

Check your understanding 1.6: A non-blocking leaf built on a 64-port switch serves one rail of an SU. How many GPU nodes can it connect? (Answer: 32. Half the ports face the nodes and half face the spines.)

1.7 The management and software plane

The fabrics carry data. A set of software components configures, monitors and automates them, and each one is an exam objective elsewhere in the book. Figure 1.8 gives the map.

Component	What it does	Fabric	Chapter / objective
Subnet Manager (SM)	Discovers the IB fabric, assigns LIDs, computes and programs routing, manages partitions and QoS	InfiniBand	11, 13, 14 / 3.1–3.3
UFM (Unified Fabric Manager)	Fabric management, monitoring, health, telemetry and HA for InfiniBand; hosts the SM	InfiniBand	12, 15, 20 / 3.1, 3.4, 5.4
NVUE	Object-based CLI and REST API for Cumulus Linux	Spectrum Ethernet	4, 21 / 2.1, 6.1
NetQ	Real-time monitoring, validation, congestion and latency insight	Spectrum Ethernet	10 / 2.6
What Just Happened (WJH)	Switch ASIC-level drop reasons in real time	Spectrum Ethernet	10, 18 / 2.5, 5.2
NVIDIA Air	Cloud-hosted digital twin for simulating the network	Spectrum Ethernet	9 / 2.4
DOCA	SDK and runtime for BlueField DPUs and SuperNICs	Adapters	8 / 2.7, 2.8
NVIDIA Network Operator	Deploys drivers, device plugins and CNIs so Kubernetes pods can use RDMA and IB	Kubernetes	16 / 4.1, 4.2
Ansible	Automation of switch configuration through NVUE	Spectrum Ethernet	22 / 6.2
Base Command Manager / Mission Control	Cluster provisioning and workload management (not a blueprint objective)	Cluster	Context only

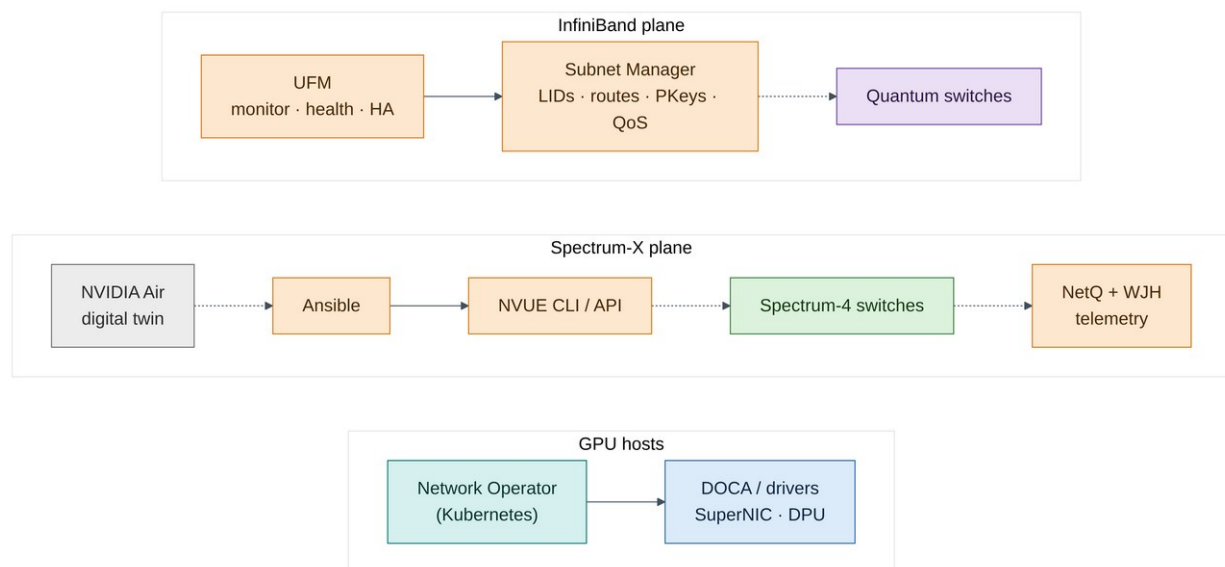


Figure 1.8: Management and software map

InfiniBand is managed centrally (SM and UFM). Spectrum-X is managed per switch through NVUE and observed fabric-wide through NetQ and WJH. The Network Operator connects Kubernetes workloads to both fabrics.

Check your understanding 1.7: Which component assigns LIDs in an InfiniBand fabric: UFM, the Subnet Manager or MLNX-OS? (Answer: the Subnet Manager. UFM usually hosts and manages the SM, but LID assignment is the SM's job.)

1.8 Putting it together: the Helix AI Factory

Now go back to the master topology (Figure 0.4) and read it with everything from this chapter:

1. **Helix-A** is a single-tenant training cluster, so InfiniBand is a natural choice: SM-controlled routing, SHARP for collectives, PKeys to separate research from production. It has 2 SUs × 32 nodes, 8 rails, 16 leaves and 8 spines, all NDR 400 Gb/s, managed by a UFM HA pair.
2. **Helix-B** serves two internal tenants (AURORA and BOREALIS) and must fit into Helix's existing Ethernet/EVPN operations, so it uses Spectrum-X: SN5600 switches plus BlueField-3 SuperNICs, rail-optimized, EVPN VRFs per tenant, monitored by NetQ.
3. **Every node** also connects to the storage fabric, the in-band management network and the OOB network. None of that traffic ever touches the compute fabrics.
4. **Kubernetes** runs on both clusters, with the NVIDIA Network Operator giving pods access to RDMA interfaces.

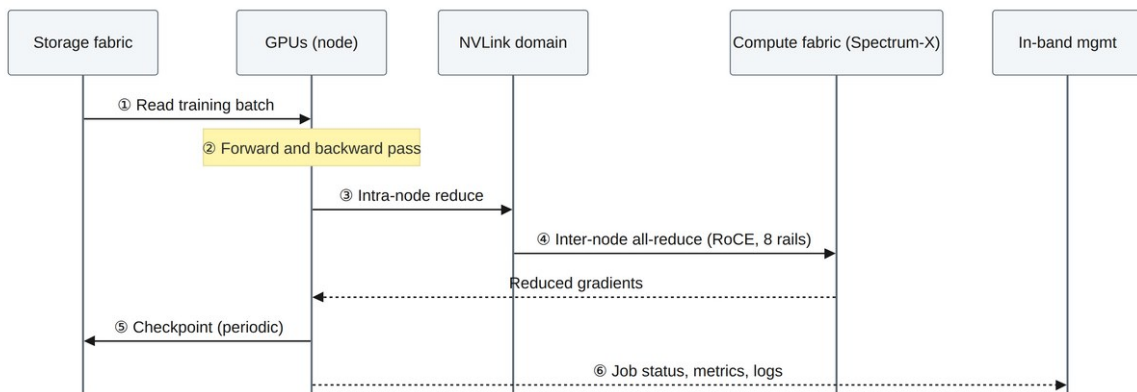


Figure 1.9: Traffic-to-network mapping for a single Helix-B training step

Following one training step shows which network carries each traffic type. ① Data loader reads a batch (storage). ② GPUs compute. ③ NVLink all-reduce inside the node. ④ Cross-node all-reduce on the compute fabric. ⑤ Periodic checkpoint to storage. ⑥ Scheduler and metrics on in-band.

Hands-on lab 1 — Network inventory of a GPU node (reference walkthrough)

Objective: Identify a GPU node's network adapters, map each GPU to its compute NIC and rail, and confirm link state, using standard host tools you will use again in Chapters 3, 17 and 19.

Environment: Any Linux GPU server with NVIDIA GPUs and ConnectX/BlueField adapters and the NVIDIA driver and DOCA-OFED/MLNX_OFED installed. If you don't have one, read the documented outputs below. They are **illustrative** and trimmed. Exact formatting varies by driver version.

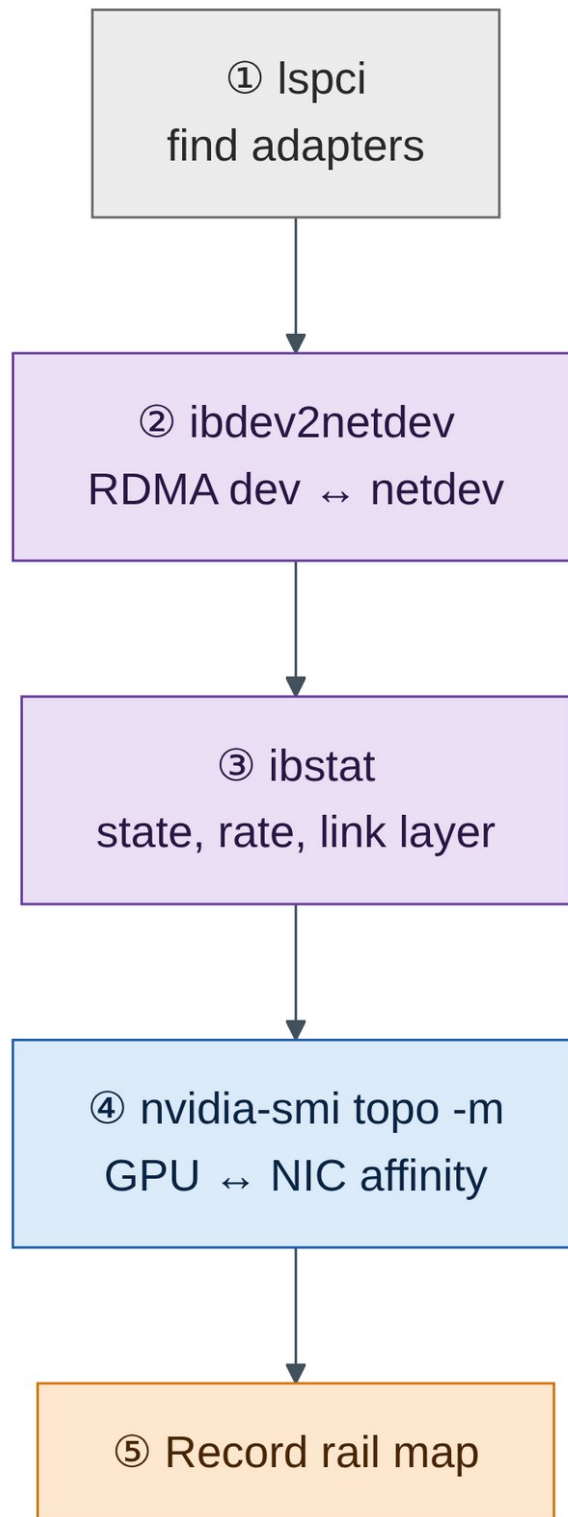


Figure 1.10: Lab 1 flow

Work from hardware up to logical mapping: list the adapters, map RDMA devices to interfaces, check link state, then read the GPU-NIC topology matrix to confirm 1:1 affinity.

Task 1 — List NVIDIA network adapters

```

hxa-sul-gpu01:~$ lspci | grep -i -E "mellanox|nvidia.*(connectx|bluefield)"
# Illustrative output (trimmed):
1a:00.0 Infiniband controller: Mellanox Technologies MT2910 Family [ConnectX-7]
3c:00.0 Infiniband controller: Mellanox Technologies MT2910 Family [ConnectX-7]
  
```

```

...                                # 8 × ConnectX-7 in total (compute)
aa:00.0 Ethernet controller: Mellanox Technologies MT43244 BlueField-3 integrated ConnectX-7
network controller
aa:00.1 Ethernet controller: Mellanox Technologies MT43244 BlueField-3 integrated ConnectX-7
network controller
...                                # BlueField-3 ports (storage / in-band)

```

What to notice: eight compute ConnectX-7 functions and separate BlueField-3 functions. Vendor strings often still say "Mellanox Technologies". NVIDIA acquired Mellanox, and PCI IDs keep the old name.

Task 2 — Map RDMA devices to network interfaces

```

hxa-sul-gpu01:~$ ibdev2netdev
mlx5_0 port 1 ==> ibp26s0 (Up)
mlx5_1 port 1 ==> ibp60s0 (Up)
mlx5_2 port 1 ==> ibp77s0 (Up)
mlx5_3 port 1 ==> ibp94s0 (Up)
mlx5_4 port 1 ==> ibp156s0 (Up)
mlx5_5 port 1 ==> ibp188s0 (Up)
mlx5_6 port 1 ==> ibp204s0 (Up)
mlx5_7 port 1 ==> ibp220s0 (Up)
mlx5_8 port 1 ==> enp170s0f0np0 (Up)    # BlueField-3 port 1 (storage)
mlx5_9 port 1 ==> enp170s0f1np1 (Up)    # BlueField-3 port 2 (in-band)

```

What to notice: `mlx5_0-mlx5_7` are InfiniBand interfaces (`ib...`), the eight compute rails. The BlueField ports show as Ethernet (`enp...`). NCCL and perfest tools refer to devices by these `mlx5_N` names.

Task 3 — Check port state and rate

```

hxa-sul-gpu01:~$ ibstat mlx5_0
CA 'mlx5_0'
  CA type: MT4129
  Number of ports: 1
  Firmware version: 28.xx.xxxx
  Port 1:
    State: Active
    Physical state: LinkUp
    Rate: 400
    Base lid: 17
    SM lid: 1
    Port GUID: 0x...
    Link layer: InfiniBand

```

What to notice: `State: Active` means the Subnet Manager has configured the port. `Physical state: LinkUp` means the cable and PHY are good. `Rate: 400` is NDR. `Base lid` is this port's local address, assigned by the SM at `SM lid: 1`. Chapter 11 explains the IB port states. Chapter 17 covers the faults these fields reveal (for example `State: Initializing` means no SM has configured the port).

Task 4 — Read the GPU-NIC topology matrix

```

hxa-sul-gpu01:~$ nvidia-smi topo -m
      GPU0  GPU1  ...  GPU7  NIC0  NIC1  ...  NIC7
GPU0      X   NV18  ...  NV18  PIX   SYS   ...  SYS
GPU1     NV18    X   ...  NV18  SYS   PIX   ...  SYS
...
GPU7     NV18  NV18  ...    X   SYS   SYS   ...  PIX

```

Legend:

X = Self
 SYS = Connection traversing PCIe as well as the SMP interconnect between NUMA nodes
 NODE = Connection traversing PCIe and the interconnect between PCIe host bridges within a NUMA node
 PHB = Connection traversing PCIe as well as a PCIe host bridge (typically the CPU)
 PXB = Connection traversing multiple PCIe bridges (without traversing the PCIe host bridge)
 PIX = Connection traversing at most a single PCIe bridge
 NV# = Connection traversing a bonded set of # NVLinks

NIC Legend:

NIC0: mlx5_0
 NIC1: mlx5_1
 ...

What to notice: each GPU has exactly one NIC marked **PIX** (same PCIe switch). That is its local rail NIC. GPU-to-GPU entries show **NV#** (NVLink). A GPU whose nearest NIC shows **SYS** or **PHB** means traffic must cross the CPU. That is a misconfiguration or a non-reference server design, and it reduces GPUDirect RDMA performance. On H100, H200 and B200 systems each GPU has 18 NVLinks, so the entry is **NV18**.

Task 5 — Record the rail map

Fill in a table like the one below for your node. Chapter 2 uses it to trace an all-reduce across the rails.

GPU	Local NIC (PIX)	RDMA device	Interface	Rail / leaf
GPU0	NIC0	mlx5_0	ibp26s0	Rail 1 → hxa-su1-leaf-r1
GPU1	NIC1	mlx5_1	ibp60s0	Rail 2 → hxa-su1-leaf-r2
...	...	...	...	...
GPU7	NIC7	mlx5_7	ibp220s0	Rail 8 → hxa-su1-leaf-r8

Common mistakes

- Assuming **mlx5_N** numbering matches GPU numbering. It usually does in reference systems, but always confirm with `nvidia-smi topo -m`.
- Mixing up BlueField ports (storage and in-band) with compute ports.
- Reading **Physical state: LinkUp** as "port working". The logical **State** must also be **Active**.

Troubleshooting scenario — "One node makes every job slow"

Symptom: After a maintenance window, multi-node jobs on Helix-A that include **hxa-su1-gpu07** run noticeably slower. Jobs that exclude that node run normally. All eight ports on **hxa-su1-gpu07** show **Active** at rate 400.

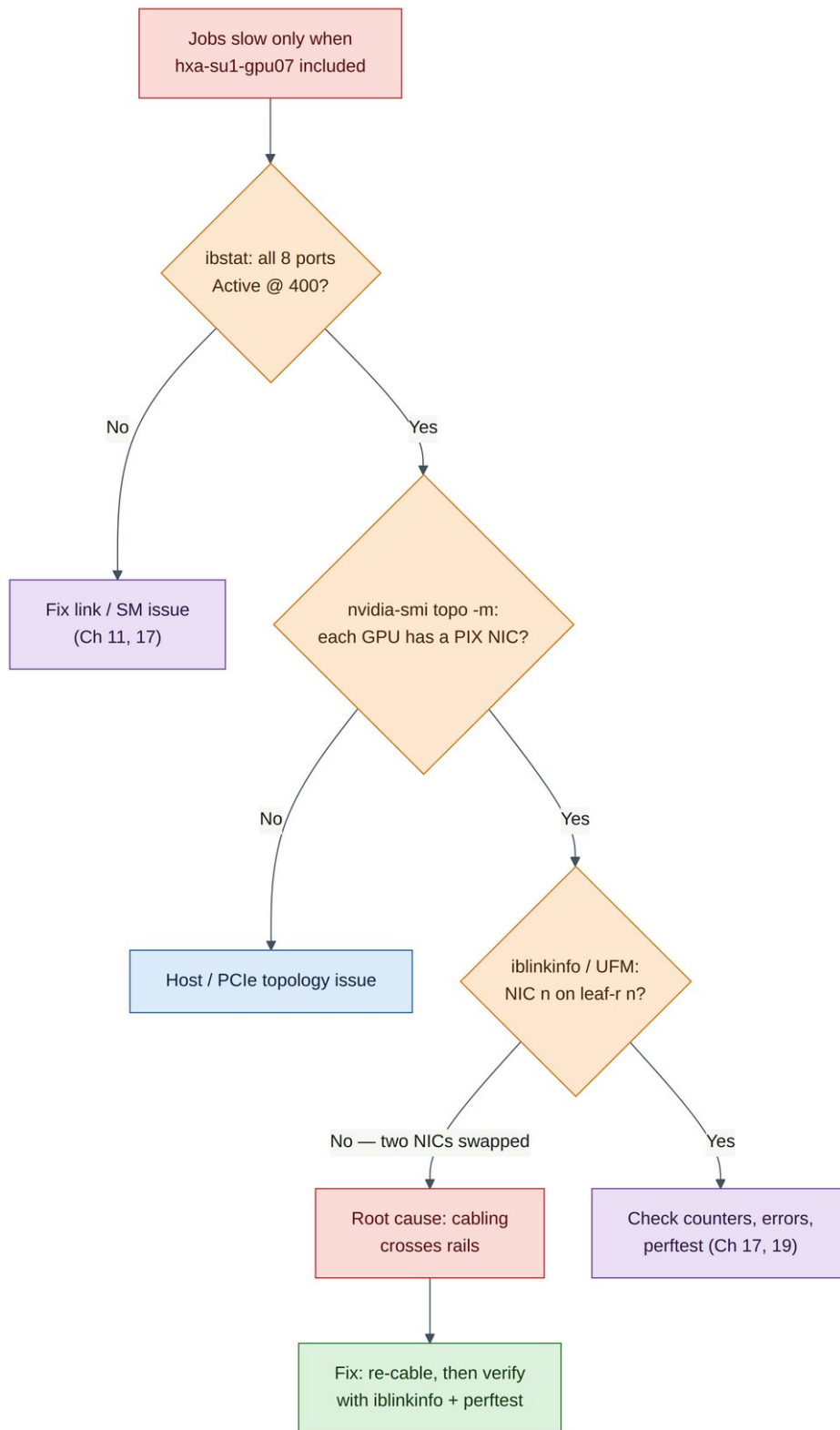


Figure 1.11: Troubleshooting flowchart — one slow node

Work from host to fabric. Links that are up at full rate can still be wired to the wrong rail, which forces same-rail traffic through the spine layer.

Investigation:

1. `ibstat` on the node shows all ports **Active**, 400 Gb/s, so it is not a link-down or speed problem.
2. `nvidia-smi topo -m` shows the expected PIX mapping, so the host PCIe topology is fine.

3. `iblinkinfo` (Chapter 17), or the UFM cable view, shows that `mlx5_2` (GPU2, rail 3) connects to `hxa-su1-leaf-r4`, and `mlx5_3` (GPU3, rail 4) connects to `hxa-su1-leaf-r3`. The two cables were swapped during maintenance.

Root cause: GPU2 traffic on this node now reaches rail 4. NCCL expects GPU2 on every node to share rail 3, so traffic that should take one hop must now go leaf → spine → leaf. That adds latency and consumes spine bandwidth, and the whole job slows to the pace of this node.

Fix: Swap the cables back and re-verify with `iblinkinfo` (or the UFM topology view) and a perftest run (Chapter 19).

Field note: Rail-crossing cabling errors are among the most common day-one problems in new AI clusters, because every link "works". Validate cabling against the planned topology before handing a cluster to users. UFM and `ibdiagnet` can compare the discovered topology to an expected one (Chapters 17 and 20).

Chapter quick reference

Command / tool	Purpose	Chapter where it is covered in depth
<code>lspci grep -i mellanox</code>	List NVIDIA (Mellanox) adapters	3
<code>ibdev2netdev</code>	Map <code>mlx5_N</code> RDMA devices to network interfaces	3, 17
<code>ibstat</code>	Adapter and port state, rate, LID, link layer	11, 17
<code>nvidia-smi topo -m</code>	GPU ↔ GPU and GPU ↔ NIC topology and affinity	2, 19
<code>iblinkinfo</code>	Fabric-wide link list: what connects to what, speed and width	17
UFM	IB fabric management, monitoring, health	12, 15, 20
NetQ	Spectrum fabric monitoring	10

Chapter FAQ

Q1. Can I run training traffic over the in-band management network if the compute fabric is down?

You technically can for small experiments, because NCCL can fall back to TCP sockets. It is not a supported production design, and performance will be far lower. The in-band network is not designed to be lossless or non-blocking, and its bandwidth per node is a fraction of the compute fabric's. Treat a compute fabric outage as a cluster outage for multi-node jobs.

Q2. Why eight compute NICs rather than two very fast ones?

One NIC per GPU keeps each GPU's data path local (same PCIe switch), avoids CPU crossings and creates eight independent rails. Rails let collective algorithms keep most traffic one hop away. Two fast NICs shared by eight GPUs would create PCIe and NUMA bottlenecks and remove rail alignment.

Q3. Is Spectrum-X "just Ethernet"?

It is standards-based Ethernet running RoCEv2, but Spectrum-X is an *end-to-end platform*: Spectrum-4 switches and SuperNICs cooperate on adaptive routing (with receiver-side reordering), telemetry-driven congestion control and performance isolation. Generic Ethernet switches and NICs won't provide those features together.

Q4. Does the BlueField-3 in a DGX node do anything for GPU training traffic?

Not for the inter-node collectives. Those go through the eight compute NICs. The BlueField-3 DPUs serve storage and in-band networking. Training is still faster because data loading and checkpoints are efficient, but the DPUs are not on the all-reduce path.

Q5. Do I need UFM if I already run a Subnet Manager on a switch?

A switch-hosted SM can run a small fabric. UFM adds fabric-wide monitoring, health analysis, telemetry, event management and HA for the SM. The NCP-AIN blueprint expects UFM skills (objectives 3.1, 3.4 and 5.4), and NVIDIA's reference architectures include UFM.

Q6. What is the difference between "scale-up" and "scale-out"?

Scale-up is GPU-to-GPU communication inside a tightly coupled NVLink domain (a node, or an NVL72 rack). Scale-out is communication between those domains across the compute fabric (InfiniBand or Spectrum-X). NCCL uses both: NVLink first, then the fabric.

Q7. Why does the storage fabric allow some oversubscription when the compute fabric doesn't?

Storage traffic is less synchronised and more tolerant of small delays than collective operations. NVIDIA's reference architecture describes storage links as slightly oversubscribed (for example 4:3), which saves cost without limiting training.

Q8. Where does NVIDIA Air fit into the architecture?

It doesn't sit in the production data path. NVIDIA Air is a cloud-hosted simulation ("digital twin") of Spectrum/Cumulus networks that you use to design, test changes and train, before touching production (objective 2.4, Chapter 9).

Q9. How do I know whether a node is IB or Ethernet on the compute side?

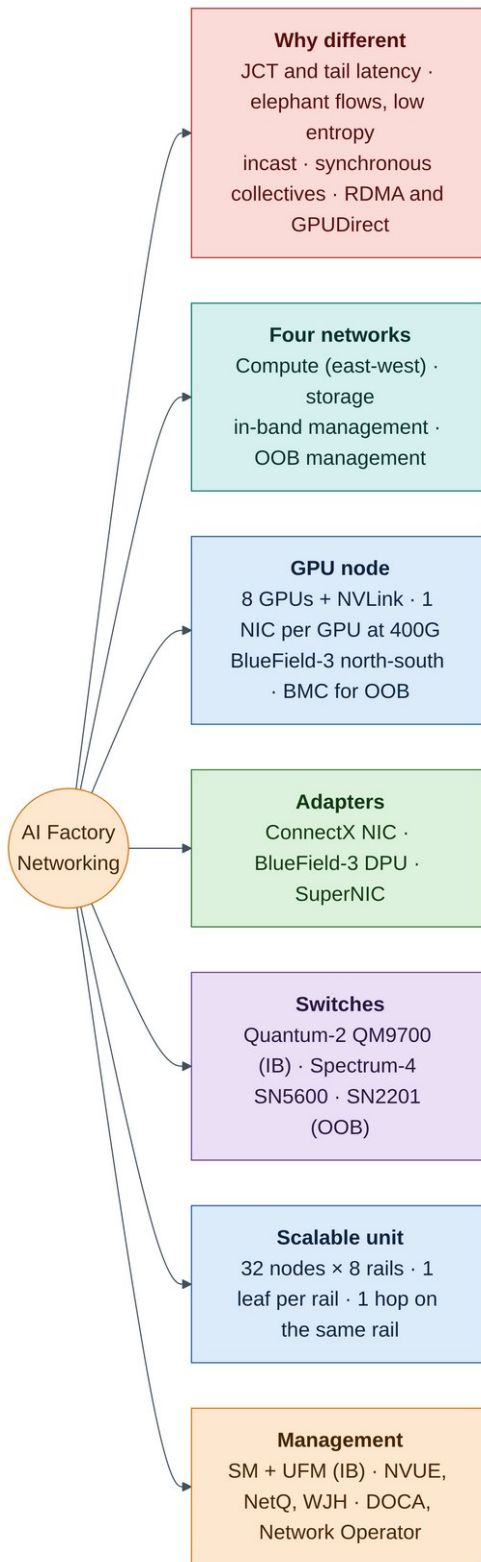
Run `ibstat` and check `Link layer:` (InfiniBand or Ethernet). Or run `ibdev2netdev: ib...` interface names indicate InfiniBand, `en...` indicates Ethernet. ConnectX-7 is a VPI adapter, so the port type is a configuration choice.

Q10. Is a scalable unit always 32 nodes?

No. 32 nodes is the SU in DGX SuperPOD designs for H100/H200/B200-class systems. Other reference architectures use different sizes (for example 4-node increments in some enterprise RAs, or a full NVL72 rack). Learn the *concept*: a repeatable, pre-validated block of compute plus its leaf switches.

Chapter summary

- An **AI factory** is a data centre designed as one computer. Its network goal is to minimise **job completion time**, which depends on **tail latency**, because synchronous collectives wait for the slowest flow.
- AI traffic is **east-west, low-entropy, elephant-flow, bursty and loss-sensitive**, so standard ECMP and lossy Ethernet perform poorly.
- AI factories use **separate networks**: compute (east-west), storage, in-band management and OOB management, plus the NVLink scale-up domain inside nodes.
- Each GPU has **its own 400 Gb/s compute NIC** on the same PCIe switch (1:1), giving 3.2 Tb/s per 8-GPU node and creating **rails**.
- **ConnectX** = high-performance NIC. **BlueField-3 DPU** = NIC plus Arm compute for north-south infrastructure offload and isolation. **SuperNIC** = east-west AI accelerator for Spectrum-X.
- **Quantum** switches (IB) and **Spectrum-4 SN5600** (Ethernet) build compute fabrics. **SN2201** serves OOB. Spectrum switches run Cumulus Linux/NVUE. IB fabrics are driven by the **Subnet Manager** and managed with **UFM**.
- A **scalable unit** is a repeatable block of nodes plus leaves. In DGX SuperPOD it is 32 nodes with one leaf per rail, so same-rail traffic is one hop.

**Figure 1.12: Chapter 1 visual recap***The whole chapter on one page.*

Further reading

- [REF: NVIDIA DGX SuperPOD Reference Architecture (B200/H200) — "Network Fabrics" and "Key Components" sections]
- [REF: NVIDIA DGX B200 User Guide — "Introduction to NVIDIA DGX B200 Systems" (network ports and roles)]

- [REF: NVIDIA Enterprise Reference Architecture Overview — "Reference Architectures: Deep Dive" (SuperNIC vs DPU roles, SN5600/SN5610)]
- [REF: BlueField-3 Administrator Quick-Start Guide (study-guide resource for Domain 1)]
- [REF: InfiniBand Essentials — NVIDIA Academy (study-guide resource for Domain 1)]
- [REF: NVIDIA Spectrum-X whitepaper and datasheets]

Figure index

Figure	Title	Type	Format
1.1	Training step: shared vs dedicated fabric	Comparison / timeline	Mermaid gantt
1.2	Traditional vs AI traffic patterns	Comparison	Mermaid flowchart
1.3	The four networks of an AI factory	Architecture	Mermaid flowchart
1.4	GPU node internals	Node internals	Mermaid flowchart
1.5	Where each adapter type sits	Architecture	Mermaid flowchart
1.6	Choosing a compute fabric	Decision tree	Mermaid flowchart
1.7	One scalable unit	Rail-optimized topology	Mermaid flowchart
1.8	Management and software map	Architecture	Mermaid flowchart
1.9	Traffic-to-network mapping for a training step	Sequence	Mermaid sequence
1.10	Lab 1 flow	Flowchart	Mermaid flowchart
1.11	One-slow-node troubleshooting	Troubleshooting flowchart	Mermaid flowchart
1.12	Chapter visual recap	Mind map (tree)	Mermaid flowchart

Chapter 1 — Q&A Pack

A. Objective map

Objective / learning outcome	Questions
1.1a Explain why AI traffic needs purpose-built fabrics	B1–B5, C1, C5, D1
1.1b Identify the networks of an AI factory and the traffic each carries	B6–B13, C2, C7, D2, F1
1.1c Describe GPU node network components and GPU-to-NIC affinity	B14–B21, C3, C6, C8, D3, F2
1.1d Tell ConnectX NICs, BlueField DPUs and SuperNICs apart	B22–B27, C4, D4, F3
1.1e Select switches and management components for each fabric	B28–B35, B42–B44, C9, D5, F4, F5
1.1f Explain and size scalable units	B36–B41, C10, D6, Challenge lab

B. Exam-style question bank

Each question is tagged [Type] [Difficulty] [Objective] [Section]. F = Foundation, P = Practitioner, E = Expert.

Section 1.1 — Why AI networking is different

B1. [MCQ-single] [F] [1.1] [1.1]

A synchronous distributed training job runs across 64 GPU nodes. Which network characteristic most directly limits its job completion time?

- A. Average link utilisation across the fabric
- B. Tail latency: the slowest transfer in each collective step
- C. The number of MAC addresses in the forwarding table
- D. The maximum number of VLANs supported by the leaf switches

B2. [MCQ-multi] [P] [1.1] [1.1]

Which TWO characteristics of AI training traffic make hash-based ECMP load balancing less effective than it is for traditional data centre traffic? (Choose TWO.)

- A. Low flow entropy (few distinct 5-tuples)
- B. Very large, long-lived flows
- C. A very high number of short-lived flows
- D. Traffic that is mostly north-south
- E. Use of IPv6 addressing

B3. [MCQ-single] [P] [1.1] [1.1]

Why is a single dropped packet so expensive for RDMA-based collective operations?

- A. The switch must reload its forwarding table after a drop.
- B. The transfer stalls while the NIC recovers the loss by retransmitting, and every GPU in the collective waits for the slowest transfer.
- C. NCCL permanently falls back to TCP after the first drop.
- D. The GPU resets its memory to guarantee consistency.

B4. [MCQ-single] [F] [1.1] [1.1]

Which technology lets a network adapter read and write GPU memory directly, without first copying the data through host system memory?

- A. GPUDirect Storage
- B. GPUDirect RDMA
- C. SR-IOV
- D. NVLink

B5. [MCQ-single] [F] [1.1] [1.1]

At the end of each training step, many GPUs send data towards the same destination at the same moment. What is this traffic pattern called?

- A. Broadcast storm
- B. Incast
- C. Hairpinning
- D. Microsegmentation

Section 1.2 — The networks of an AI factory

B6. [Scenario] [F] [1.1] [1.2]

Users on Helix-A report that model checkpoints take much longer to write than last week. GPU-to-GPU collective performance is normal. Which network should you investigate first?

- A. Compute fabric
- B. Storage fabric
- C. Out-of-band management network
- D. NVLink domain

B7. [Scenario] [F] [1.1] [1.2]

After a kernel panic, `hxb-gpu09` stops responding to SSH. You need to power-cycle it remotely through its BMC. Which network do you use?

- A. In-band management
- B. Compute fabric
- C. Out-of-band management
- D. Storage fabric

B8. [Scenario] [F] [1.1] [1.2]

A training job starts by pulling a container image from a registry outside the cluster. Which network carries the image download?

- A. Compute fabric
- B. In-band management network
- C. Out-of-band management network
- D. NVLink domain

B9. [MCQ-multi] [P] [1.1] [1.2]

Which TWO are valid reasons to keep the compute fabric physically separate from the storage fabric? (Choose TWO.)

- A. Storage bursts cannot cause congestion that delays collective operations.
- B. The compute fabric has different design rules (lossless, rail-aligned, non-blocking).
- C. It reduces the total number of cables in the data centre.
- D. NVLink requires a separate storage network to function.
- E. It allows all servers to share one IP subnet.

B10. [MCQ-single] [F] [1.1] [1.2]

Which traffic is carried by the compute (east-west) fabric?

- A. BMC Redfish sessions

- B. NCCL all-reduce between GPUs in different nodes
- C. DNS and NTP for the cluster
- D. User SSH sessions to the login node

B11. [MCQ-single] [P] [1.1] [1.2]

GPU0 and GPU5 are in the **same** DGX B200 node and exchange data during an all-reduce. Which path does this traffic take?

- A. ConnectX-7 NIC → rail leaf → ConnectX-7 NIC
- B. NVLink / NVSwitch inside the node
- C. BlueField-3 DPU → storage fabric
- D. PCIe → CPU → system memory → PCIe

B12. [MCQ-single] [P] [1.1] [1.2]

In NVIDIA's DGX SuperPOD reference architecture, which network is described as being slightly oversubscribed (for example 4:3) without harming training performance?

- A. Compute fabric
- B. Storage fabric
- C. NVLink domain
- D. None — every network must be non-blocking

B13. [MCQ-single] [F] [1.1] [1.2]

Which network must stay reachable even when the in-band management network is down?

- A. Compute fabric
- B. Storage fabric
- C. Out-of-band management network
- D. Tenant EVPN overlay

Section 1.3 — Inside a GPU node

B14. [MCQ-single] [F] [1.1] [1.3]

How many ConnectX-7 ports does a DGX B200 dedicate to the compute fabric?

- A. 2
- B. 4
- C. 8
- D. 16

B15. [MCQ-single] [F] [1.1] [1.3]

A node has 8 GPUs, each with a dedicated 400 Gb/s compute NIC. What is the node's total scale-out bandwidth?

- A. 400 Gb/s
- B. 800 Gb/s
- C. 1.6 Tb/s
- D. 3.2 Tb/s

B16. [MCQ-single] [P] [1.1] [1.3]

What is the primary role of the two BlueField-3 DPUs in a DGX B200?

- A. Carrying the eight rails of compute fabric traffic
- B. Storage and in-band management connectivity
- C. Running the InfiniBand Subnet Manager
- D. Providing NVLink connectivity between GPUs

B17. [Output] [P] [1.1] [1.3]

Review this excerpt of `nvidia-smi topo -m`:

	GPU3	NIC2	NIC3	NIC4
GPU3	X	SYS	PIX	SYS

What does the **PIX** entry tell you?

- A. GPU3 and NIC3 are connected through NVLink.
- B. GPU3 and NIC3 share a PCIe switch (at most one PCIe bridge), so NIC3 is GPU3's local rail NIC.
- C. Traffic between GPU3 and NIC3 crosses the CPU interconnect.
- D. NIC3 is disabled.

B18. [Output] [E] [1.1] [1.3]

On a newly delivered, non-reference GPU server, `nvidia-smi topo -m` shows the nearest NIC to GPU4 as **SYS**. What is the most likely effect?

- A. None. **SYS** is the best possible affinity.
- B. GPU4's RDMA traffic must cross the inter-socket (SMP) interconnect, which reduces GPUDirect RDMA performance.
- C. GPU4 can't use NVLink.
- D. GPU4 traffic uses the out-of-band network.

B19. [Output] [P] [1.1] [1.3]

`ibstat mlx5_2` shows **State: Initializing** and **Physical state: LinkUp**. What does this indicate?

- A. The cable is faulty.
- B. The physical link is up, but the Subnet Manager has not yet configured the port to Active.
- C. The port is running Ethernet instead of InfiniBand.
- D. The port is administratively disabled on the NIC.

B20. [Output] [F] [1.1] [1.3]

Given this output, which RDMA devices are the node's compute-fabric rails?

```
mlx5_0 port 1 ==> ibp26s0 (Up)
...
mlx5_7 port 1 ==> ibp220s0 (Up)
mlx5_8 port 1 ==> enp170s0f0np0 (Up)
mlx5_9 port 1 ==> enp170s0f1np1 (Up)
```

- A. mlx5_8 and mlx5_9
- B. mlx5_0 to mlx5_7
- C. All ten devices
- D. None. InfiniBand interfaces don't appear in `ibdev2netdev`.

B21. [Output] [P] [1.1] [1.3]

On a Helix-B node, `ibstat` for a compute-port ConnectX/SuperNIC device shows **Link layer: Ethernet**. What does this mean?

- A. The adapter is faulty. Compute ports must always show InfiniBand.
- B. The port is running Ethernet (RoCE), which is expected for a Spectrum-X compute fabric.
- C. The Subnet Manager has assigned an Ethernet LID.
- D. The port is connected to the OOB network.

Section 1.4 — NICs, DPUs and SuperNICs

B22. [MCQ-single] [F] [1.1] [1.4]

Which adapter has on-board Arm cores and memory, so it can run infrastructure services such as virtual switching and firewalling independently of the host CPU?

- A. ConnectX-7 NIC
- B. BlueField-3 DPU
- C. NVSwitch
- D. OSFP transceiver

B23. [MCQ-single] [P] [1.1, 2.8] [1.4]

In a Spectrum-X fabric, adaptive routing sprays the packets of one flow across multiple paths. Which component ensures the data is placed correctly in memory despite out-of-order arrival?

- A. The spine switch
- B. The SuperNIC at the receiver
- C. The Subnet Manager
- D. NetQ

B24. [MCQ-multi] [F] [1.1] [1.4]

Which TWO components make up the NVIDIA Spectrum-X platform's end-to-end compute fabric? (Choose TWO.)

- A. Spectrum-4 (e.g. SN5600) Ethernet switches
- B. BlueField-3 or ConnectX-8 SuperNICs
- C. Quantum-2 InfiniBand switches
- D. UFM Enterprise
- E. NVSwitch chips

B25. [Scenario] [P] [1.1] [1.4]

A GPU-cloud provider needs to offload tenant isolation, storage encryption and a distributed firewall from the host CPU to the network card. Which adapter type fits best?

- A. ConnectX-7 in InfiniBand mode
- B. BlueField-3 DPU
- C. SuperNIC dedicated to east-west traffic
- D. Any 1 GbE NIC

B26. [MCQ-single] [F] [1.1, 2.7] [1.4]

Which NVIDIA software framework is used to program and run services on BlueField DPUs and SuperNICs?

- A. NCCL
- B. DOCA
- C. CUDA
- D. NVUE

B27. [MCQ-single] [F] [1.1] [1.4]

In Spectrum-X reference designs, how are SuperNICs typically deployed in a GPU node?

- A. One per node, facing the storage network
- B. One per GPU, facing the east-west compute fabric
- C. One per rack, facing the OOB network
- D. Only in the spine switches

Section 1.5 — Switches and management

B28. [MCQ-single] [F] [1.1] [1.5]

Which NVIDIA switch is typically used for the 1 GbE out-of-band management network in DGX SuperPOD designs?

- A. SN5600
- B. QM9700
- C. SN2201
- D. Q3400

B29. [MCQ-single] [F] [1.1] [1.5]

Which description matches the Spectrum-4 SN5600?

- A. 48 × 1 GbE ports with 4 × 100 GbE uplinks
- B. 64 × 800 GbE OSFP ports (51.2 Tb/s), usable as 128 × 400 GbE with breakout
- C. 64 × NDR 400 Gb/s InfiniBand ports
- D. 32 × 100 GbE QSFP28 ports

B30. [MCQ-single] [F] [1.1] [1.5]

A Quantum-2 QM9700 switch provides how many NDR 400 Gb/s InfiniBand ports?

- A. 32
- B. 48
- C. 64
- D. 128

B31. [MCQ-single] [P] [1.1, 3.1] [1.5]

An engineer proposes configuring per-switch InfiniBand routes on Helix-A's Quantum-2 switches with `nv set` commands, as on Cumulus. What is wrong with this plan?

- A. Nothing. Each IB switch computes its own routes from CLI configuration.
- B. InfiniBand routes are computed and programmed centrally by the Subnet Manager, and Quantum-2 switches run MLNX-OS, not Cumulus/NVUE.
- C. InfiniBand routing can only be set from the GPU nodes.
- D. InfiniBand has no routing. All traffic is flooded.

B32. [MCQ-single] [F] [1.1] [1.5]

Which operating system runs on Spectrum Ethernet switches and provides the NVUE configuration interface?

- A. MLNX-OS
- B. Cumulus Linux
- C. DOCA OS
- D. UFM OS

B33. [Compare] [P] [1.1] [1.5]

Which pairing correctly matches the multi-tenancy mechanism for each fabric?

- A. InfiniBand: VRFs · Spectrum-X: PKeys
- B. InfiniBand: PKeys · Spectrum-X: BGP-EVPN VRFs and VNIs
- C. InfiniBand: VLANs · Spectrum-X: LIDs
- D. Both: PKeys

B34. [Compare] [P] [1.1] [1.5]

Which capability listed in this chapter is specific to the InfiniBand fabric and performs reduction operations inside the switches?

- A. WJH

- B. SHARP
- C. EVPN type-5 routes
- D. PFC

B35. [Read-diagram] [P] [1.1] [1.5, Fig 1.6]

Using the decision tree in Figure 1.6: a company is building a multi-tenant AI inference cloud and its network team runs BGP-EVPN everywhere today. Which compute fabric does the tree recommend?

- A. InfiniBand with PKeys
- B. Spectrum-X Ethernet
- C. The in-band management network
- D. NVLink only

Section 1.6 — Scalable units

B36. [MCQ-single] [F] [1.1] [1.6]

What is a scalable unit (SU) in NVIDIA AI factory reference architectures?

- A. A single modular switch chassis
- B. A repeatable block of GPU nodes together with the leaf switches and cabling that serve them
- C. A Kubernetes namespace reserved for GPU jobs
- D. A UFM licence tier

B37. [MCQ-single] [F] [1.1] [1.6]

How many DGX nodes make up one SU in the DGX SuperPOD reference architectures for H100/H200/B200 systems?

- A. 8
- B. 16
- C. 32
- D. 64

B38. [Calculation] [P] [1.1] [1.6]

A rail leaf uses a 64-port switch and must be non-blocking (equal downlink and uplink bandwidth at the same port speed). How many GPU nodes can it serve on its rail?

- A. 16
- B. 32
- C. 48
- D. 64

B39. [Calculation] [E] [1.1] [1.6]

Helix plans a third InfiniBand cluster: **3 SUs** × 32 nodes, 8 rails, 64-port NDR switches, non-blocking two-tier fat-tree. How many spine switches are needed?

- A. 8
- B. 12
- C. 16
- D. 24

B40. [Read-diagram] [P] [1.1, 1.2] [1.6, Fig 1.7]

In Figure 1.7, GPU3 on node 5 sends to GPU3 on node 20 in the same SU. GPU0 maps to rail 1. How many switches does the traffic cross, and which one?

- A. One switch: leaf R4
- B. One switch: leaf R3
- C. Three switches: leaf, spine, leaf

D. None. The traffic uses NVLink.

B41. [Troubleshooting] [E] [1.1, 5.5] [Troubleshooting scenario]

Jobs slow down only when `hxa-su1-gpu07` takes part. All its ports show **Active** at 400 Gb/s, and `nvidia-smi topo -m` looks correct. Which tool most directly shows that two of the node's cables land on the wrong rail leaves?

- A. `ibstat`
- B. `iblinkinfo` (or the UFM topology view)
- C. `lspci`
- D. `nvidia-smi`

B42. [MCQ-single] [F] [1.1, 3.1] [1.7]

Which component assigns LIDs to ports in an InfiniBand fabric?

- A. UFM's web interface directly
- B. The Subnet Manager
- C. MLNX-OS on every switch independently
- D. The ConnectX firmware

B43. [MCQ-single] [F] [1.1, 2.4] [1.7]

Before changing Helix-B's production switches, the team wants to test the change on a simulated copy of the Spectrum fabric. Which NVIDIA tool fits?

- A. UFM
- B. NVIDIA Air
- C. SHARP
- D. `ibdiagnet`

B44. [MCQ-single] [F] [1.1, 4.1] [1.7]

What is the role of the NVIDIA Network Operator?

- A. It runs the InfiniBand Subnet Manager inside Kubernetes.
- B. It deploys and manages the drivers, device plugins and CNIs that give Kubernetes pods RDMA and high-speed networking.
- C. It replaces NetQ for monitoring Spectrum switches.
- D. It configures BGP-EVPN on leaf switches.

C. Deep-understanding set

- C1. [True/False + justify] [P]** "Adding more ECMP paths always fixes congestion for AI training traffic."
- C2. [True/False + justify] [F]** "In a DGX SuperPOD-style design, storage traffic and GPU-to-GPU collectives share the same eight compute NICs."
- C3. [True/False + justify] [P]** "Physical state: LinkUp in `ibstat` proves a port is ready to carry RDMA traffic."
- C4. [True/False + justify] [P]** "A Spectrum-4 switch with ordinary third-party NICs provides the full Spectrum-X adaptive-routing capability."
- C5. [Order] [P]** Put the parts of one data-parallel training step in order: (a) inter-node all-reduce on the compute fabric; (b) read batch from storage; (c) forward and backward pass; (d) intra-node reduction over NVLink; (e) optimiser applies the averaged gradients.
- C6. [Order] [F]** Put the Lab 1 tasks in their logical order: (a) `nvidia-smi topo -m`; (b) `ibstat`; (c) `lspci`; (d) record the rail map; (e) `ibdev2netdev`.
- C7. [Fill-in] [F]** The four physically separate networks of an AI factory are the _____ fabric, the _____ fabric, the _____ management network and the _____ management network.
- C8. [Fill-in] [F]** The command that maps `mlx5_N` RDMA device names to Linux interface names is _____.
- C9. [Design task] [P]** List which management or monitoring tool Helix uses for (i) the InfiniBand fabric, (ii) the Spectrum-X fabric's real-time monitoring, (iii) Spectrum switch configuration, (iv) Kubernetes RDMA enablement.
- C10. [Calculation] [E]** A Spectrum-X SU of 32 nodes uses 8 rail leaves. Each node NIC runs at 400 GbE. Each SN5600 leaf uses 800 GbE ports broken out to 2×400 GbE towards the nodes, and 800 GbE uplinks to the spines. For a non-blocking leaf, how many 800 GbE ports does each leaf use for downlinks and for uplinks, and how many 64-port SN5600 spines does the SU need?

D. Interview and design-defence questions

- D1.** "Explain to a CIO why GPU utilisation depends on the network, in two minutes."
- D2.** "Why do AI factories use four separate networks? Wouldn't one big converged network be cheaper?"
- D3.** "Walk me through the data path of GPUDirect RDMA between two GPUs on different nodes."
- D4.** "When would you choose a BlueField-3 DPU, and when a SuperNIC?"
- D5.** "InfiniBand or Spectrum-X for a new 256-node training cluster? Defend your choice."
- D6.** "What is a scalable unit and why does it matter operationally, not just for performance?"

E. Flashcards

Front	Back
AI factory	A data centre designed as one computer whose job is to produce AI (training, tokens) at scale
JCT	Job completion time: set by the slowest participant in each synchronous step
Tail latency	Latency of the slowest few percent of transfers; it drives collective performance
Elephant flow	A very large, long-lived flow, typical of AI collectives

Front	Back
Low entropy	Few distinct flow 5-tuples, so ECMP hashing distributes load poorly
Incast	Many senders to one receiver at the same moment
RDMA	NIC-to-NIC memory transfer that bypasses the CPU and kernel stack
GPUDirect RDMA	NIC reads and writes GPU memory directly over PCIe
RoCE	RDMA over Converged Ethernet (RoCEv2 runs over UDP/IP)
Compute fabric	East-west network for GPU-to-GPU traffic between nodes
Storage fabric	Network for dataset reads and checkpoint writes
In-band management	Network for cluster services, provisioning, SSH and external access
OOB management	1 GbE network for BMCs and device management ports
DGX B200 compute NICs	8 × ConnectX-7, up to 400 Gb/s each
DGX B200 BlueField-3 role	Storage and in-band management
PIX (topo -m)	GPU and NIC share one PCIe switch (best affinity)
SYS (topo -m)	The path crosses the CPU socket interconnect (poor affinity)
DPU	NIC plus Arm cores and memory, for infrastructure offload and isolation (north-south)
SuperNIC	East-west AI accelerator for Spectrum-X; reorders sprayed packets
Spectrum-X	Spectrum-4 switches plus SuperNICs, working end to end
SN5600	64 × 800 GbE Spectrum-4 switch (51.2 Tb/s)
QM9700	Quantum-2 switch with 64 × NDR 400 Gb/s InfiniBand ports
SN2201	48 × 1 GbE OOB management switch
Subnet Manager	Assigns LIDs and computes routes for an InfiniBand fabric
SU (SuperPOD)	32 nodes plus 8 rail leaves: a repeatable building block

F. Cross-domain preview questions

These questions preview later chapters but can be answered from this chapter alone.

F1. [MCQ-single] [P] [1.1 → 2.2] Which property of the Helix-B compute fabric makes lossless or near-lossless Ethernet features (covered in Chapter 5) important?

- A. It carries RoCE traffic for synchronous collectives, where a single drop stalls a transfer.
- B. It carries BMC traffic.
- C. It uses 1 GbE links.
- D. It has no spines.

F2. [MCQ-single] [P] [1.1 → 5.5] Which field of `ibstat` tells you the port's local InfiniBand address assigned by the SM?

- A. Port GUID • B. Base lid • C. Rate • D. Firmware version

F3. [MCQ-single] [P] [1.1 → 2.7] A Helix-B engineer must install software to configure SuperNIC functionality. Which framework will Chapter 8 cover?

- A. DOCA • B. NetQ • C. OpenSM • D. NCCL

F4. [MCQ-single] [P] [1.1 → 3.4] Which tool will you use in Chapter 15 to monitor InfiniBand link status and bandwidth utilisation?

- A. NetQ • B. UFM • C. NVIDIA Air • D. Ansible

F5. [MCQ-single] [P] [1.1 → 3.2] Helix-A must stop RESEARCH workloads from talking to PRODUCTION workloads on the same InfiniBand fabric. Which mechanism will Chapter 13 configure?

A. EVPN VRFs · B. PKeys · C. VLAN ACLs · D. PFC priorities

G. Challenge lab — design Helix-C

Brief: Helix wants a third cluster, **Helix-C**, for a new tenant. It has 32 GPU nodes (8 GPUs each), needs multi-tenancy with the existing EVPN operations, and runs 400 GbE per GPU. Produce:

1. The compute fabric choice, with justification.
2. The adapter plan per node (compute and north-south).
3. The compute fabric topology: number of rail leaves, downlink and uplink port counts per leaf, and number of spines (assume SN5600, 64 × 800 GbE ports, 2 × 400 GbE breakout to nodes).
4. The networks each node connects to, and what each carries.
5. The management, monitoring and automation tools, mapped to exam objectives.
6. Three validation checks you would run before handing the cluster to users.

H. Answer key

Pre-quiz

PQ1 — C. The compute (east-west) fabric carries inter-node GPU traffic. In-band carries services, storage carries data and checkpoints, and OOB carries BMC and management ports.

PQ2 — C. Eight ConnectX-7 ports, one per GPU, up to 400 Gb/s each.

PQ3 — B. An SU is a repeatable block of nodes plus the leaves and cabling that serve them. It is not a chassis, namespace or licence.

Section B

B1 — B. Synchronous collectives finish only when the slowest transfer completes, so tail latency sets JCT. A: averages hide the stragglers that stall a step. C, D: table sizes and VLAN counts don't set training performance. (See 1.1.)

B2 — A, B. Few 5-tuples and huge long-lived flows mean ECMP hashing pins elephants to paths and collides them. C: many short flows help ECMP. D: AI traffic is east-west. E: the address family doesn't matter. (See 1.1, Figure 1.2.)

B3 — B. Loss forces recovery by retransmission, which stalls that transfer, and the collective waits for it. A: switches don't reload on a drop. C: NCCL doesn't permanently fall back because of one drop. D: GPUs don't reset. (See 1.1.)

B4 — B. GPUDirect RDMA lets the NIC access GPU memory directly. A: GPUDirect Storage is the storage-to-GPU data path, not NIC RDMA between nodes. C: SR-IOV is PCIe virtualisation. D: NVLink is GPU-to-GPU, mostly within a node. (See 1.1, 1.3.)

B5 — B. Incast means many-to-one bursts. A: broadcast storms are flooding loops. C: hairpinning means traffic returning out of the interface it arrived on. D: microsegmentation is a security policy technique. (See 1.1.)

B6 — B. Checkpoint writes go to storage over the storage fabric. Collectives are normal, which rules out the compute fabric (A). C: OOB doesn't carry data. D: NVLink stays inside the node. (See 1.2.)

B7 — C. BMC access uses the OOB network, which works when the host OS is down. A: in-band depends on the host OS. B, D: those networks carry data, not management. (See 1.2.)

B8 — B. Access to external services and registries goes over in-band management. A: the compute fabric is GPU-to-GPU only. C: OOB is for BMCs and device management. D: NVLink is intra-node. (See 1.2.)

B9 — A, B. Performance isolation and different design rules. C: separation *adds* cables. D: NVLink has no such requirement. E: separation creates more subnets, not fewer. (See 1.2.)

B10 — B. NCCL inter-node collectives use the compute fabric. A: BMC traffic is OOB. C, D: DNS, NTP and SSH are in-band. (See 1.2.)

B11 — B. Intra-node GPU traffic uses NVLink/NVSwitch. A: NICs are for inter-node traffic. C: the DPU handles north-south traffic. D: the CPU path is what the design avoids. (See 1.3, Figure 1.4.)

B12 — B. The SuperPOD reference architecture describes slight (4:3) oversubscription for storage. A: the compute fabric is non-blocking. C: NVLink isn't a switched network in this sense. D: this is false according to the reference architecture. (See 1.2, FAQ Q7.)

B13 — C. OOB must survive in-band failures. A, B, D: those are data or tenant networks. (See 1.2.)

B14 — C. Eight: one per GPU. A: two is the number of BlueField-3 cards. B: four is the number of OSFP cages, each carrying two ports. D: 16 is wrong. (See 1.3.)

- B15 — D.** $8 \times 400 \text{ Gb/s} = 3.2 \text{ Tb/s}$. A: one NIC's worth. B, C: miscalculations. (See 1.3.)
- B16 — B.** DGX B200 BlueField-3 cards serve storage and in-band management. A: the ConnectX-7s carry the rails. C: the SM runs on UFM or switches. D: NVLink is GPU hardware. (See 1.3.)
- B17 — B.** PIX means at most a single PCIe bridge, the same PCIe switch, so NIC3 is the local rail NIC. A: NVLink shows as **NV#**. C: crossing the CPU interconnect is **SYS**. D: topo output doesn't show admin state. (See Lab 1, Task 4.)
- B18 — B.** SYS means the path crosses the SMP interconnect between NUMA nodes, so GPUDirect RDMA performance suffers. A: PIX is the best, not SYS. C: NVLink is unaffected. D: the OOB network has nothing to do with it. (See Lab 1, Task 4.)
- B19 — B.** LinkUp is the physical layer. Initializing means the SM hasn't brought the port to Active. A: a faulty cable would not show LinkUp. C: the link layer is a separate field. D: a disabled port would not show LinkUp. (See Lab 1, Task 3.)
- B20 — B.** The eight **ib...** interfaces **mlx5_0-mlx5_7** are the compute rails. The **enp...** interfaces are BlueField Ethernet ports. D: IB devices do appear in **ibdev2netdev**. (See Lab 1, Task 2.)
- B21 — B.** ConnectX/SuperNIC ports run Ethernet (RoCE) on Spectrum-X, and **ibstat** reports **Link layer: Ethernet**. A: Ethernet is expected here. C: LIDs are an InfiniBand concept. D: the link layer says nothing about which network the port connects to. (See FAQ Q9.)
- B22 — B.** DPUs have Arm cores for infrastructure services. A: ConnectX has offloads but no general-purpose OS. C: NVSwitch is a GPU interconnect. D: a transceiver is optics. (See 1.4.)
- B23 — B.** The receiving SuperNIC reorders and places the sprayed packets (direct data placement). A: switches spray the packets but don't reorder them. C: the SM is InfiniBand. D: NetQ monitors the fabric. (See 1.4.)
- B24 — A, B.** Spectrum-X = Spectrum-4 switches + SuperNICs. C: Quantum-2 is InfiniBand. D: UFM is IB management. E: NVSwitch is scale-up. (See 1.4.)
- B25 — B.** Infrastructure offload and isolation is the DPU's role. A, C: both focus on data movement, not hosting services. D: a 1 GbE NIC has none of these capabilities. (See 1.4.)
- B26 — B.** DOCA programs BlueField and SuperNICs. A: NCCL is the collectives library. C: CUDA is GPU programming. D: NVUE is switch configuration. (See 1.4, 1.7.)
- B27 — B.** One per GPU, east-west. A: that describes a DPU role. C, D: both are wrong placements. (See 1.4, Figure 1.5.)
- B28 — C.** The SN2201 has $48 \times 1 \text{ GbE}$ ports for OOB. A: the SN5600 is a 51.2 Tb/s Spectrum-4 switch. B: the QM9700 is InfiniBand. D: the Q3400 is XDR InfiniBand. (See 1.5.)
- B29 — B.** $64 \times 800 \text{ GbE}$ with breakout to $128 \times 400 \text{ GbE}$. A: that is the SN2201. C: that is the QM9700. D: that is an older 100G switch. (See 1.5.)
- B30 — C.** 64 NDR ports in 32 OSFP cages. A: 32 is the number of cages, not ports. (See 1.5.)
- B31 — B.** InfiniBand routing comes from the SM, and Quantum-2 runs MLNX-OS. (Newer NVOS-based IB switches have an NVUE-style CLI, but routes still come from the SM.) A: this is the exam trap in 1.5. C: hosts don't program fabric routing. D: InfiniBand does use routing tables (LFTs) programmed by the SM. (See 1.5.)
- B32 — B.** Cumulus Linux provides NVUE. A: MLNX-OS runs on Quantum switches. C, D: neither is a switch OS. (See 1.5.)
- B33 — B.** InfiniBand uses PKeys. Spectrum-X uses EVPN VRFs and VNIs. The other pairings are reversed or invented. (See 1.5 table.)

B34 — B. SHARP is in-network aggregation. A: WJH is Spectrum drop telemetry. C: type-5 routes are EVPN. D: PFC is Ethernet flow control. (See 1.5.)

B35 — B. Multi-tenant plus existing EVPN operations leads to Spectrum-X in Figure 1.6. C, D: neither is a compute fabric choice. (See Figure 1.6.)

B36 — B. A repeatable block of nodes, leaves and cabling. (See 1.6.)

B37 — C. 32 nodes per SU. (See 1.6.)

B38 — B. 32 downlinks plus 32 uplinks = 64 ports. (See 1.6.)

B39 — B. 3 SUs × 8 leaves = 24 leaves. 24 × 32 uplinks = 768 ports. $768 \div 64 = 12$ spines. A: 8 is the two-SU answer. (See 1.6 worked example.)

B40 — A. GPU0 → rail 1, so GPU3 → rail 4. Both nodes' NIC4 connects to leaf R4, which is one switch hop. B: off by one. C: that is the path for traffic between rails. D: NVLink is intra-node only. (See Figure 1.7.)

B41 — B. `iblinkinfo` and the UFM topology view show which switch port each HCA connects to. A: `ibstat` shows only local port state. C, D: those are host-local tools. (See the troubleshooting scenario.)

B42 — B. The SM assigns LIDs. A: UFM hosts or manages the SM, but LID assignment is the SM's function. C: switches don't assign LIDs independently. D: NIC firmware doesn't assign LIDs. (See 1.7.)

B43 — B. NVIDIA Air provides a digital twin. A: UFM is InfiniBand management. C: SHARP is aggregation. D: `ibdiagnet` is IB diagnostics. (See 1.7.)

B44 — B. The Network Operator deploys drivers, device plugins and CNIs for RDMA in Kubernetes. A: the SM doesn't run inside Kubernetes through the operator. C, D: the operator doesn't monitor or configure switches. (See 1.7.)

Section C

C1 — False. Hash-based ECMP pins each elephant flow to one path. More paths don't prevent two elephants hashing onto the same link. Adaptive routing and congestion control are needed (Chapters 6 and 14).

C2 — False. Storage uses the BlueField-3 DPUs and the storage fabric. The eight compute NICs carry only compute traffic.

C3 — False. LinkUp is physical only. The logical **State** must be **Active**, meaning the SM has configured the port.

C4 — False. Full Spectrum-X adaptive routing depends on SuperNICs reordering packets at the receiver. The platform works end to end.

C5 — b, c, d, a, e. Read, compute, reduce locally over NVLink, reduce across nodes on the fabric, then apply.

C6 — c, e, b, a, d. Hardware first, then the device map, state, affinity, and finally the record.

C7 — compute; storage; in-band; out-of-band (OOB).

C8 — `ibdev2netdev`.

C9 — (i) UFM, with the SM hosted on UFM; (ii) NetQ, plus WJH for drop reasons; (iii) NVUE, automated with Ansible; (iv) NVIDIA Network Operator.

C10 — Each leaf serves 32 nodes × 400 GbE = 12.8 Tb/s down, which is **16 × 800 GbE ports** broken out to 32 × 400 GbE. Non-blocking needs **16 × 800 GbE uplinks**. Across 8 leaves that is 8 × 16 = 128 uplink ports of 800 GbE, and $128 \div 64 = 2$ spines.

Section D — model answers

D1. A strong answer covers these points: training is synchronous, so every step waits for the slowest GPU-to-GPU transfer. On a slow or lossy network GPUs sit idle, and idle GPUs are expensive. A purpose-built fabric (lossless, adaptive routing, one 400 Gb/s NIC per GPU) turns network time into compute time. Frame it as utilisation and JCT, which relate directly to cost per model.

Weak answers: talk only about bandwidth numbers, or never connect the network to GPU idle time.

D2. A strong answer covers these points: performance isolation (storage bursts can't hurt collectives); different design rules (lossless and rail-aligned vs routed enterprise access); security and OOB survivability; separate operational tooling. Acknowledge that smaller enterprise designs converge storage and in-band into one north-south network, while the compute fabric always stays separate.

Weak answers: just say "best practice" with no reasoning, or claim everything must always be separate.

D3. A strong answer covers these points: GPU memory → PCIe switch → local NIC (PIX) → fabric (one hop on the same rail) → remote NIC → PCIe switch → remote GPU memory. No CPU copies and no kernel stack. The NCCL/verbs setup is done by the CPU, but the data bypasses it.

Weak answers: include system memory in the data path, or forget GPU-NIC affinity.

D4. A strong answer covers these points: a DPU is for north-south infrastructure offload, isolation, security and storage services, with Arm cores and DOCA services. A SuperNIC is for east-west AI traffic in Spectrum-X, one per GPU, with adaptive routing reordering and congestion control. Both use DOCA.

Weak answers: treat the two as interchangeable, or say "a DPU is just a faster NIC".

D5. A strong answer covers these points: set out the criteria (tenancy, operations skills, tooling, scale, SHARP value, existing Ethernet/EVPN estate). For a single-tenant, largest-scale training cluster, InfiniBand's SM, SHARP and native losslessness are compelling. For multi-tenant AI clouds or Ethernet-centric operations, Spectrum-X with EVPN, NetQ and NVUE fits. Commit to a choice and state the trade-offs.

Weak answers: claim one is universally better, or don't mention operations.

D6. A strong answer covers these points: a pre-validated, repeatable block of compute plus leaves. It gives predictable performance, standard cable maps, easier cabling validation, faster deployment, and incremental growth through spines. Error detection is easier because every SU looks the same.

Weak answers: describe it only as "32 servers".

Section F

F1 — A. RoCE collectives are loss-sensitive.

F2 — B. Base lid.

F3 — A. DOCA.

F4 — B. UFM.

F5 — B. PKeys.

Section G — worked solution: Helix-C

- Fabric: Spectrum-X Ethernet.** The tenant needs multi-tenancy integrated with existing BGP-EVPN operations (Figure 1.6), and the design specifies 400 GbE.
- Adapters:** 8 × SuperNICs per node (BlueField-3 SuperNIC or ConnectX-8 SuperNIC), one per GPU, for east-west traffic. 1–2 × BlueField-3 DPUs for storage, in-band and tenant-edge offload. A BMC port for OOB.
- Topology:** 8 rail leaves (SN5600). Each leaf has 16 × 800 GbE ports broken out to 32 × 400 GbE node links, plus 16 × 800 GbE uplinks, so it is non-blocking. 128 uplinks ÷ 64 ports = **2 × SN5600 spines**. Same-rail traffic is one hop. Traffic between rails is leaf → spine → leaf.

4. **Networks:** compute (Spectrum-X, RoCE collectives); storage (Ethernet/RoCE through the DPUs, datasets and checkpoints); in-band (services, scheduling, registry, SSH); OOB (BMC and switch management, SN2201).
5. **Tools:**
 - NVUE for switch configuration (2.1, 6.1) and Ansible to automate it (6.2)
 - NetQ for monitoring (2.6), with WJH and in-band telemetry for drops and congestion (2.5, 5.2)
 - NVIDIA Air to simulate changes (2.4)
 - DOCA on the SuperNICs and DPUs (2.7, 2.8)
 - NVIDIA Network Operator for Kubernetes (4.1, 4.2)
 - `cl-resource-query` for switch resource checks (5.1)
6. **Validation:**
 - Cabling and rail check with LLDP/NetQ topology validation against the plan.
 - Host affinity check: `nvidia-smi topo -m` shows PIX for every GPU.
 - Performance baseline: `ib_write_bw/ib_write_lat` per rail and `nccl-tests` all-reduce across all nodes (Chapter 19), with RoCE lossless QoS verified (Chapter 5).

Chapter 2 — GPU-to-GPU Communication, NCCL and Rail-Optimized Topologies

Exam map

Domain	1 — AI Data Center Design and Optimization (5%)
Objectives	1.2 Describe rail-optimized topologies for high-performance AI workloads • 1.3 Describe GPU-to-GPU communications
Also prepares	5.3 Verify low-latency interconnects (Chapter 19) • 5.5 ibdiagnet (Chapter 17) • 2.2 / 3.3 adaptive routing (Chapters 6, 14)
What you must be able to do	Explain the difference between scale-up (NVLink) and scale-out (fabric) communication. Trace the data path between two GPUs in the same node and in different nodes. Name the main collective operations and the parallelism strategies that use them. Explain how NCCL builds rings and trees and why it wants rails. Describe a rail-optimized topology, what PXN adds, and how to check rail cabling. Size a two-tier non-blocking fat-tree.

"Do I know this already?" pre-quiz

PQ1. In a rail-optimized topology, which NICs connect to the same leaf switch?

- A. All eight NICs of one node • B. NIC *n* of every node in the scalable unit • C. Only the NICs used for storage • D. NICs chosen at random to balance load

PQ2. Which collective operation does data-parallel training use to combine gradients?

- A. Broadcast • B. All-to-all • C. All-reduce • D. Scatter

PQ3. NCCL runs an all-reduce across 16 GPUs and reports an algorithm bandwidth (algbw) of 100 GB/s. Approximately what bus bandwidth (busbw) does nccl-tests report?

- A. 100 GB/s • B. 187.5 GB/s • C. 200 GB/s • D. 1,600 GB/s

Opening problem: the spines are hot but the leaves are cool

Helix-A's first scalable unit is running. The network team is surprised by their monitoring. During large training jobs, several spine links run at close to line rate while many leaf-to-node links sit well below it. The data science team reports that NCCL all-reduce bandwidth between 32 nodes is about 30% below NVIDIA's published numbers for similar systems.

The network is not faulty. Every link is Active at 400 Gb/s and there are no errors. The problem is *how GPU traffic maps onto the fabric*. Some jobs were launched with a setting that let NCCL cross rails freely, and a handful of nodes had been cabled with two NICs swapped. Traffic that should have stayed on one rail leaf was going up to the spines and back down.

To understand why, you need to know how GPUs talk to each other: inside a node over NVLink, and between nodes over the fabric. You also need to know how NCCL, the library that orchestrates this, chooses its paths. That is the subject of this chapter.

2.1 Scale-up and scale-out

GPU-to-GPU communication happens at two levels. Keeping them separate in your head is the key to the rest of this chapter.

	Scale-up	Scale-out
Scope	GPUs inside one NVLink domain: one HGX/DGX node, or a GB200/GB300 NVL72 rack	Between NVLink domains, across the data centre
Interconnect	NVLink + NVSwitch	InfiniBand (Quantum) or Spectrum-X Ethernet, through one NIC per GPU
Per-GPU bandwidth (example)	H100: 900 GB/s NVLink 4 · B200: 1.8 TB/s NVLink 5 (bidirectional)	400 Gb/s (≈50 GB/s) per GPU per direction with NDR / 400 GbE
Latency	Lowest; memory-semantic load/store	Low, but higher than NVLink; RDMA over a switched fabric
Managed by	NVIDIA driver and fabric manager	Subnet Manager/UFM (IB) or NVUE/NetQ (Ethernet)
Exam objectives	1.3	1.2, 1.3 and most of Domains 2, 3, 5

Look at the bandwidth numbers. An H100's NVLink bandwidth (900 GB/s) is roughly **eighteen times** the bandwidth of its 400 Gb/s scale-out NIC in one direction (50 GB/s). A well-designed communication library therefore does as much work as possible inside the NVLink domain and sends only what it must over the fabric.

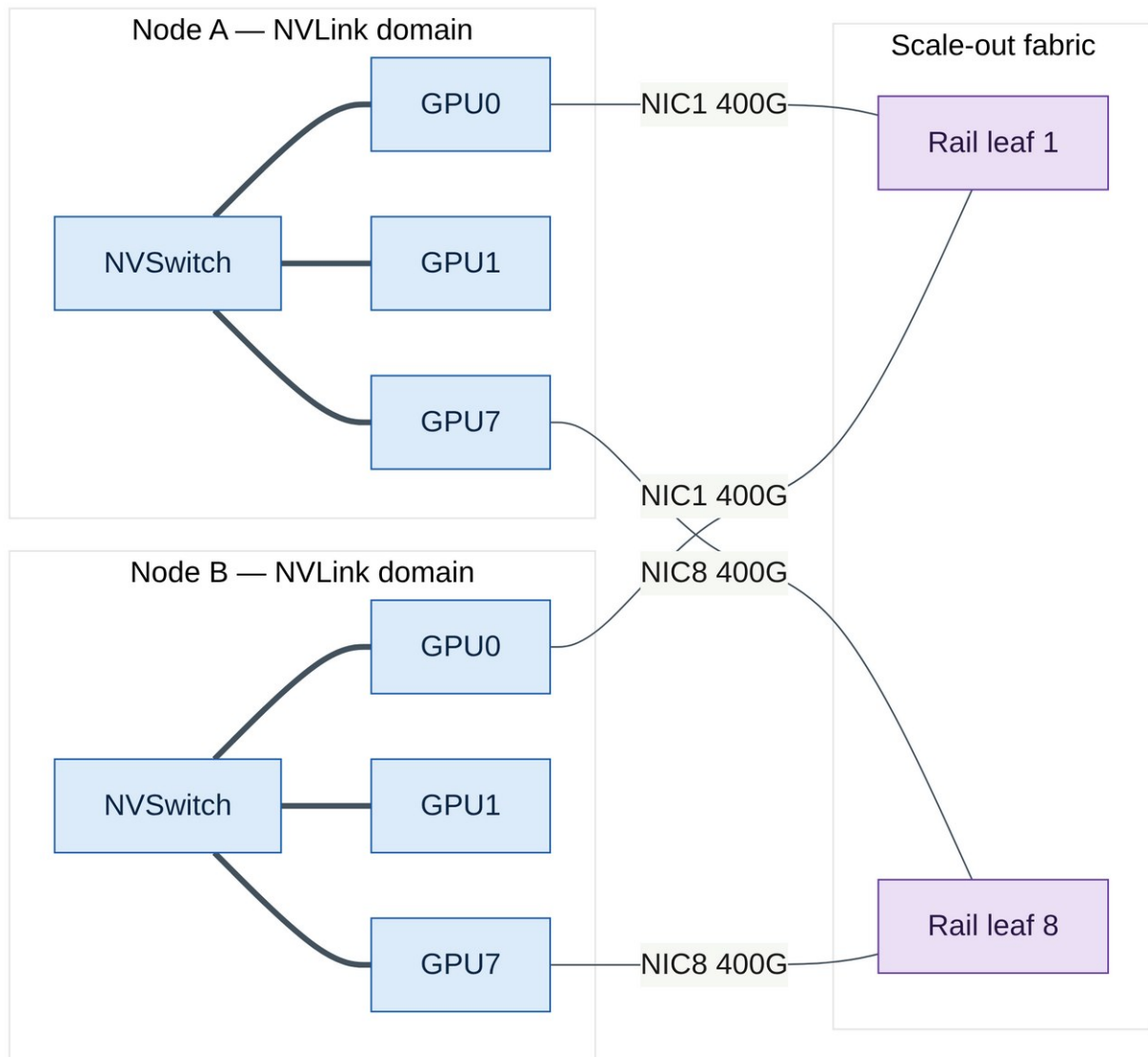


Figure 2.1: Scale-up versus scale-out

Inside each node, GPUs talk over NVLink through NVSwitch (thick lines). Between nodes, each GPU uses its own NIC and the compute fabric. The NVLink tier has an order of magnitude more bandwidth per GPU than the fabric tier.

NVLink and NVSwitch generations

Generation	Example GPU / system	NVLink bandwidth per GPU (bidirectional)	NVLink domain
NVLink 4 + NVSwitch 3rd gen	H100 / H200 in HGX/DGX	900 GB/s (18 links)	8 GPUs per node
NVLink 5 + NVLink Switch	B200 / B300 in HGX/DGX	1.8 TB/s (18 links)	8 GPUs per node
NVLink 5 + NVLink Switch trays	GB200 / GB300 NVL72	1.8 TB/s	72 GPUs per rack (130 TB/s total NVLink bandwidth)

Exam focus (1.3): Know that **NVLink is the scale-up path and the NICs are the scale-out path**. Two GPUs in the same node never use their NICs to talk to each other when NVLink is available. In an NVL72 rack, all 72 GPUs are in *one* NVLink domain, so a much larger share of traffic stays off the scale-out fabric.

Version note: Third-generation NVSwitch and later can perform reductions inside the switch (**NVLink SHARP**, used

by NCCL's **NVLS** algorithm). Do not confuse it with **InfiniBand SHARP** in Quantum switches (Chapter 14), which reduces data inside the scale-out fabric.

Check your understanding 2.1: Why does NCCL prefer to reduce data within a node before sending it across the fabric? (Answer: NVLink provides roughly an order of magnitude more bandwidth per GPU than the scale-out NIC, so reducing locally first cuts the volume the slower fabric must carry.)

2.2 GPU-to-GPU data paths

A GPU can reach another GPU by four paths. They differ enormously in performance. Chapter 19 (objective 5.3) teaches you to *measure* them; here you learn to *recognise* them.

#	Path	When it happens	Relative performance
①	NVLink P2P (through NVSwitch)	Both GPUs in the same NVLink domain	Best
②	PCIe P2P	Same node, no NVLink between the GPUs (e.g. PCIe-only servers)	Much lower than NVLink
③	GPUDirect RDMA over the fabric	Different nodes; NIC reads/writes GPU memory directly	Best inter-node path
④	Host-staged copy	Different nodes, but GPUDirect RDMA is unavailable: GPU → host memory → NIC → ... → host memory → GPU	Extra copies, CPU and memory-bus load, higher latency

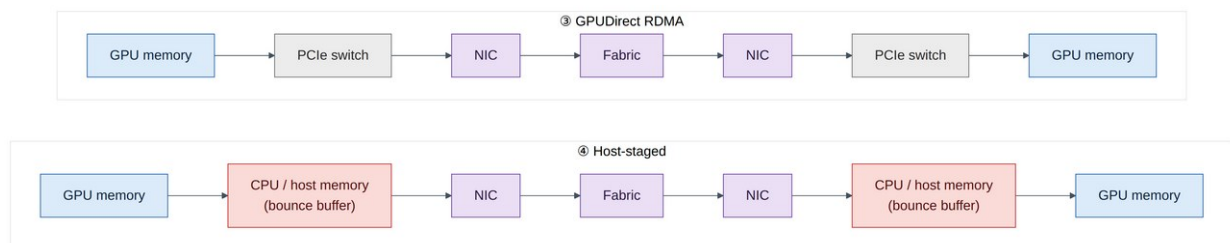


Figure 2.2: Inter-node data paths: GPUDirect RDMA versus host staging

With GPUDirect RDMA (top, ③) the NIC reads GPU memory directly through the PCIe switch. Without it (bottom, ④) data is copied into host memory first and crosses the CPU root complex twice per side.

What GPUDirect RDMA needs

GPUDirect RDMA works only when all of these are in place:

1. **An RDMA-capable NIC** (ConnectX, BlueField, SuperNIC) running InfiniBand or RoCE.
2. **A peer-memory path in the driver stack:** the `nvidia-peermem` kernel module (shipped with the NVIDIA GPU driver) lets the RDMA stack register GPU memory. Newer stacks can use Linux DMA-BUF instead, which NVIDIA now recommends; it needs the GPU driver's open kernel modules, CUDA 11.7 or later and Linux kernel 5.12 or later.
3. **Good PCIe topology:** the NIC and GPU should share a PCIe switch (**PIX** or **PXB** in `nvidia-smi topo -m`). NCCL's `NCCL_NET_GDR_LEVEL` variable decides how far apart a GPU and NIC can be before NCCL stops using GPUDirect RDMA between them.
4. **PCIe settings that allow peer-to-peer traffic**, for example no ACS redirection forcing P2P traffic up to the root complex. On bare metal, NCCL's troubleshooting guide recommends disabling ACS and IO virtualisation (VT-d/IOMMU) where they redirect P2P traffic; virtual machines need ACS, so enable ATS on the NICs instead.

Exam trap: "GPUDirect Storage" and "GPUDirect RDMA" are different. GPUDirect **Storage** moves data between storage and GPU memory. GPUDirect **RDMA** moves data between a NIC and GPU memory, which is what inter-node collectives use.

Check your understanding 2.2: NCCL logs show inter-node traffic running, but not with GPUDirect RDMA. Which kernel module should you check first? (Answer: `nvidia-peermem`, or the `DMA-BUF` path on newer stacks.)

2.3 Collective operations and parallelism

Distributed AI jobs do not send arbitrary messages. They use a small set of **collective operations**, and each training or inference strategy leans on particular ones.

Collective	What it does	Data each GPU ends with
All-reduce	Every GPU contributes a buffer; the element-wise sum (or other reduction) is returned to all GPUs	The full reduced buffer
Reduce-scatter	Reduce across GPUs, but each GPU receives only its slice of the result	1/n of the reduced buffer
All-gather	Each GPU contributes a slice; every GPU receives all slices	The full concatenated buffer
Broadcast	One root sends the same buffer to all GPUs	The root's buffer
Reduce	Reduce to a single root	Root only
All-to-all	Each GPU sends a distinct piece to every other GPU	n pieces, one from each peer
Send / Recv (P2P)	Point-to-point transfer	—

An all-reduce is mathematically a **reduce-scatter followed by an all-gather**. That is exactly how NCCL's ring algorithm implements it (Section 2.4).

Parallelism strategy	What is split	Dominant communication	Where it usually runs
Data parallel (DDP, ZeRO/FSDP)	The training data; each GPU holds a model copy or shard	All-reduce of gradients (DDP); reduce-scatter + all-gather (FSDP/ZeRO)	Across nodes (scale-out)
Tensor parallel	Individual layers across GPUs	Frequent all-reduce / all-gather of activations	Inside the NVLink domain (latency-sensitive)
Pipeline parallel	Groups of layers across GPUs	Point-to-point send/rcv between stages	Across nodes
Expert parallel (Mixture-of-Experts)	Experts across GPUs	All-to-all token routing	Across nodes, and very demanding
Context / sequence parallel	The sequence dimension	All-gather / ring-style exchanges	Often within the NVLink domain

Field note: All-to-all traffic (Mixture-of-Experts models) is the hardest pattern for any fabric. Every GPU talks to every other GPU at once, including GPUs on *other* rails. This is where PXN (Section 2.5), adaptive routing (Chapters 6 and 14) and congestion control matter most.

	Start	All-reduce	All-gather	Reduce-scatter
GPU0	A	A+B+C+D	A, B, C, D	Σ slice 0
GPU1	B	A+B+C+D	A, B, C, D	Σ slice 1
GPU2	C	A+B+C+D	A, B, C, D	Σ slice 2
GPU3	D	A+B+C+D	A, B, C, D	Σ slice 3

Figure 2.3: Collective operations at a glance

Four GPUs, each starting with one block (A–D). Read across to see what each GPU ends up with after each collective.

Check your understanding 2.3: Which collective dominates Mixture-of-Experts training, and why is it hard on the fabric? (Answer: *all-to-all*. Every GPU exchanges data with every other GPU at the same time, including across rails, which creates many-to-many congestion.)

2.4 NCCL: how GPUs actually communicate

The **NVIDIA Collective Communications Library (NCCL, pronounced "nickel")** implements collectives for frameworks such as PyTorch, JAX and TensorFlow. As a network engineer you will rarely write NCCL code, but you will constantly read its logs, set its environment variables and test with its benchmarks. NCCL is how GPU traffic gets onto your fabric.

What NCCL does at start-up:

1. **Topology detection.** It discovers GPUs, NVLink/NVSwitch, PCIe switches, NICs and their affinity (the same information `nvidia-smi topo -m` shows) and builds an internal graph. `NCCL_TOPO_DUMP_FILE` writes this graph to a file.
2. **Transport selection.** For each pair of GPUs it chooses P2P over NVLink or PCIe, shared memory, or the network (`NET/IB` for InfiniBand and RoCE through verbs, `NET/Socket` as a TCP fallback).
3. **Channel construction.** It builds several parallel **channels**, each a ring or tree, so that all NVLinks and all NICs carry traffic at once.
4. **Algorithm and protocol selection** per operation and message size.

NCCL algorithm	Idea	Notes
Ring	GPUs form a logical ring; data moves in $n-1$ steps of reduce-scatter and $n-1$ steps of all-gather	Bandwidth-optimal for large messages; latency grows with the number of GPUs
Tree	Reduce up a (double binary) tree, broadcast back down	Lower latency at large scale and for smaller messages
CollNet	Offloads part of the reduction to the network (InfiniBand SHARP)	Needs a SHARP-capable Quantum fabric; see Chapter 14
NVLS	Uses NVLink SHARP in NVSwitch for in-node (or in-rack) reduction	Needs NVSwitch 3rd gen or later

NCCL protocol	Trade-off
LL (low latency)	Very low latency, lower peak bandwidth; small messages
LL128	Good balance, used with NVLink for medium messages
Simple	Highest bandwidth for large messages

Ring all-reduce: the arithmetic you need

In a ring all-reduce of a buffer of size **S** across **n** GPUs:

- The buffer is split into **n** chunks.
- **Reduce-scatter phase:** $n - 1$ steps. Each GPU sends one chunk to its neighbour and adds the chunk it receives.
- **All-gather phase:** $n - 1$ steps. The reduced chunks are passed around the ring until everyone has all of them.
- Each GPU therefore sends (and receives) $2 \times (n - 1) / n \times S$ bytes. For large n that is almost exactly **2S**, which does not grow with the number of GPUs. That is why ring all-reduce scales well for bandwidth.

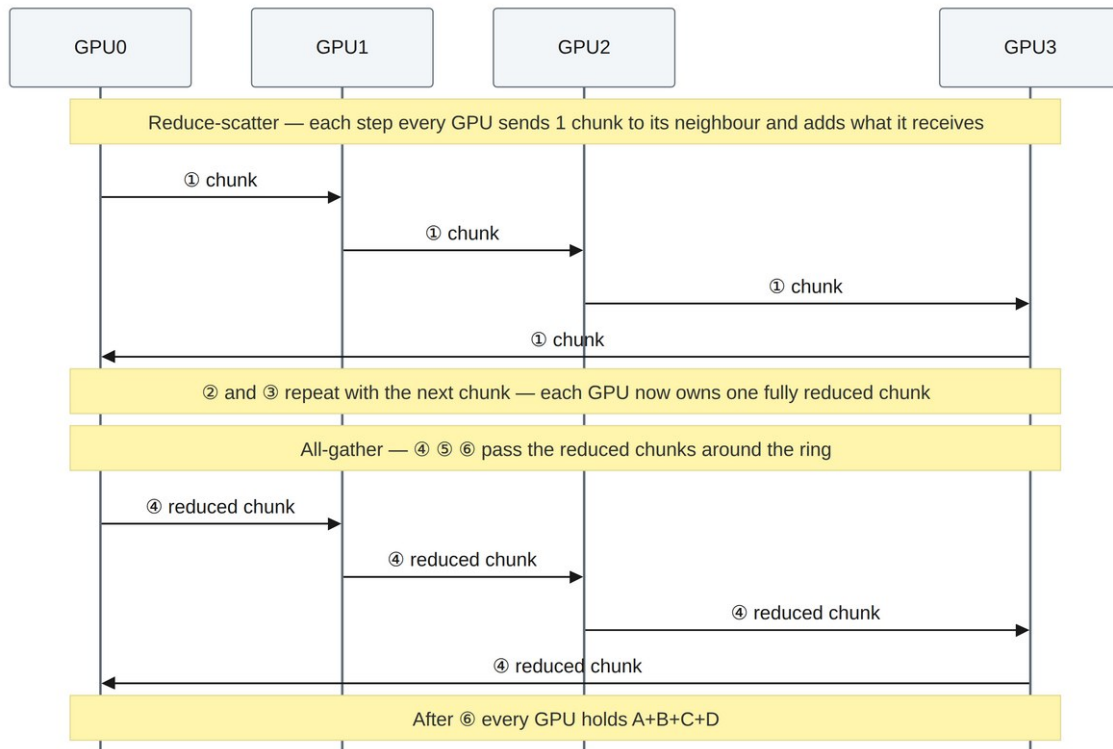


Figure 2.4: Ring all-reduce across four GPUs

Each GPU sends to its right-hand neighbour. After three reduce-scatter steps (① – ③) each GPU owns one fully reduced chunk. After three all-gather steps (④ – ⑥) every GPU holds the whole result. Each GPU sends $2 \times (n-1)/n$ of the buffer in total.

algbw versus busbw in nccl-tests

The standard benchmark, **nccl-tests** (e.g. `all_reduce_perf`), reports two bandwidth figures:

- **algbw (algorithm bandwidth)** = message size ÷ time. It is easy to understand but depends on n .
- **busbw (bus bandwidth)** = algbw × a correction factor that reflects the data each GPU actually moves. For all-reduce the factor is $2 \times (n - 1) / n$. busbw can be compared directly with link and NVLink speeds, whatever the number of GPUs.

Collective	busbw factor
All-reduce	$2(n-1)/n$
All-gather, reduce-scatter	$(n-1)/n$
Broadcast, reduce	1

Exam focus (1.3, 5.3): Expect a question that gives algbw and n and asks for busbw, or asks which figure you compare against link speed (**busbw**). Example: 16 GPUs, algbw 100 GB/s → busbw = $100 \times 2 \times 15/16 = 187.5$ GB/s.

Check your understanding 2.4: An all-gather across 8 GPUs reports algbw = 80 GB/s. What is busbw? (Answer: $80 \times 7/8 = 70 \text{ GB/s}$.)

2.5 Rail-optimized topologies

Chapter 1 introduced rails. Now you can see *why* they exist.

A **rail-optimized topology** connects NIC n (serving GPU n) of every node in a scalable unit to the same leaf switch, **rail leaf n** . NCCL builds its inter-node rings and trees so that GPU n on one node exchanges data with GPU n on other nodes. Collective traffic therefore stays **on one rail, one switch hop away**, and the spine layer carries mainly traffic between SUs and between rails.

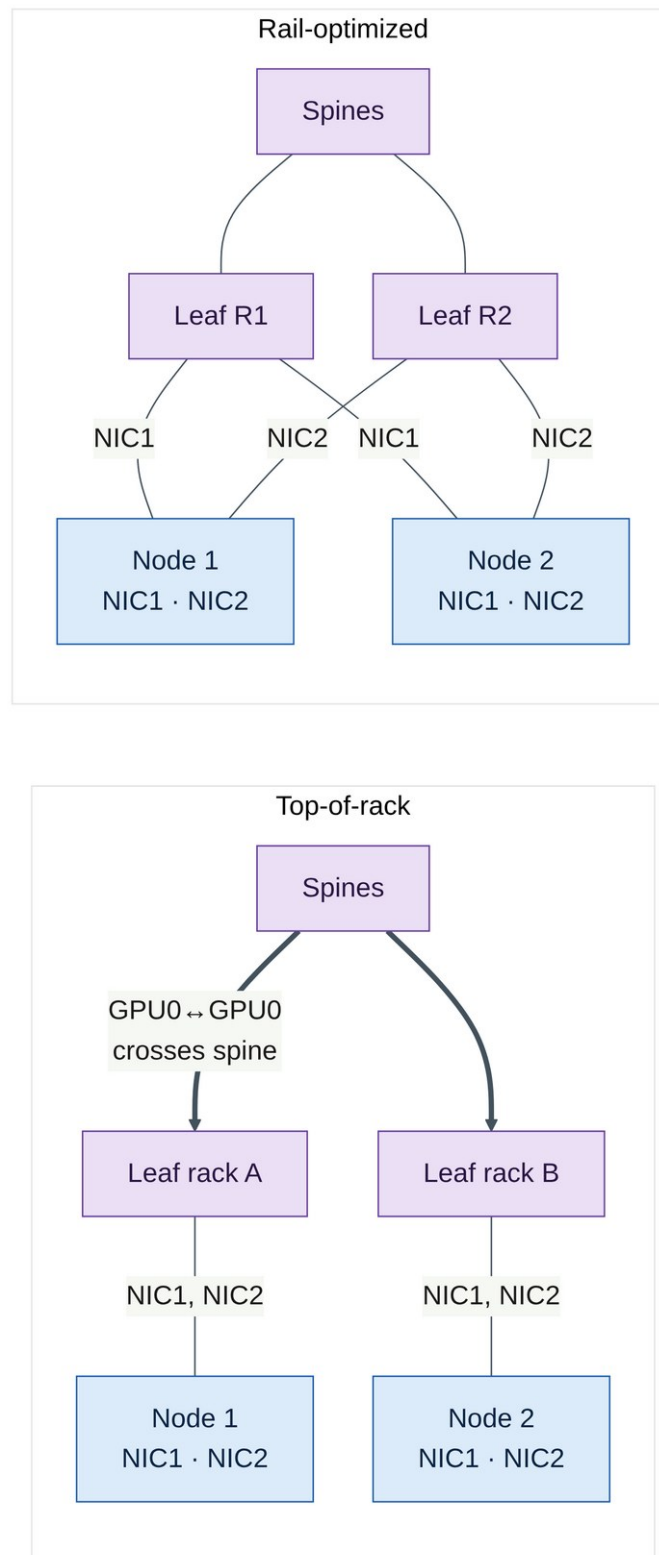


Figure 2.5: Rail-optimized versus top-of-rack (ToR) design

Left: rail-optimized. GPU n of every node reaches the others through rail leaf n (one hop). Right: a traditional ToR design connects all NICs of a node to the same leaf. Same-index GPUs on nodes in different racks must then cross the spine (three hops).

Benefits of rail optimization

Benefit	Why
Fewer hops for collectives	Same-rail traffic: node → rail leaf → node (one switch)
Less spine load	Most collective traffic never leaves the rail leaf

Benefit	Why
Lower and more predictable latency	Fewer hops and fewer shared queues mean less tail latency
Failure containment	A rail leaf failure affects one rail of an SU; NCCL can often route around it through other rails and NVLink
Cleaner scaling	Adding SUs adds rail leaves; spines connect rails and SUs

What about traffic between rails? PXN

Some traffic must reach a *different* index on a remote node. All-to-all and pipeline send/rcv between mismatched GPUs are examples. Without help, GPU0 on node A would send through NIC1 (rail 1) to reach GPU3 on node B. Its rail-1 leaf would forward the traffic up to a spine and down to rail leaf 4.

PXN (PCI × NVLink), introduced in NCCL 2.12, fixes this with the node's own NVLink. GPU0 writes its data over NVLink into the memory of **GPU3 on its own node**. GPU3's NIC (rail 4) then sends it to rail leaf 4, which delivers it to GPU3 on node B in one hop. NCCL can also **aggregate** messages from several local GPUs heading for the same destination into fewer network messages. NVIDIA reported more than **2× all-to-all performance** improvement from PXN.

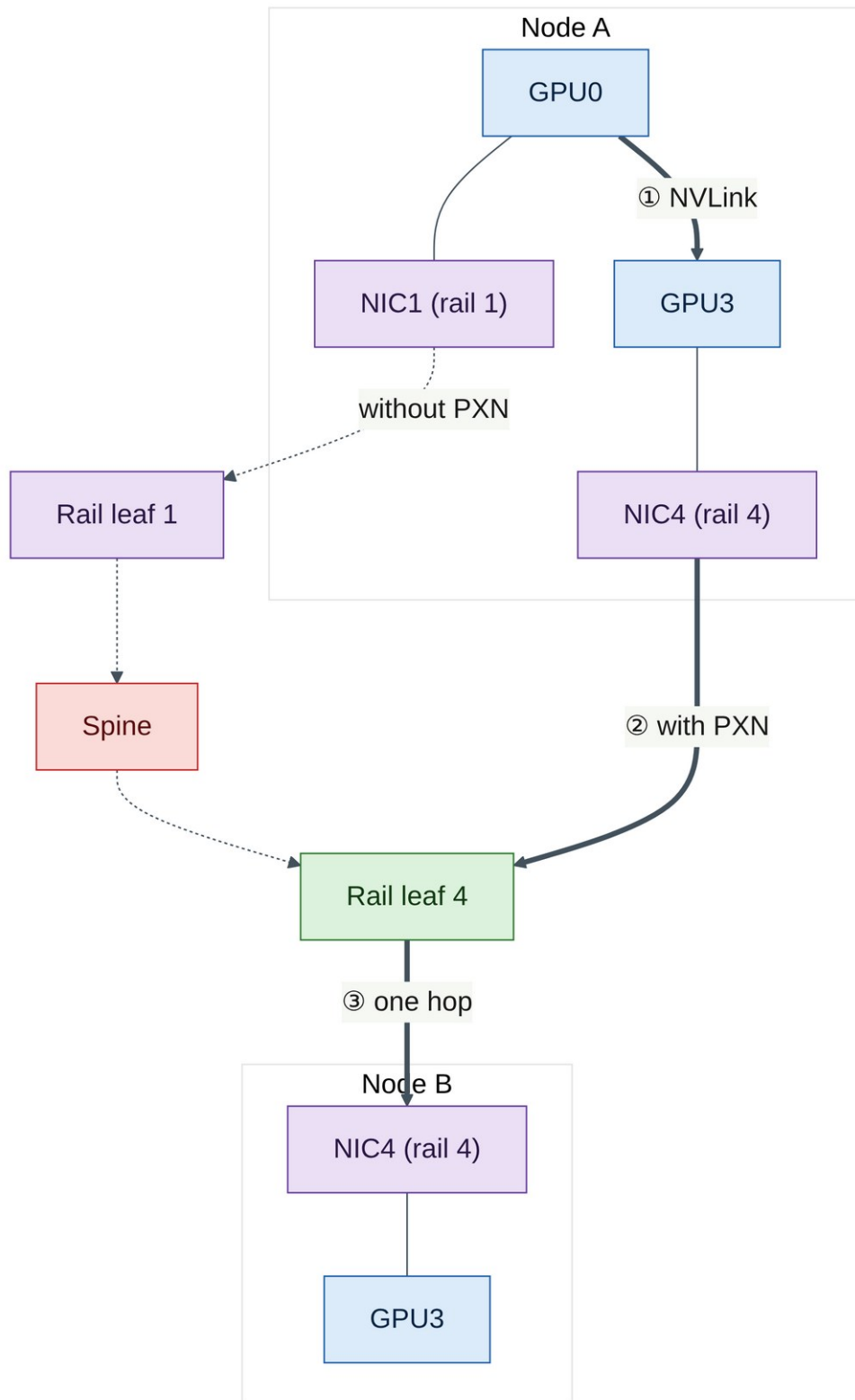


Figure 2.6: PXN: using NVLink to reach the right rail

GPU0 on Node A needs to reach GPU3 on Node B. Without PXN (red) traffic leaves on rail 1 and crosses the spine. With PXN (green) GPU0 first moves the data over NVLink to local GPU3 (①), whose NIC sends it on rail 4 (②), and it arrives in one switch hop (③).

Exam trap: PXN does **not** route traffic through the CPU. The path is GPU → NVLink → peer GPU → PCIe switch → peer GPU's NIC. The point is to avoid both the CPU and the extra spine hops.

Variations you may meet

- **Rail-only** designs (proposed in research for very large LLM training) remove spine connectivity between rails altogether and rely on NVLink for all traffic between rails. They save switch cost but need all rail-to-rail traffic to go through NVLink first. NVIDIA's DGX SuperPOD reference architectures do not use rail-only; they use rail-optimized fat-trees with a spine layer.
- **Multi-plane** designs split each NIC's bandwidth across several independent fabric planes (you will see a `plane` field in `NCCL_IB_HCA`, and `NCCL_IB_RAIL_POLICY` assigns rails and planes to match the reference architecture). Multi-plane guidance is new and changing, so check the current NCCL and reference architecture documents.

Check your understanding 2.5: With PXN, which NIC does GPU0 on node A use to send data to GPU5 on node B, and why? (Answer: NIC6, the rail-6 NIC attached to local GPU5. The data first moves over NVLink to local GPU5, so it can cross the fabric on rail 6 in one hop.)

2.6 Fat-trees, bisection bandwidth and scaling

Rail-optimized leaves sit inside a **fat-tree** (folded Clos) topology. The exam expects you to understand the vocabulary and do simple sizing.

- **Non-blocking:** every leaf has as much uplink bandwidth as downlink bandwidth (1:1). Any half of the endpoints can talk to the other half at full rate.
- **Bisection bandwidth:** the bandwidth across the narrowest cut that divides the network into two equal halves. A non-blocking fat-tree has *full* bisection bandwidth.
- **Oversubscription:** downlink bandwidth ÷ uplink bandwidth at a tier. 1:1 is non-blocking. The Chapter 1 storage example (4:3) is oversubscribed.

Maximum size of a non-blocking fat-tree

With switches of **radix k** (k ports, all the same speed):

Tiers	Maximum endpoints (non-blocking)	k = 64 (e.g. QM9700 at NDR)	GPU nodes at 8 NICs each
2 (leaf-spine)	$k^2 / 2$	2,048	256
3 (leaf-spine-core)	$k^3 / 4$	65,536	8,192

Where the two-tier formula comes from: each leaf uses $k/2$ ports down and $k/2$ up. Each spine has k ports, so it can connect to at most k leaves. With k leaves × $k/2$ endpoints per leaf, you get $k^2/2$ endpoints.

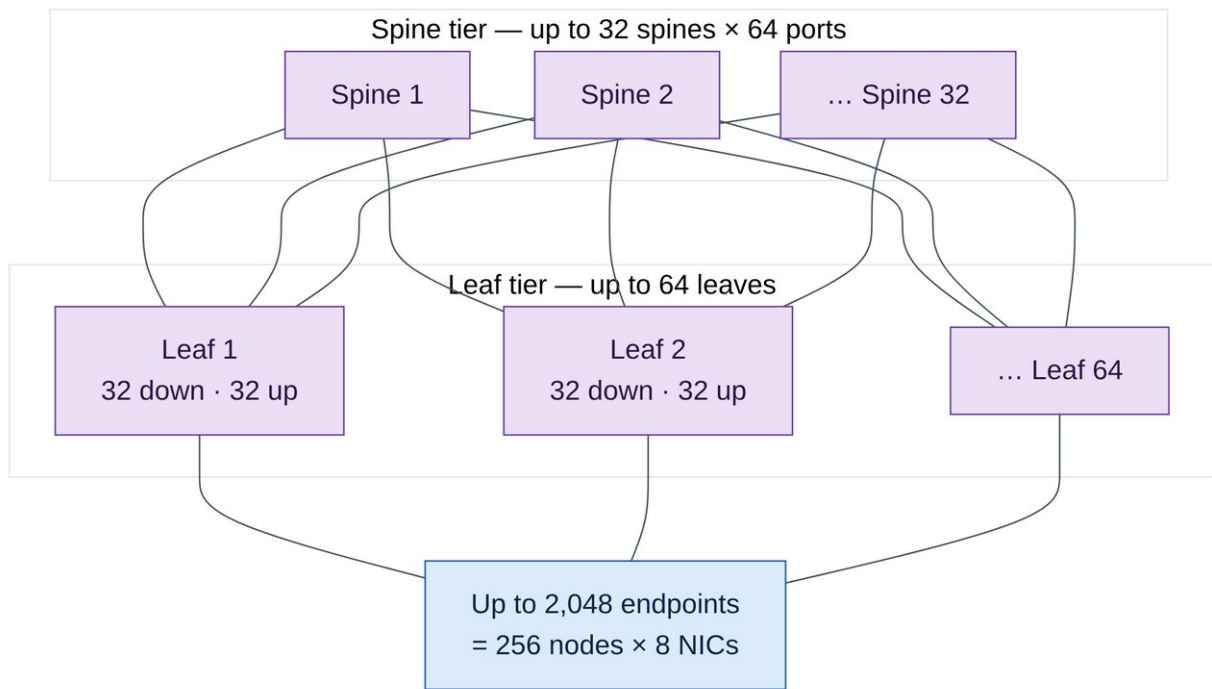


Figure 2.7: Two-tier fat-tree limits

With 64-port switches, a two-tier non-blocking fat-tree supports up to 64 leaves × 32 endpoints = 2,048 endpoints (256 eight-GPU nodes, or 8 SUs of 32 nodes). Beyond that you need a third tier.

Field note: In real deployments, the spine tier is often organised into **spine groups aligned with rails**, so that rail leaf *n* in every SU connects to the same spine group. This keeps traffic between SUs on the same rail predictable. Always follow the reference architecture cable plan. (The public DGX SuperPOD RAs do not describe spine grouping, so confirm the layout against your cable plan.)

Check your understanding 2.6: What is the maximum number of 8-GPU nodes in a non-blocking two-tier fat-tree built from 64-port switches, if every node uses 8 fabric ports? (Answer: $2,048 \div 8 = 256$ nodes.)

2.7 NCCL settings every AI network engineer should know

You will meet these variables in job scripts, Kubernetes manifests and troubleshooting sessions. Know what each one does. Leave exact defaults to the NCCL documentation for your version, because they change between releases.

Variable	What it controls	Typical use by the network team
<code>NCCL_DEBUG=INFO</code>	Log level (VERSION, WARN, INFO, TRACE)	First step in any investigation: shows NICs, transports, GDR usage, rings
<code>NCCL_DEBUG_SUBSYS</code>	Filters debug output by subsystem (e.g. <code>INIT, NET</code>)	Focus on network selection
<code>NCCL_IB_HCA</code>	Which RDMA devices (HCAs) NCCL may use, e.g. <code>mlx5_0, mlx5_1...</code> or <code>^mlx5_8</code> to exclude	Keep NCCL on compute rails, away from storage/in-band ports
<code>NCCL_SOCKET_IFNAME</code>	Which IP interface NCCL uses for bootstrap/socket traffic	Point bootstrap at the in-band interface
<code>NCCL_IB_GID_INDEX</code>	GID index for RoCE	Select the RoCEv2 GID (Chapter 3)
<code>NCCL_IB_TC</code> / <code>NCCL_IB_SL</code>	Traffic class (RoCE) / service level (IB) on NCCL traffic	Match the fabric's lossless QoS class (Chapters 5, 14)

Variable	What it controls	Typical use by the network team
<code>NCCL_IB_DISABLE=1</code>	Disables the IB/RoCE transport and forces sockets	Useful only as a diagnostic; disastrous if left set
<code>NCCL_NET_GDR_LEVEL</code>	Maximum GPU-NIC distance (<code>PIX</code> , <code>PXB</code> , <code>PHB</code> , <code>SYS</code>) for GPUDirect RDMA	Troubleshoot "no GDRDMA" on odd topologies
<code>NCCL_CROSS_NIC</code>	Whether rings/trees may use different NICs (rails) on different nodes	<code>0</code> keeps each ring on one rail (rail-optimized fabrics)
<code>NCCL_PXN_DISABLE</code>	Turns PXN off (<code>1</code>)	Diagnostic comparison only
<code>NCCL_ALGO</code> / <code>NCCL_PROTO</code>	Force algorithm (Ring, Tree, CollNet, NVLS) / protocol (LL, LL128, Simple)	Benchmark comparisons
<code>NCCL_TOPO_DUMP_FILE</code>	Writes NCCL's detected topology to a file	Confirm NCCL sees the expected GPU-NIC affinity
<code>NCCL_IB_ADAPTIVE_ROUTING</code>	Uses adaptive routing on IB	Must match the fabric's AR configuration (Chapter 14)

Reading `NCCL_DEBUG=INFO` output

The lines below are **illustrative** and trimmed. Exact wording varies between NCCL versions.

```
hxa-sul-gpu01:12345:12345 [0] NCCL INFO NCCL version 2.x.x+cudaXX.X
hxa-sul-gpu01:12345:12345 [0] NCCL INFO NET/IB : Using [0]mlx5_0:1/IB [1]mlx5_1:1/IB ...
[7]mlx5_7:1/IB ; 00B ibp26s0:10.10.1.11<0>
hxa-sul-gpu01:12345:12390 [0] NCCL INFO Channel 00/16 :    0    1    2    3    4    5    6    7    8 ...
hxa-sul-gpu01:12345:12390 [0] NCCL INFO Channel 00/0 : 0[0] -> 1[1] via P2P/CUMEM
hxa-sul-gpu01:12345:12390 [0] NCCL INFO Channel 00/0 : 7[7] -> 8[0] [send] via NET/IB/0/GDRDMA
```

What to notice:

- `NET/IB : Using [0]mlx5_0:1/IB ...` means NCCL found the eight compute HCAs. With RoCE the suffix reads `/RoCE`.
- `via P2P/...` means intra-node traffic over NVLink/P2P.
- `via NET/IB/0/GDRDMA` means inter-node traffic over IB **with GPUDirect RDMA**. Missing `GDRDMA` means host staging (path ④).
- `via NET/Socket` means NCCL fell back to TCP sockets. Performance will be very poor; find out why.

2.8 Validating rails and GPU communication

Two tools confirm that the fabric matches the design and that GPUs can use it at full speed.

1. **`ibdiagnet --rail_validation`** (InfiniBand, Chapter 17). This checks that all HCAs connected to the same leaf switch have the **same PCIe address (BDF)** in their servers. In other words, NIC *n* of every node lands on the same rail leaf. Mismatches are reported as *node rail connectivity mismatch* warnings, with details in `ibdiagnet2.rails` and warnings in `ibdiagnet2.log`. A `--rail_validation_opt regex='...'` option restricts the check to matching node names.
2. **`nccl-tests`** (Chapter 19). For example, `all_reduce_perf` across all nodes shows whether busbw approaches the expected per-rail bandwidth.

```
# Rail validation on Helix-A (run from a host with fabric access, e.g. the UFM server)
ufm-al:~$ ibdiagnet --rail_validation --rail_validation_opt regex='hxa-sul-gpu.*'
...
-I- Rail optimized topology validation
-W- Node rail connectivity mismatch ...          # illustrative warning line
...
-I- Rails info dumped to /var/tmp/ibdiagnet2/ibdiagnet2.rails
```

```
# NCCL all-reduce across 2 nodes × 8 GPUs (MPI launch; paths and flags vary by site)
$ mpirun -np 16 -H hxa-su1-gpu01:8,hxa-su1-gpu02:8 \
  -x NCCL_DEBUG=INFO -x NCCL_IB_HCA=mlx5_0,mlx5_1,mlx5_2,mlx5_3,mlx5_4,mlx5_5,mlx5_6,mlx5_7 \
  ./build/all_reduce_perf -b 8 -e 8G -f 2 -g 1
#      size  count  type  redop  root  time    algbw    busbw  #wrong
#      (B)
8589934592 ...      float  sum    -1    ...     ...     ...      0    # illustrative
```

Flag	Meaning
-b 8 -e 8G	Start at 8 bytes, end at 8 GB
-f 2	Multiply the size by 2 each step
-g 1	One GPU per process (MPI launches one process per GPU)

Exam focus (1.2, 5.5): Link `ibdiagnet --rail_validation` with "verify the cabling of a rail-optimized topology". Link `busbw` from `nccl-tests` with "compare against the expected link bandwidth".

Hands-on lab 2 — Trace an all-reduce across the rails (reference walkthrough)

Objective: Run a two-node NCCL all-reduce and prove from the logs that intra-node traffic uses NVLink, inter-node traffic uses GPUDirect RDMA on the correct rails, and the fabric delivers the expected busbw.

Environment: Two GPU nodes with NCCL, `nccl-tests` and MPI installed, and the rail map you recorded in Lab 1. The outputs shown are illustrative.

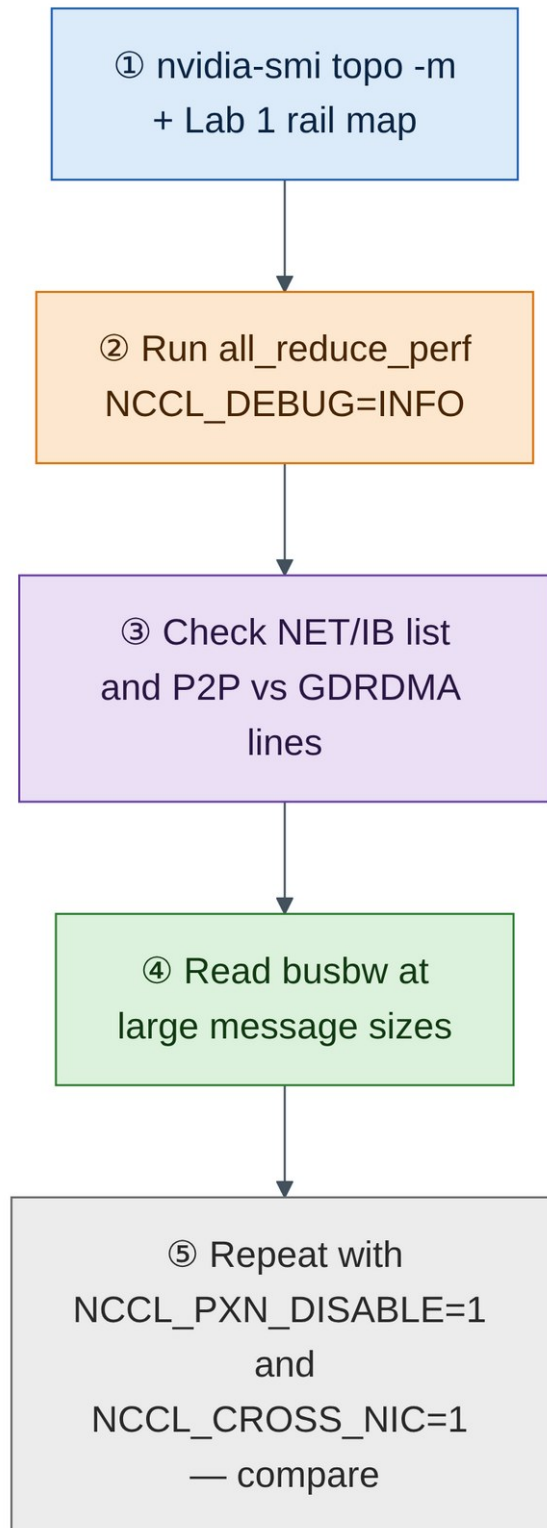


Figure 2.8: Lab 2 flow

From topology to proof: confirm affinity, run with NCCL_DEBUG, read the transports, check busbw, then repeat with a deliberate change to see the effect.

Task 1 — Confirm what NCCL detected

Run the all-reduce with `NCCL_DEBUG=INFO` and `NCCL_DEBUG_SUBSYS=INIT,NET`. In the output, confirm that:

- **NET/IB** : Using lists `mlx5_0` to `mlx5_7` and not the BlueField ports (`mlx5_8`, `mlx5_9`). If the BlueField ports appear, set `NCCL_IB_HCA` to the compute devices only.

- The OOB/bootstrap interface is the one you expect (set it with `NCCL_SOCKET_IFNAME`).

Task 2 — Classify every transport line

Log fragment	Meaning	Expected?
<code>0[0] -> 1[1] via P2P/...</code>	GPU0 → GPU1, same node, NVLink	Yes
<code>7[7] -> 8[0] [send] via NET/IB/0/GDRDMA</code>	Last GPU of node 1 → first GPU of node 2 over IB with GDR	Yes
<code>... via NET/IB/3 (no GDRDMA)</code>	Host-staged inter-node traffic	No. Investigate nvidia-peermem and topology.
<code>... via NET/Socket/...</code>	TCP fallback	No. Investigate IB/RoCE availability.

Task 3 — Read the bandwidth

At large sizes (1 GB and above), record busbw. With 8×400 Gb/s NICs per node, each GPU's rail NIC carries about 50 GB/s in each direction, and NCCL runs parallel rings over all eight rails. The ceiling for a multi-node all-reduce is therefore the node's combined scale-out bandwidth (8×50 GB/s = 400 GB/s), and a healthy two-node run typically reaches roughly 85–92% of it (Chapter 19). NVIDIA does not publish a general reference figure, so compare against your platform's acceptance results.

Task 4 — Experiment

Repeat the run with `NCCL_PXN_DISABLE=1`, then with `NCCL_CROSS_NIC=1`. Compare busbw and the spine-port counters in UFM. On a correctly cabled rail-optimized fabric, all-reduce usually changes little, but all-to-all (`alltoall_perf`) is much more sensitive to PXN.

Common mistakes

- Letting NCCL use storage/in-band ports because `NCCL_IB_HCA` is not set.
- Reading algbw instead of busbw when comparing with link speed.
- Testing with small messages only. Latency dominates there, and bandwidth problems stay hidden.

Troubleshooting scenario — "Inter-node all-reduce is only a third of expected"

Symptom: After a GPU driver upgrade on Helix-A, `all_reduce_perf` between two nodes shows busbw at about a third of the previous value. Intra-node (single node) results are unchanged. `ibstat` shows all ports **Active** at 400 Gb/s, and the fabric error counters are clean.

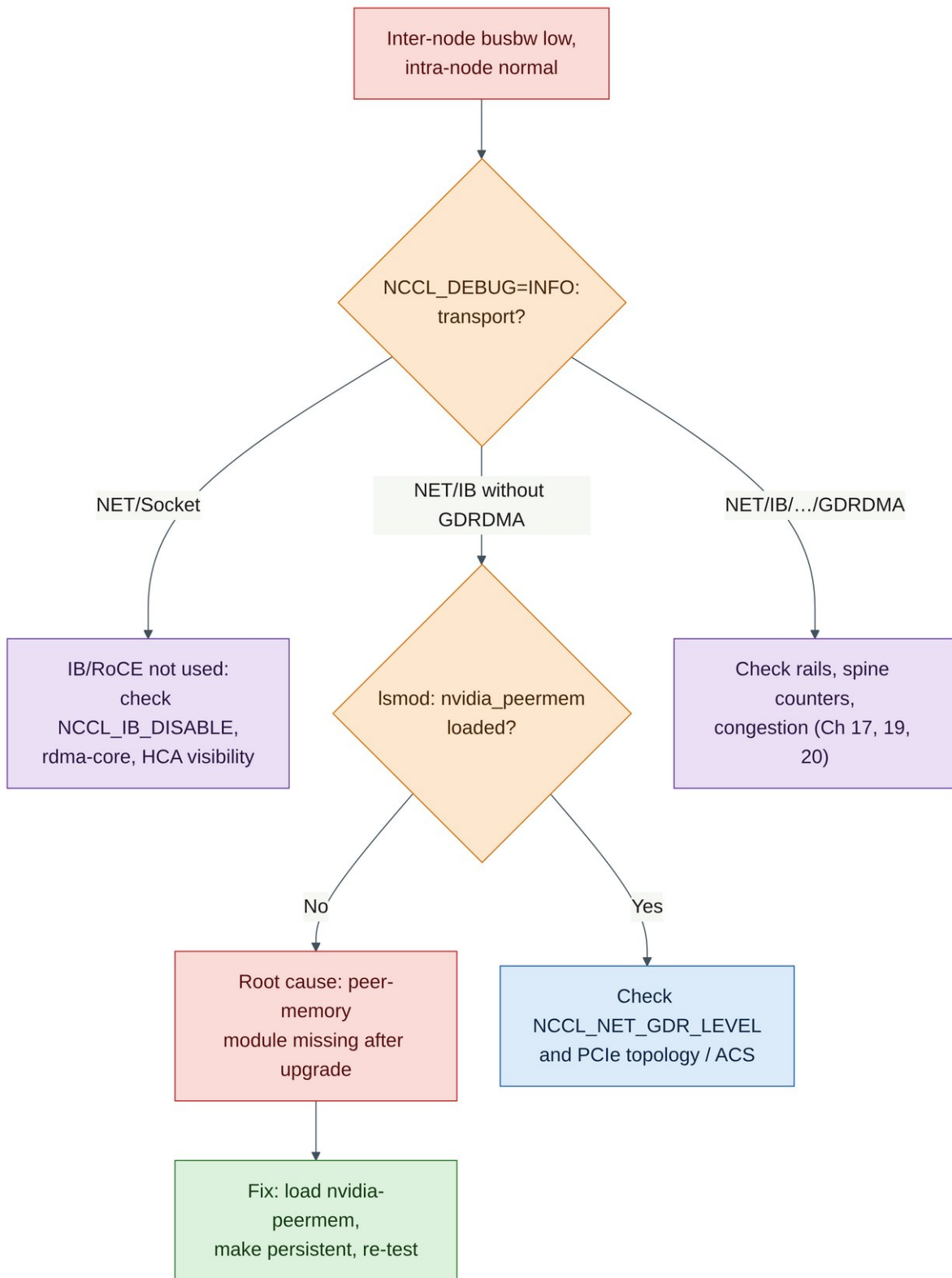


Figure 2.9: Troubleshooting flowchart — slow inter-node collectives

Start from what NCCL reports about its transports before blaming the fabric. A missing GDRDMA tag after a driver change points to the peer-memory module.

Investigation:

1. `NCCL_DEBUG=INFO` shows **via** `NET/IB/0` for inter-node channels, **without** `/GDRDMA`. The fabric is being used, but with host staging.
2. `lsmod | grep peermem` returns nothing. The driver upgrade did not reload the `nvidia-peermem` module.

3. Intra-node NVLink traffic is unaffected, which fits the symptom.

Root cause: Without the peer-memory module, the RDMA stack can't register GPU memory. NCCL falls back to copying through host memory (path ④), adding copies and loading the CPU memory bus.

Fix: `modprobe nvidia-peermem`, make it load at boot (your distribution's module-load mechanism, or the Network Operator/GPU Operator in Kubernetes), and re-run `nccl-tests`. Confirm that `GDRDMA` appears and `busbw` recovers.

Field note: Put a check for `GDRDMA` in the NCCL log into your post-maintenance validation. It is one `grep`, and it catches one of the most common silent performance regressions in GPU clusters.

Chapter quick reference

Item	Key fact
NVLink 4 (H100) / NVLink 5 (B200)	900 GB/s / 1.8 TB/s per GPU, bidirectional
NVL72	72 GPUs in one NVLink domain; 130 TB/s total NVLink bandwidth
GPUDirect RDMA	NIC ↔ GPU memory directly; needs <code>nvidia-peermem</code> (or <code>DMA-BUF</code>) and good PCIe affinity
All-reduce	= reduce-scatter + all-gather; data parallel gradients
All-to-all	MoE expert parallelism; the hardest pattern for fabrics
Ring all-reduce traffic per GPU	$2(n-1)/n \times S$
busbw (all-reduce)	$algbw \times 2(n-1)/n$
Rail-optimized	NIC <i>n</i> of every node → rail leaf <i>n</i> ; collectives one hop
PXN	GPU → NVLink → peer GPU's NIC on the destination's rail
Two-tier fat-tree max endpoints	$k^2/2$ (2,048 at $k = 64$)
<code>ibdiagnet --rail_validation</code>	Checks same-BDF HCAs share a leaf; outputs <code>ibdiagnet2.rails</code>
<code>NCCL_DEBUG=INFO</code>	Shows HCAs, transports, GDRDMA

Chapter FAQ

Q1. Why doesn't NCCL just use all NICs for all traffic, like a big bond?

NCCL does use all NICs at once, but through separate channels, each tied to a GPU–NIC pair with good affinity. Bonding NICs across GPUs would force data across PCIe/CPU paths and break rail alignment. The parallelism comes from many channels, not from link aggregation.

Q2. Is rail optimization only for InfiniBand?

No. The same principle applies to Spectrum-X Ethernet (Helix-B uses rail leaves too). The rail concept belongs to the GPU/NIC layout and NCCL, not to any one fabric technology.

Q3. If NVL72 has 72 GPUs in one NVLink domain, does it still need a scale-out fabric?

Yes, for anything bigger than one rack and for storage and management. NVL72 shrinks the share of traffic that must cross the scale-out fabric but doesn't remove it. NVIDIA's GB200/GB300 SuperPOD reference architectures still include InfiniBand or Spectrum-X compute fabrics.

Q4. What is the difference between NVLS and CollNet?

Both offload reductions into switches. NVLS uses **NVLink SHARP** inside NVSwitch (scale-up). CollNet uses **InfiniBand SHARP** in Quantum switches (scale-out). Chapter 14 covers IB SHARP.

Q5. Should I set `NCCL_CROSS_NIC=0` everywhere?

Not blindly. On a correctly cabled rail-optimized fabric, keeping rings on the same rail is usually best, and NCCL's automatic choice normally handles it. Forcing values is a tuning and diagnostic step. Validate with `nccl-tests` before and after.

Q6. Why is busbw sometimes higher than a single NIC's line rate?

Within a node, NVLink carries part of the traffic, and busbw normalises for the collective's data volume. For multi-node runs, compare busbw with the per-GPU scale-out bandwidth and expect it to be somewhat lower because of protocol overhead. Single-node busbw reflects NVLink speed.

Q7. What does a "hot spine" usually mean in a rail-optimized cluster?

Either legitimate traffic between rails or SUs (all-to-all, pipeline stages across SUs), or something wrong: crossed cables, `NCCL_CROSS_NIC` forcing rings across rails, or jobs placed across SUs without need. Check rail validation and job placement first.

Q8. Does the exam expect NCCL programming?

No. It expects you to understand how NCCL uses the network (objective 1.3) and to use tools that verify interconnect performance (objective 5.3). Knowing the main environment variables and log lines is enough.

Q9. Is PCIe P2P between GPUs ever used on DGX/HGX systems?

Rarely for GPU-to-GPU traffic, because NVLink is available. PCIe carries GPU ↔ NIC traffic (GPUDirect RDMA) and host traffic. PCIe-only servers without NVLink rely on PCIe P2P or host memory.

Q10. Where does job placement fit in?

The scheduler (Slurm or Kubernetes) should place a job's nodes inside as few SUs as possible and keep whole nodes together. Good placement keeps traffic on rail leaves. Poor placement pushes it through spines even on a perfect fabric.

Chapter summary

- **Scale-up** (NVLink/NVSwitch) connects GPUs inside a node or NVL72 rack at up to 1.8 TB/s per GPU. **Scale-out** (IB or Spectrum-X) connects domains at 400 Gb/s per GPU through one NIC per GPU.
- Inter-node GPU traffic should use **GPUDirect RDMA**, which needs an RDMA NIC, **nvidia-peermem** (or DMA-BUF) and good GPU–NIC PCIe affinity. Without it, traffic is staged through host memory.
- Data parallel uses **all-reduce** (or reduce-scatter + all-gather). Tensor parallel stays inside NVLink. Pipeline uses **send/recv**. MoE uses **all-to-all**.
- **NCCL** detects topology, picks transports and builds ring/tree channels. Algorithms: Ring, Tree, CollNet (IB SHARP), NVLS (NVLink SHARP). Protocols: LL, LL128, Simple.
- Ring all-reduce moves $2(n-1)/n \times S$ per GPU. `nccl-tests` **busbw** = **algbw** × $2(n-1)/n$ for all-reduce.
- **Rail-optimized** fabrics keep same-index GPU traffic on one leaf (one hop). **PXN** uses NVLink to hand data to the GPU whose NIC is on the destination's rail.
- A two-tier non-blocking fat-tree supports $k^2/2$ endpoints (2,048 with 64-port switches).
- Validate with `ibdiagnet --rail_validation`, `NCCL_DEBUG=INFO` (look for **GDRDMA**) and `nccl-tests busbw`.

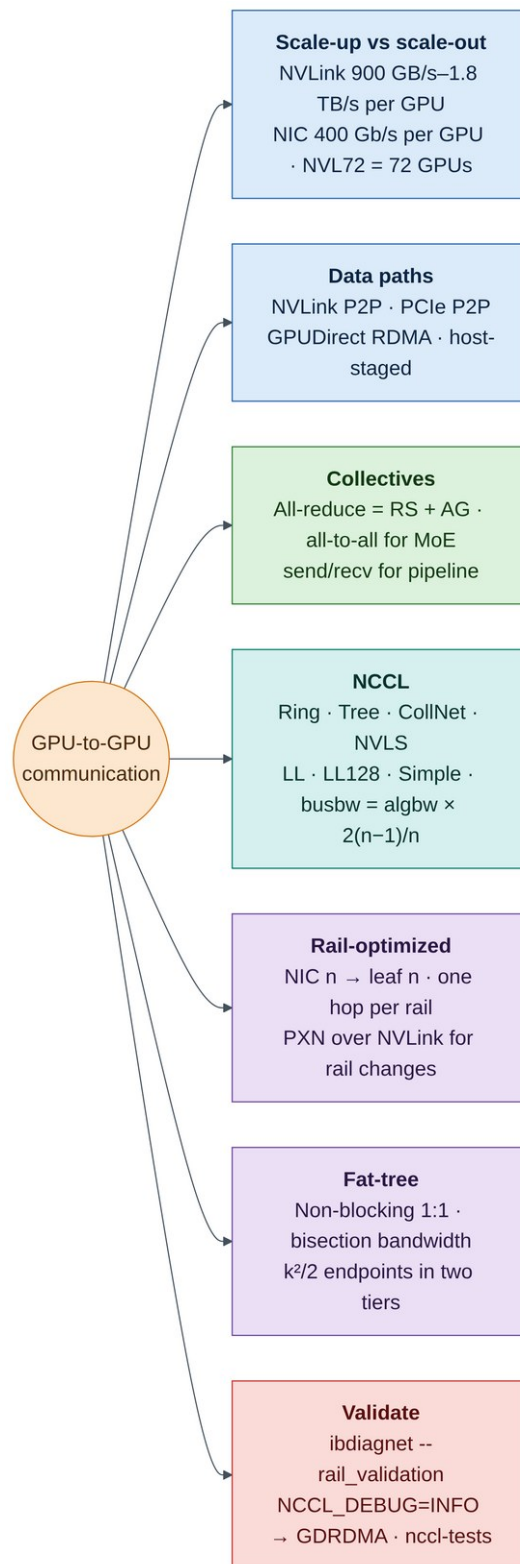


Figure 2.10: Chapter 2 visual recap

The whole chapter on one page.

Further reading

- [REF: NCCL User Guide — Environment Variables (docs.nvidia.com/deeplearning/nccl/)]
- [REF: NVIDIA Technical Blog — "Doubling all2all Performance with NVIDIA Collective Communication Library 2.12" (PXN)]
- [REF: ibdiagnet User Manual — "Rail Optimized Topology Validation"]

- [REF: NVIDIA Technical Blog — "NVIDIA GB200 NVL72 Delivers Trillion-Parameter LLM Training and Real-Time Inference" (NVLink 5 figures)]
- [REF: nccl-tests repository — PERFORMANCE.md (algbw and busbw definitions)]
- [REF: NVIDIA DGX SuperPOD Reference Architecture — Compute fabric and rail-optimized design]

Figure index

Figure	Title	Type	Format
2.1	Scale-up versus scale-out	Architecture	Mermaid flowchart
2.2	GPUDirect RDMA versus host staging	Data path comparison	Mermaid flowchart
2.3	Collective operations at a glance	Concept grid	Mermaid block
2.4	Ring all-reduce across four GPUs	Sequence	Mermaid sequence
2.5	Rail-optimized versus ToR design	Topology comparison	Mermaid flowchart
2.6	PXN: using NVLink to reach the right rail	Packet walk	Mermaid flowchart
2.7	Two-tier fat-tree limits	Topology	Mermaid flowchart
2.8	Lab 2 flow	Flowchart	Mermaid flowchart
2.9	Slow inter-node collectives	Troubleshooting flowchart	Mermaid flowchart
2.10	Chapter visual recap	Mind map (tree)	Mermaid flowchart